

**Открытая олимпиада школьников (информатика)
(№64 Перечня олимпиад школьников, 2025/2026 уч.год)**

Содержание

Задания для 10 и 11 класса	2
Заключительный этап.....	2
Отборочный этап. Первый тур.....	11
Отборочный этап. Второй тур.....	14
Задания для 9 класса	22
Заключительный этап.....	22
Отборочный этап. Первый тур.....	30
Отборочный этап. Второй тур.....	34
Задания для 5–8 класса	40
Заключительный этап.....	40
Отборочный этап. Первый тур.....	40
Отборочный этап. Второй тур.....	54

Задания для 10 и 11 класса

Заключительный этап (приведен один из вариантов заданий)

1. Кодирование информации. Системы счисления (2 балла)

[Два умножения]

Про некоторое положительное вещественное число меньше 1 известно следующее:

1. Если это число умножить на 2_{10}^{100} , результат окажется целым числом.
2. Если это число умножить на 9_{10} и перевести в запись в двоичную систему, окажется, что эта запись числа не содержит значащих нулей.

Сколько существует таких чисел?

В ответе укажите целое число. Если таких чисел не существует или их бесконечно много, укажите в ответе NULL.

Примечание: число 0.1_2 содержит значащий ноль.

Пример ввода ответа: 17

Ответ: 50

Решение

Обратим внимание, что в соответствии с первым условием такое число содержит не более 100 значащих разрядов после запятой. Умножение на 9_{10} — это умножение на 1001_2 . Тогда эту арифметическую операцию можно представить как умножение исходного числа на $8_{10} = 1000_2$ и сложение результата с исходным числом, то есть сдвиг двоичной записи числа на 3 разряда влево и сложение с исходным числом. Тогда, чтобы в результате последней операции (сложения с исходным числом) получились только единичные разряды в двоичной записи, после сдвига всем разрядам исходного числа должны соответствовать противоположные (для «0» — «1», а для «1» — «0») значения разрядов с такими же индексами в сдвинутой записи. А значит, дробная часть исходного числа должна содержать чередование последовательностей из трех единиц и трех нулей.

Эта последовательность должна начинаться сразу после разделительной запятой, например, 0.111_2 , 0.111000111_2 , 0.111000111000111_2 и т.д. или отстоять от неё на 1 или 2 значащих нуля:

- 0.0111_2 , 0.0111000111_2 , 0.00111000111000111_2 и т.д.
- 0.00111_2 , 0.00111000111_2 , 0.00111000111000111_2 и т.д.

Например:

$$0.0111000111_2 \cdot 1001_2 = 11.1000111_2 + 0.0111000111_2 = 11.111111111_2.$$

В любом другом случае после сдвига на 3 разряда влево окажется хотя бы один индекс разряда, для которого и в сдвинутом, и в исходном числе окажутся одинаковые значения. Например, число 0.000111_2 после умножения на 1001_2 даст результат 0.111111_2 , который будет содержать только единицы в дробной части, но будет содержать значащий ноль в целой части, а значит, не подходит под условие.

Следовательно, нужно посчитать количество чисел, содержащих не более 100 разрядов после запятой в двоичной записи, таких, чтобы эти разряды образовывали последовательность с чередованием трех единиц и трех нулей и не имели либо имели 1 или 2 значащих нуля между разделителем и этой последовательностью. Легко подсчитать, что таких последовательностей может быть ровно $17 + 17 + 16 = 50$, что и будет ответом на задание.

2. Кодирование информации. Объём информации (1 балл)

[Соты]

Петя пишет систему управления роботом, который перемещается по плоскости с гексагональной разметкой — правильными шестиугольниками, примыкающими друг к другу одним из ребер. Каждая команда программы перемещает робота на смежный шестиугольник в одном из шести возможных направлений. Петя решил записывать программу в память робота как последовательность кодов команд, используя для записи каждой команды минимально возможное, одинаковое для всех команд количество бит. Петя посчитал, что тогда в сегмент памяти, который у него есть, можно поместить программу длиной ровно 4096 команд.

Вася обратил внимание на такую особенность робота Пети — любая его программа всегда имеет количество команд, кратное 128. Он предложил рассматривать программу робота как последовательность подпрограмм, длиной 128 команд, кодируя каждую такую подпрограмму минимально возможным, одинаковым для всех возможных подпрограмм количеством бит.

Определите максимальную длину программы (кратную 128), которую можно записать в сегмент памяти, кодируя по методу Васи. В ответе укажите целое число.

Пример ввода ответа: 512

Ответ: 4736

Решение

Обозначим максимальную длину программы, помещающуюся в память при способе кодирования Пети, за A , а длину подпрограммы при способе кодирования Васи за B .

Поскольку количество различных направлений (и различных команд) равно 6, для хранения одной команды понадобится $\lceil \log_2 6 \rceil = 3$ бит. Тогда программа, закодированная способом Пети, будет занимать $3A$ бит, и это будет размером имеющегося сегмента памяти.

У Васи всего будет 6^B вариантов различных подпрограмм из B команд, и одна подпрограмма займёт в памяти $\lceil \log_2 6^B \rceil = \lceil B \log_2 6 \rceil$ бит.

Тогда всего Вася сможет записать в имеющийся сегмент памяти максимум $\left\lfloor \frac{3A}{\lceil B \log_2 6 \rceil} \right\rfloor$ подпрограмм длины B , и длина такой программы будет равна $\left\lfloor \frac{3A}{\lceil B \log_2 6 \rceil} \right\rfloor \cdot B$ командам.

Например, если $A = 4096$ и $B = 128$, то:

$$\left\lfloor \frac{3A}{\lceil B \log_2 6 \rceil} \right\rfloor \cdot B = \left\lfloor \frac{3 \cdot 4096}{\lceil 128 \log_2 6 \rceil} \right\rfloor \cdot 128 = \left\lfloor \frac{12288}{330} \right\rfloor \cdot 128 = 4736.$$

3. Основы логики (2 балла)

[Переусложнили...]

Обозначим за X некоторое натуральное число, записанное с помощью $n + 1$ бит, т.е. $X = (x_n x_{n-1} \dots x_0)_2$. Например, если $n = 3$ и $X = 0101_2$, то $x_3 = 0, x_2 = 1, x_1 = 0, x_0 = 1$.

Даны логические функции $A(X)$ и $B(X)$:

$$A(X) = \bigwedge_{i=0}^{n-1} (x_i \rightarrow x_{i+1}) \wedge (x_n \rightarrow x_0)$$

$$B(X) = \bigvee_{i=0}^{\lfloor \frac{n-1}{2} \rfloor} \neg \left(M(x_i, x_{i+\lfloor \frac{n+1}{2} \rfloor}, 1) \equiv M(x_i, x_{i+\lfloor \frac{n+1}{2} \rfloor}, 0) \right)$$

Здесь $\lfloor a \rfloor$ — значение a с округлением вниз, $\lceil a \rceil$ — значение a с округлением вверх.

Функция $M(a, b, c)$ задана таблицей истинности:

a	b	c	$M(a, b, c)$
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Сколько существует чисел X таких, что $A(X) = B(X)$, если $n = 19$? В ответе введите целое положительное число.

Примечание: битовая последовательность может начинаться с нуля.

Пример ввода ответа: 17

Ответ: 1022

Решение

Рассмотрим функцию $A(X)$. Она представляет собой конъюнкцию импликаций и равна 1 только в том случае, когда все импликации равны истине. Это значит, что не должно встречаться импликаций вида $1 \rightarrow 0$, и в битовой последовательности X не должно быть последовательности 10. Последний член конъюнкции дополнительно накладывает ограничение, что не должно быть ситуации, когда $x_n = 1$ и $x_0 = 0$. То есть если $A(X) = 1$, то последовательность равна либо 000...000, либо 111...111, где общее количество цифр равно $n + 1$. В противном случае $A(X) = 0$.

Рассмотрим функцию $B(X)$. Используемая в ней функция $M(a, b, c)$ — функция большинства. Из неё видно, что $M(x_i, x_{i+\lfloor \frac{n+1}{2} \rfloor}, 1) = x_i \vee x_{i+\lfloor \frac{n+1}{2} \rfloor}$, а $M(x_i, x_{i+\lfloor \frac{n+1}{2} \rfloor}, 0) = x_i \wedge x_{i+\lfloor \frac{n+1}{2} \rfloor}$. Заменяв отрицание эквиваленции на исключающее "ИЛИ", получим:

$$B(X) = \bigvee_{i=0}^{\lfloor \frac{n-1}{2} \rfloor} \left((x_i \vee x_{i+\lfloor \frac{n+1}{2} \rfloor}) \oplus (x_i \wedge x_{i+\lfloor \frac{n+1}{2} \rfloor}) \right) = \bigvee_{i=0}^{\lfloor \frac{n-1}{2} \rfloor} (x_i \oplus x_{i+\lfloor \frac{n+1}{2} \rfloor})$$

Из этой формулы видно, что функция $B(X)$ работает с "половинами" битовой последовательности и равна 0 только тогда, когда "половины" битовой последовательности совпадают (например, 110110).

Если $A(X) = B(X) = 1$, то последовательность равна либо 000...000, либо 111...111, но при этом имеет разные "половины". Здесь мы получаем противоречие, и значит, что вариант с $A(X) = B(X) = 1$ не подходит, и эти две последовательности также не подходят.

Далее рассмотрим два случая для разной чётности значения n при условии $A(X) = B(X) = 0$.

Если n нечётно, то длина последовательности чётна, и сама последовательность делится ровно на две части. Первая половина последовательности ($\frac{n+1}{2}$ элементов) задаётся произвольно, а вторая половина равна первой. Тогда

существует $2^{\frac{n+1}{2}}$ вариантов этой последовательности. В них входят две последовательности, описанные выше, они нам не подходят. Тогда в итоге будет $2^{\frac{n+1}{2}} - 2$ варианта последовательности.

Если n чётно, то длина последовательности нечётна, и центральный элемент не входит ни в первую половину, ни во вторую. Первая половина последовательности ($\frac{n}{2}$ элементов) задаётся произвольно, а вторая половина равна первой. При этом центральный элемент также произволен. Тогда существует $2^{\frac{n}{2}+1}$ вариантов этой последовательности. В них входят две последовательности, описанные выше, они нам не подходят. Тогда в итоге будет $2^{\frac{n}{2}+1} - 2$ варианта последовательности.

4. Кодирование информации. Алгоритмы обработки кодированной информации (1 балл)

[Взломщик поневоле]

Вася изучает алгоритмы шифрования и решил придумать свой собственный алгоритм. Он решил, что его алгоритм будет рассчитан на шифрование слов из букв современного латинского алфавита. Для каждой буквы в тексте Вася применяет следующие шаги:

1. Выяснить номер этой буквы в алфавите (Вася пронумеровал буквы от 1 до 26).
2. Выяснить номер в алфавите i -й буквы ключа.
3. Перемножить два данных числа — это и есть Васин код для буквы текста.
4. Увеличить i на 1 и взять по модулю длины ключа.

Исходно $i = 0$, а нумерация букв в ключе и в тексте для шифрования начинается с нуля.

Чтобы проверить свой алгоритм, Вася взял фразу «the quick brown fox jumps over the lazy dog», убрал из неё пробелы, повторил её 15 раз подряд и зашифровал полученный текст.

В результате он получил следующую последовательность чисел (см. [файл по ссылке](#)). Потом Вася решил проверить, можно ли расшифровать результат и понял, что потерял ключ. Единственное, что он помнит: все символы в ключе были различными. Помогите Васе выяснить, каким был ключ. В ответе введите последовательность строчных букв латинского алфавита без пробелов. Если же восстановить ключ невозможно, укажите в качестве ответа NULL.

Пример ввода ответа: abcdef

Ответ: skjhbweryugqfptaczd

Решение

Основная идея задачи: взять получившуюся последовательность чисел (из файла), перевести кодовую фразу в набор чисел и разделить числа из последовательности на числа, полученные из кодовой фразы. Так как буквы в ключе различны, вычисление останавливается в тот момент, когда появляется уже имеющаяся в ключе буква.

Проще всего решить эту задачу программно:

```
from itertools import cycle
```

```
with open('file.txt') as file:
    encoded = [int(x) for x in file.read().split()]

print(encoded)

phrase = [ord(ch) - ord('a') + 1 for ch in 'thequickbrownfoxjumpsoverthelazydog'] * 15
print(phrase)

key = []

for encoded_char, phrase_char in zip(encoded, cycle(phrase)):
    key_char = encoded_char // phrase_char

    if key and key[0] == key_char:
        print('Found a key')
        break

    key.append(key_char)
    print(encoded_char, phrase_char, key_char, sep='\t')

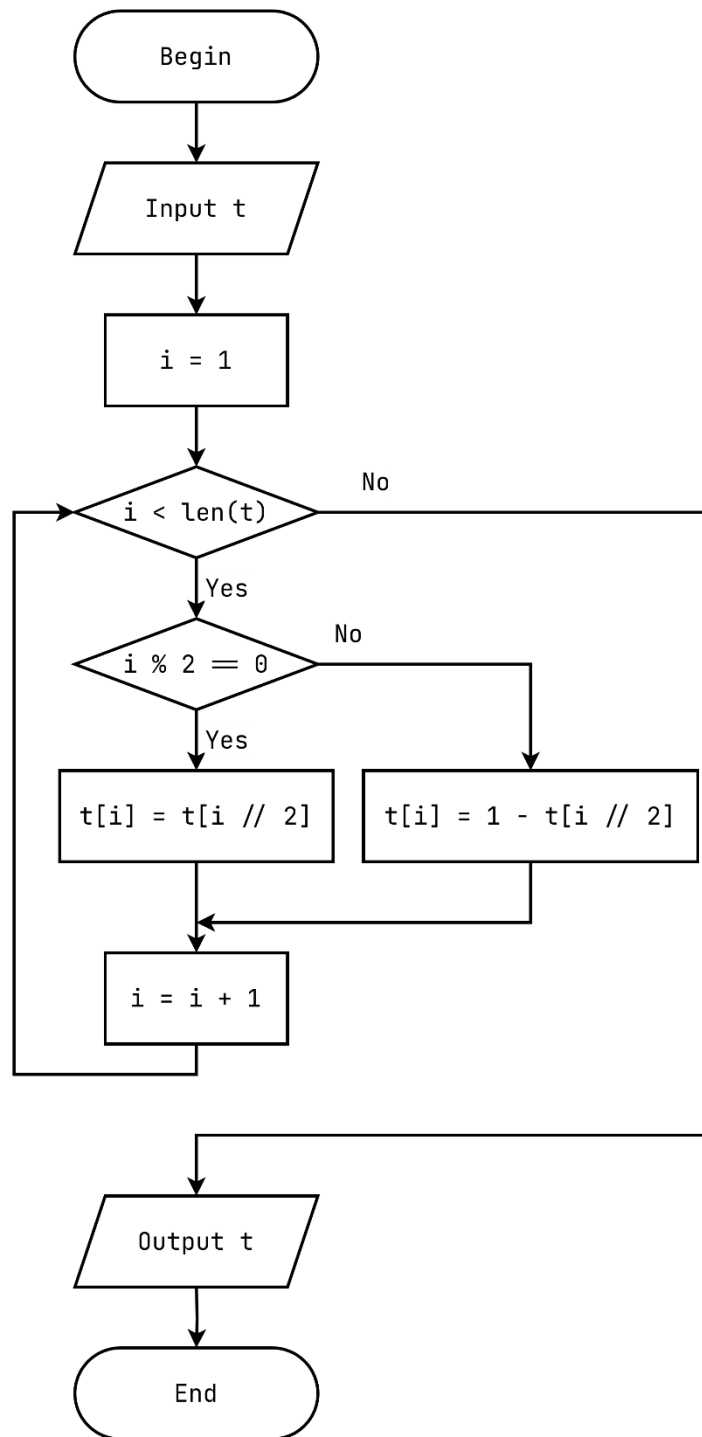
print(''.join([chr(k + ord('a') - 1) for k in key]))
```

Для варианта выше программа выдаёт ответ skjhbweryugqfptaczd.

5. Алгоритмизация и программирование. Анализ алгоритма, заданного в виде блок-схемы (2 балла)

[Спуска вечность]

Аксель любит строить разные последовательности, и вчера ему пришёл в голову алгоритм, который показан ниже в виде блок-схемы:



В качестве t Аксель вводит массив, содержащий битовую последовательность из 2^{32} нулей. Нумерация элементов массива начинается с нуля.

Определите, какая последовательность из 8 бит будет находиться, начиная с индекса 4294967124 (4294967124, 4294967125, ..., 4294967131). В ответ введите последовательность бит в порядке возрастания их индексов в последовательности без пробелов.

Пример ввода ответа: 01010101

Ответ: 10011001

Решение

Анализ алгоритма (это алгоритм генерации **последовательности Туэ-Морса**) показывает, что для вычисления элемента с индексом i не нужно вычислять все элементы до него. На самом деле, достаточно переписать этот алгоритм в виде рекурсивного:

```

def solve(i):
    if i == 0:
        return 0
    elif i % 2 == 0:
        return solve(i // 2)
  
```

```
else:
    return 1 - solve(i // 2)
```

```
start = 4294967124
for s in range(start, start+8):
    print(solve(s))
```

Значение, возвращаемое функцией solve в базовом случае (при $i = 0$), меняется в зависимости от того, чем исходно заполнен массив t (0, если нулями, и 1, если единицами).

Если массив исходно заполнен нулями, а искомый фрагмент битовой последовательности начинается с индекса 4294967124, программа выдаёт ответ 10011001.

6. Телекоммуникационные технологии (3 балла)

[Управление перегрузкой]

Основная задача протокола TCP – обеспечение надежной доставки данных. Одна из возможных проблем – потеря сообщений на промежуточных устройствах в силу ошибок или переполнения памяти. Для предотвращения таких потерь используется **управление перегрузкой в TCP**.

В управлении перегрузкой отправитель регулирует скорость передачи данных, постепенно увеличивая ее (мы все же хотим передавать данные как можно быстрее) и уменьшая скорость передачи при возникновении ошибки. Существует множество алгоритмов управления перегрузкой. Одни из них – Tahoe. В нем используются следующие понятия:

- **RTT (Round-Trip Time)** — номинальное время полного пути пакета, то есть интервал между отправкой сегмента TCP и получением соответствующего ACK (подтверждения). Учтите, что это эталонное значение. В течение времени, равного RTT, отправитель может отправить большое количество пакетов, не дожидаясь ACK.
- **cwnd (Congestion Window)** — окно перегрузки, переменная на стороне отправителя, определяющая максимальное количество неподтверждённых сегментов, которое можно отправить в сеть без ACK. Этим окном как раз ограничивается скорость передачи. Чем больше окно, тем быстрее (интенсивнее) отправляются TCP-сегменты.
- **MSS (Maximum Segment Size)** — максимальный размер полезной нагрузки TCP-сегмента (обычно 1460 байт для Ethernet); используется как единица для роста cwnd.
- **ssthresh (Slow Start Threshold)** — пороговое значение окна перегрузки; разделяет фазы медленного старта (см. далее) и предотвращения перегрузки. В Tahoe изначально ssthresh не задаётся фиксированно (принимается равным бесконечности), а устанавливается динамически при первой потере. Будем считать (несколько упрощая), что при первой потере $ssthresh = cwnd/2$.
- **Slow Start** — фаза управления перегрузкой, где cwnd удваивается каждый RTT (при начальном значении, равном 1 MSS), чтобы быстро «нащупать» реальную пропускную способность сети.
- **Congestion Avoidance** — фаза, наступающая после достижения ssthresh. В этой фазе cwnd растёт линейно (+1 MSS за RTT), чтобы стабилизировать передачу без перегрузки.

Сначала скорость передачи растёт согласно Slow Start, а как случается первая потеря, рассчитывается ssthresh, после чего cwnd сбрасывается в 1 MSS, заново запускается Slow Start (cwnd удваивается каждый RTT), но уже до установленного ssthresh. Достигнув ssthresh, алгоритм переходит в режим Congestion Avoidance, при котором рост cwnd + 1 MSS за RTT, пока не произойдет новая потеря, после чего ssthresh уменьшается вдвое снова. Это создаёт «зубчатую» кривую cwnd.

Например, если потеря случилась при $cwnd=256$ MSS, то $ssthresh = cwnd/2=128$ MSS, cwnd сбрасывается в 1 MSS, возврат в Slow Start — cwnd удваивается каждый RTT до нового ssthresh (128 MSS). Достигнув ssthresh, переход в Congestion Avoidance — рост cwnd +1 MSS за RTT, пока не произойдет новая потеря.

Пусть передается файл из 64 сегментов (по 1460 байт) передаётся по сети с $RTT = 120$ мс. Используется алгоритм TCP Tahoe.

- Slow Start: cwnd удваивается каждый RTT, начиная с 1 MSS.
- После потери: $ssthresh = cwnd / 2$, $cwnd = 1$ MSS.
- Достигнув ssthresh, переход к Congestion Avoidance (+1 MSS/RTT).
- Потеря происходит при $cwnd = 16$ MSS.

Найдите общее время передачи всех 64 сегментов в миллисекундах. Выберите наиболее близкий к полученному значению вариант ответа:

- 590
- 840
- 1150
- 1430
- 1710
- 1990
- 2160
- 2270
- 2400
- 2550

Ответ: 1430

Решение

В этой задаче требуется аккуратно реализовать алгоритм, описанный в условии. Это можно сделать или вручную (что будет достаточно трудоёмко) или создать имитационную программную модель, например, такую:

```
SEGMENTS_COUNT = 64
RTT = 120
MAX_CWND = 16

cwnd = 1
sssthresh = None
segments_transferred = 0

rounds = 0

while segments_transferred < SEGMENTS_COUNT:
    if cwnd == MAX_CWND:
        sssthresh = cwnd // 2
        cwnd = 1

    segments_transferred += cwnd
    rounds += 1

    print(rounds, cwnd, segments_transferred, sssthresh, sep='\t')

    if sssthresh is None or cwnd < sssthresh:
        cwnd *= 2
    else:
        cwnd += 1

print(rounds * RTT)
```

Программа выдаёт 1440 миллисекунд, и ближайший ответ из имеющихся – 1430.

7. Технологии сортировки и фильтрации данных (3 балла)

[Расширяя круг общения]

Для анализа социальных сетей часто применяют понятие **социального графа**. В социальном графе пользователи являются вершинами, а каждое ребро показывает, что пользователи добавили друг друга в список друзей.

Исследователь собрал социальный граф для некоторых пользователей социальной сети Y. Этот граф хранится в виде одной таблицы friends со следующими полями:

- first_user_id — целое положительное число, идентификатор первого пользователя;
- second_user_id — целое положительное число, идентификатор второго пользователя.

При этом пара полей (first_user_id, second_user_id) является первичным ключом.

Каждая такая пара для удобства хранится дважды. Например, если пользователи с ID 1 и 2 добавили друг друга в список друзей, таблица будет содержать две записи:

first user id	second user id
1	2
2	1

Исследователь собирает данные аккуратно и даёт несколько гарантий:

- Дружба между двумя пользователями всегда хранится двумя строками;
- Нет ситуаций, когда пользователь добавил в список друзей самого себя.

Два различных пользователя знают друг друга **через одно рукопожатие**, если существует третий пользователь, отличающийся от них и являющийся им другом, но при этом сами пользователи не являются друзьями. Иначе говоря, в социальном графе между этими пользователями нет ребра, но есть путь через одну вершину. Например, если пользователи с ID 1 и 3 не добавляли друг друга в список друзей, но пользователь с ID 2 имеет в списке друзей этих пользователей, то пользователи с ID 1 и 3 знают друг друга через одно рукопожатие.

Определите, сколько различных пользователей знает пользователь с ID 10 через одно рукопожатие. Обратите внимание, что самого пользователя и его прямых друзей считать не нужно. В ответе введите одно целое неотрицательное число. Если пользователь с указанным ID не встречается в таблице, введите 0.

Примечание: файл с базой данных доступен [на сайте](#).

Пример ввода ответа: 17

Ответ: 5

Решение

Самый простой способ: найти всех пользователей, которые являются друзьями друзей нужного пользователя, и вычеркнуть из этого списка лишних (самого пользователя и его непосредственных друзей).

Получим друзей пользователя с ID 5:

```
Select second_user_id From friends
Where first_user_id = 10;
```

Получим список: 9, 11, 181, 182, 183.

Дальше напишем запрос, который найдёт всех друзей пользователей из списка выше, не включая исходного пользователя:

```
Select friends.second_user_id From friends  
Where first_user_id In (9, 11, 181, 182, 183) And second_user_id <> 10;
```

Получим список из 5 элементов: 8, 12, 184, 85, 186.

Теперь необходимо убрать из ответа пересечение двух списков. В этом варианте пересечение пустое, но в других вариантах может встретиться несколько таких пользователей, которые влияют на ответ. Это относится, например, к пользователю с ID 5.

В итоге в этом варианте ответом будет 5.

8. Технологии программирования (3 балла)

[Новый язык]

Имя входного файла	стандартный ввод
Имя выходного файла	стандартный вывод
Ограничение по времени	1,5 секунды
Ограничение по памяти	256 МБ

Глеб решил изучить новый для себя язык программирования. Так как C++ показался ему слишком простым, он выбрал язык Bassembly. Язык этот пока молодой, поэтому найти удалось только его документацию.

Доступные регистры на данном языке: `eax`, `ecx`, `edx`, `esi` и `edi`.

Каждый регистр хранит **беззнаковое 32-битное целое число**. Все арифметические операции над регистрами выполняются по модулю: при переполнении старшие биты отбрасываются. Например, число эквивалентно 0, а число -1 эквивалентно .

Bassembly поддерживает пять команд:

`add, sub, mul, inc, print`

Обозначим через `reg`, `reg1`, `reg2`, `reg3` произвольные регистры из списка выше, а через `const` — неотрицательное целое число от 0 до $2^{32}-1$.

Команда `add`

Команда `add` может иметь несколько форм.

- `add 0 const` — выполнить операцию `eax += const`.
- `add 1 reg` — выполнить операцию `eax += reg`.
- `add 2 reg1 reg2` — выполнить операцию `reg1 += reg2`.

Команда `sub`

Команда `sub` также может иметь несколько форм.

- `sub 0 const` — выполнить операцию `eax -= const`.
- `sub 1 reg` — выполнить операцию `eax -= reg`.
- `sub 2 reg1 reg2` — выполнить операцию `reg1 -= reg2`.

Команда `mul`

Команда `mul` имеет две формы.

- `mul 1 reg1` — вычислить произведение `eax * reg1` и сохранить его в пару регистров `[ecx:eax]`.
- `mul 2 reg1 reg2` — вычислить произведение `reg1 * reg2` и сохранить его в пару регистров `[ecx:eax]`.

Здесь запись `[ecx:eax]` обозначает 64-битное число, у которого старшие 32 бита записываются в `ecx`, а младшие 32 бита — в `eax`.

Команда `inc`

Команда `inc reg` выполняет операцию `reg += 1`.

Команда `print`

Команда `print reg` выводит текущее значение регистра `reg`.

Все регистры в начале выполнения программы равны 0.

Но Глеб ещё только учится писать программы на этом языке, поэтому иногда допускает опечатки.

Если название команды в строке записано неверно, то строка считается ошибочной. Если же название команды записано верно, то гарантируется, что все остальные элементы этой строки корректны, кроме, возможно, названий регистров. Таким образом, при корректно записанной команде опечатки могут встречаться только в названиях регистров.

Программа обрабатывается построчно сверху вниз. Если в некоторой строке обнаруживается опечатка, то все предыдущие корректные строки считаются уже выполненными. В частности, должны быть выведены все значения, которые были напечатаны командами `print` в предыдущих строках.

После этого необходимо вывести номер первой ошибочной строки и завершить выполнение.

Если ошибочных строк в программе нет, требуется выполнить все команды по порядку и вывести все значения, которые будут напечатаны командами `print`.

Формат входных данных

В первой строке дано одно число n — количество команд ($1 \leq n \leq 10^5$).

В следующих n строках содержится описание программы, по одной строке на каждую команду.

Строки программы нумеруются от 1 до n в порядке их следования во входных данных.

Формат выходных данных

Если в программе встречается ошибочная строка, необходимо вывести все значения, которые были напечатаны командами `print` до первой ошибки, по одному в строке. После этого в отдельной строке необходимо вывести номер первой ошибочной строки.

Если ошибочных строк в программе нет, требуется вывести все значения, которые будут напечатаны командами `print`, по одному в строке.

Примеры

Стандартный ввод	Стандартный вывод
5 add 0 1000000000 mul 2 eax eax print eax print ecx mal 2 eax eax	2808348672 232830643 5
7 add 0 10 sub 2 ecx eax inc ecx print ecx sub 2 ecx ecx print ecx print eax	4294967287 0 10

Решение

Это задача на реализацию. Основными идейными сложностями являются корректное выполнение операций по модулю и обработка некорректных программ.

Различие между задачами состояло в функциях, необходимых для реализации, и количестве аргументов у функций `add` и `sub`.

Основной идеей было исполнение и проверка на корректность построчно. Если мы в какой-то момент встречаем некорректную строку, то после неё сразу завершаемся (делая `exit(0)`).

При написании на языке C++ можно использовать тип данных `unsigned int`, тогда все операции, кроме умножения, будут эквивалентны обычным операциям с этим типом данных. При написании на языке Python необходимо после каждой операции брать остаток по модулю 2^{32} .

Для умножения можно было использовать тип данных `unsigned long long` в C++. Тогда после перемножения двух чисел, старшую и младшую часть можно распределить взятием по модулю и делением на 2^{32} . Для Python необходимо было просто перемножить два числа и правильно их записать в регистры.

При встрече команды `print` необходимо было сразу выводить значение регистра, который был указан как аргумент. Тогда при встрече некорректной строки все `print` выведут свои значения, что и требуется по условию.

Теперь рассмотрим, как можно было обрабатывать некорректные строки. Будем считывать в следующем порядке:

1. Считываем первое слово (в Python считаем строку, разделим её и посмотрим на первое слово), если слово не является одной из доступных команд, то выводим номер строки и завершаемся.
2. Если же название команды корректно, то гарантируется корректность количества аргументов. Тогда необходимо проверить только имена регистров. Для этого можно было создать `std::set` с допустимыми регистрами и проверять, есть ли в нём такое имя.

Также для уменьшения количества кода каждую функцию можно было выразить через себя же с большим количеством аргументов, тогда проверка регистров была бы только в функции с максимальным количеством аргументов.

Так как при корректности имени команды некорректным могло быть только имя регистра, то данный подход рассматривает все возможные случаи.

Тогда итоговое время работы будет $O(n)$ и затраченная память $O(1)$.

9. Технологии программирования (4 балла)

[Лазерный лабиринт]

Имя входного файла	стандартный ввод
Имя выходного файла	стандартный вывод
Ограничение по времени	2 секунды
Ограничение по памяти	256 МБ

Андрею на день рождения подарили две очень интересные вещи: лабиринт и лазер.

Лабиринт представляет собой матрицу из n строк и m столбцов. В матрице могут встречаться пустые клетки `.`, стены `#`, а также два типа зеркал: `/` и `\`.

После этого Андрей начинает запускать лазер из различных точек лабиринта. Когда луч оказывается в некоторой клетке, происходит следующее:

- если луч находится в пустой клетке, то он продолжает двигаться в том же направлении;
- если луч находится в клетке с зеркалом /, то направление луча изменяется следующим образом:
 - вверх → вправо;
 - вправо → вверх;
 - вниз → влево;
 - влево → вниз;
- если луч находится в клетке с зеркалом \, то направление луча изменяется следующим образом:
 - вверх → влево;
 - влево → вверх;
 - вниз → вправо;
 - вправо → вниз.

После этого луч пытается перейти в соседнюю клетку в новом направлении.

Если луч пытается выйти за границы лабиринта или перейти в клетку со стеной, его движение немедленно заканчивается.

Вам необходимо ответить на q запросов. В каждом запросе заданы клетка (x, y) и начальное направление луча. Для каждого запроса требуется определить:

- количество переходов между соседними клетками, которое совершит луч до остановки;
- либо вывести -1, если луч будет двигаться бесконечно, то есть попадёт в цикл.

Формат входных данных

В первой строке дано два целых числа n и m — количество строк и столбцов в матрице соответственно ($1 \leq n, m \leq 500$).

В следующих n строках дано по m символов — описание соответствующей строки матрицы. Гарантируется, что каждая клетка содержит один из символов ., #, / или \.

В следующей строке дано одно целое число q — количество запросов ($1 \leq q \leq 10^5$).

В следующих q строках дано описание запросов, по одному в строке.

Каждый запрос имеет вид $x\ y\ dir$ — стартовая позиция луча и его начальное направление ($1 \leq x \leq n, 1 \leq y \leq m, dir \in \{left, right, up, down\}$).

Направления left, right, up и down соответствуют движению влево, вправо, вверх и вниз соответственно.

Гарантируется, что клетка (x, y) не является стеной.

Формат выходных данных

Для каждого запроса выведите одно целое число в отдельной строке.

Если луч в некоторый момент остановится, выведите количество переходов между соседними клетками, которое он совершит. Если же луч будет двигаться бесконечно, выведите -1.

Обратите внимание, что если луч несколько раз посещает одну и ту же клетку, но с разными направлениями, то это считаются разными состояниями.

Примеры

Стандартный ввод	Стандартный вывод
3 3 /.\ ... \./ 4 1 2 left 1 2 right 1 2 up 1 2 down	-1 -1 1 3
3 3/. #\/ 5 1 1 down 2 3 left 2 3 right 2 3 down 2 1 right	2 6 1 5 3

Решение

Заметим, что отличие между задачами состояло только в типах зеркала в лабиринте. В коде это отличие отображается в обработке зеркала, что не влияет на сложность.

Одной из сложностей задачи является возможность попасть в цикл.

Основной идеей в данной задаче было dfs с сохранением состояний. Состоянием является тройка (x, y, dir) , где dir — одно из четырёх направлений.

Из каждого состояния есть ровно один исход:

- После отражения и шага мы попадаем в новую клетку, тогда движение продолжается;
- Мы находимся в стене или за границами поля, тогда движение луча завершается.

Сначала покажем, как обрабатывать циклы.

Давайте для состояний заведём массив $dp[n][m][4]$, в котором будем хранить ответ для соответствующего запроса. Тогда при входе в состояние будем пометать, что текущее состояние находится в обработке. Если в какой-то момент мы придём в такое состояние, то это означает, что мы попали в цикл. Тогда все состояния, что мы обошли ранее в текущем обходе, так же попадут в цикл и будут иметь ответ -1 . Это так же можно сохранить в массиве dp .

Покажем теперь, как обрабатывать запрос, когда мы не попадаем в цикл.

В dp при попадании в стену давайте сохраним ответ как 0 , а при выходе за границу сразу же будем возвращать 0 .

Тогда $dp[x][y][dir]$ будет равно $dp[x'][y'][dir'] + 1$, где x' — новая позиция по x , y' — новая позиция по y , dir' — новое направление.

Итого у нас есть 4 различных состояния в массиве dp :

- Ещё не начали обработку, тогда при заходе в такое состояние мы рекурсивно считаем для него ответ;
- Состояние находится в обработке, тогда при заходе в него мы запоминаем, что ответом будет -1 ;
- Состояние уже было посчитано и ответ в нём -1 , тогда мы просто возвращаем -1 , так как мы попадём в цикл;
- Состояние уже было посчитано и ответ в нём не -1 , тогда мы так же возвращаем значение в текущем состоянии, так как луч завершит движение.

У нас получился обход dfs , который сохраняет ответ для своего состояния, что работает быстро, а именно за $O(nm + q)$ по времени и $O(nm)$ по памяти, так как каждое состояние будет обработано не более 1 раза.

Отборочный этап. Первый тур (приведен один из вариантов заданий)

1. Кодирование информации. Системы счисления (1 балл)

[Циклические сдвиги]

Запись исходного числа в шестнадцатеричной системе счисления содержит ровно 200 цифр, причем в ней встречаются только две различные шестнадцатеричные цифры.

Определена следующая последовательность операций:

3. Полученное на предыдущей итерации число переводится в двоичную систему счисления.
4. Получившаяся запись числа циклически сдвигается вправо на один разряд (младший разряд исходного числа становится старшим разрядом нового числа).
5. Новое число переводится в шестнадцатеричную систему счисления.

Если после какой-то операции появляются ведущие нули, они сохраняются.

Указанную последовательность операций последовательно применили 7 раз и обнаружили, что все получающиеся шестнадцатеричные числа начинаются с цифр 2, 4, 5 или 9. Какие две цифры встречались в записи исходного числа? В ответе укажите две шестнадцатеричные цифры в порядке возрастания без разделителей.

Пример записи ответа: 0A

Ответ: 4A

2. Кодирование информации. Системы счисления (3 балла)

[Как можно меньше]

Положительное вещественное число меньше 1 при записи в восьмеричной системе счисления имеет 999 значимых разрядов после запятой, при этом используются только три цифры: 1, 3 и 4. Каждая цифра встречается минимум один раз, при этом порядок цифр неизвестен. Какая минимальная сумма цифр может быть у записи этого же числа в шестнадцатеричной системе счисления? В ответе укажите целое число в десятичной системе счисления.

Пример записи ответа: 171717

Ответ: 1501

3. Кодирование информации. Количество информации (2 балла)

[Хорошие и плохие пароли]

Петя написал генератор паролей длиной 8 символов. Каждый символ с равной вероятностью может быть заглавной или строчной латинской буквой, десятичной арабской цифрой или одним спецсимволом из набора `_#`

Вася сказал, что **хороший** пароль обязательно должен содержать в себе как минимум одну заглавную и одну строчную латинскую букву, одну арабскую цифру и один спецсимвол. Сколько бит информации несёт сообщение, что сгенерированный пароль является хорошим? Ответ округлите до двух знаков после запятой **в меньшую сторону**.

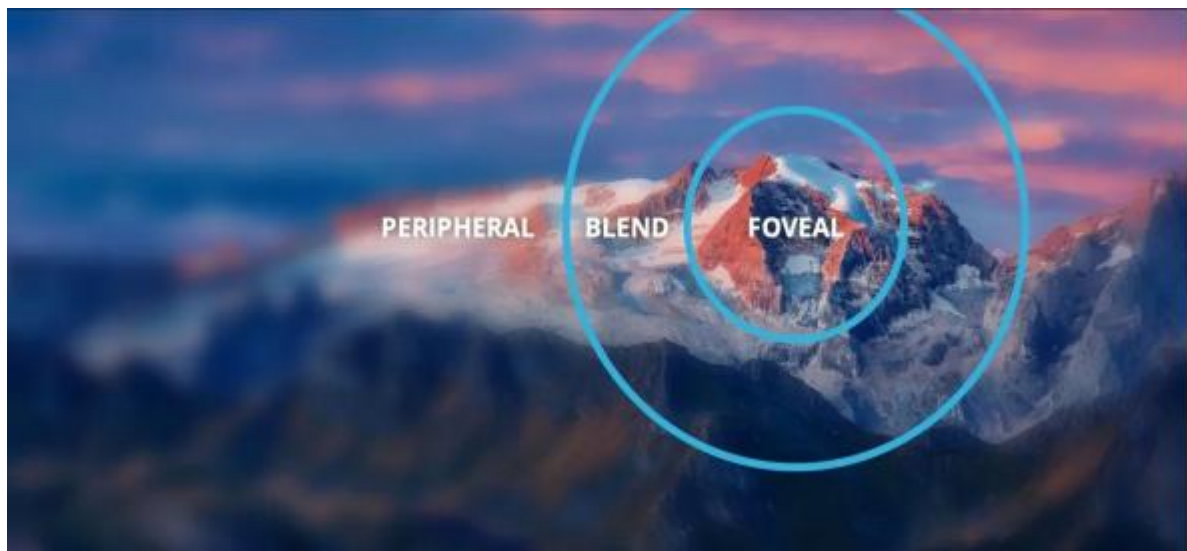
Примеры записи ответа: 3.14, 3,14

Ответ: 2.75

4. Кодирование информации. Объём информации (1 балл)

[Foveated rendering]

Илья изучает технологии оптимизации видеотрансляции. Недавно он узнал о технологии **«foveated rendering»**, позволяющей отслеживать взгляд пользователя и показывать в высоком качестве только ту часть изображения, на которую направлен взгляд, а остальную часть изображения показывать в более низком качестве.



Размер кадра в видео составляет 2560×1440 пикселей, каждый пиксель может быть одного из 65536 цветов, для каждого пикселя в кадре хранится значение его цвета, закодированное с использованием минимального, одинакового для всех цветов количества бит. Частота кадров в видео — 120 кадров/с.

Илье стало интересно, на сколько меньше КБайт памяти займёт видео длиной X секунд, если центральная (foveal) область будет составлять 10% изображения и к ней не будет применяться сжатие вовсе, радиус внешней границы области вокруг центральной (blend) будет в 2 раза больше радиуса центральной области и вместо хранения цвета **каждого** пикселя будет храниться цвет **каждого второго** пикселя. Для остальной части изображения (peripheral) вместо хранения цвета **каждого** пикселя будет храниться цвет **каждого четвёртого** пикселя.

Выяснилось, что искомое видео стало занимать на 5184000 Кбайт меньше. При каком наименьшем X это возможно? В качестве ответа укажите одно целое число.

Пример записи ответа: 171717

Ответ: 10

5. Основы логики. Анализ и упрощение логических выражений (1 балл)

[Налево, направо, налево, направо...]

Дано логическое выражение, записанное в следующей форме:

$$\oplus \wedge (\oplus X \oplus X \dots \oplus XX) (\wedge A \wedge B C) \oplus X \oplus X \dots \oplus XX$$

Здесь знаком $\oplus$ обозначается операция «исключающее ИЛИ».

Такая форма записи читается следующим образом: сначала указывается операция, а затем её операнды, которые могут быть переменными, константами и логическими выражениями (в этом случае они также начинаются со знака операции и читаются по таким же правилам). Выражения в скобках при вычислении имеют приоритет и вычисляются раньше.

Например, выражение $\wedge V S T P$ в заданной форме записи будет преобразовано к стандартной форме следующим образом $\wedge V S T P = \wedge (V S T) P = (S V T) \wedge P$.

В приведённом выражении на месте первого многоточия $\oplus X$ повторяется 2026 раз (с учётом трёх выписанных повторений), на месте второго многоточия — 4051 раз (также с учётом трёх выписанных повторений).

Упростите логическое выражение и запишите его в стандартной форме записи.

Примечание: переменные вводятся большими латинскими буквами; логические операции обозначаются, соответственно, not, and и or. Скобки используются только для изменения порядка выполнения операций. Если порядок выполнения операций очевиден из их приоритетов, дополнительное использование скобок считается ошибкой. Пробелы ставятся между логическими операциями и переменными. Если ответ равен константе, нужно ввести 0, если выражение эквивалентно значению FALSE, или 1, если выражение эквивалентно значению TRUE.

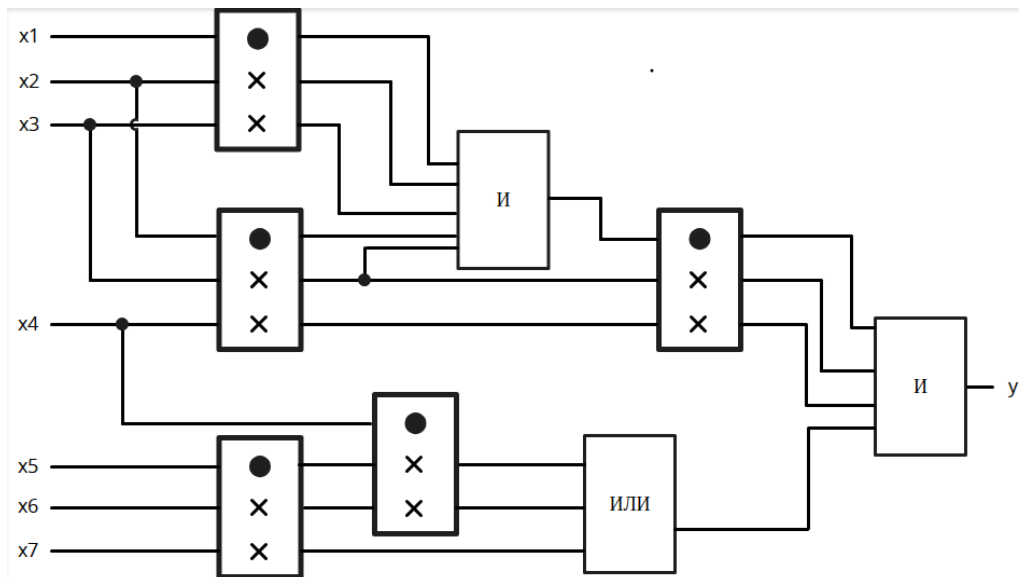
Пример записи ответа: (A or not B) and C

Ответ: A and B and C and X

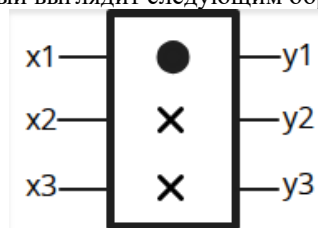
6. Основы логики. Синтез логического выражения по схеме (2 балла)

[Квантовая схема]

Дана логическая схема:



В ней используется вентиль Фредкина, который выглядит следующим образом:



Этот вентиль принимает на вход три бита и отдаёт на выход три бита. Таблица истинности вентиля выглядит следующим образом:

x1	x2	x3	y1	y2	y3
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	0
0	1	1	0	1	1
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	0	1
1	1	1	1	1	1

Сколько существует наборов входных значений, чтобы на выход схемы, представленной в начале, пришло значение 1? В ответе укажите целое положительное число.

Пример записи ответа: 171717

Ответ: 7

7. Алгоритмизация и программирование. Формальный исполнитель (2 балла)

[Удвоение подстрок]

Дана строка 132465. К ней применяется следующая последовательность преобразований:

- N символов в середине строки удваиваются. Например, при $N = 1$ строка abc превращается в строку $abbc$, а строка $abcd$ при $N = 2$ — в строку $abcbcd$. В случае несовпадения чётности длины строки с чётностью N данное преобразование пропускается.
- N символов в конце строки удваиваются. Например, при $N = 1$ строка abc превращается в строку $abcc$.
- N символов в начале строки удваиваются. Например, при $N = 1$ строка abc превращается в строку $aabc$.
- N увеличивается на 2.

Данная последовательность преобразований была применена к строке 15 раз. Определите, какие символы будут в строке в позициях с индексами 100, 200, 300, 400, 500 и 600, если символы строки нумеруются с 0, а начальное значение $N = 4$? В ответе укажите подряд без пробелов 6 цифр — цифры на искомым позициях.

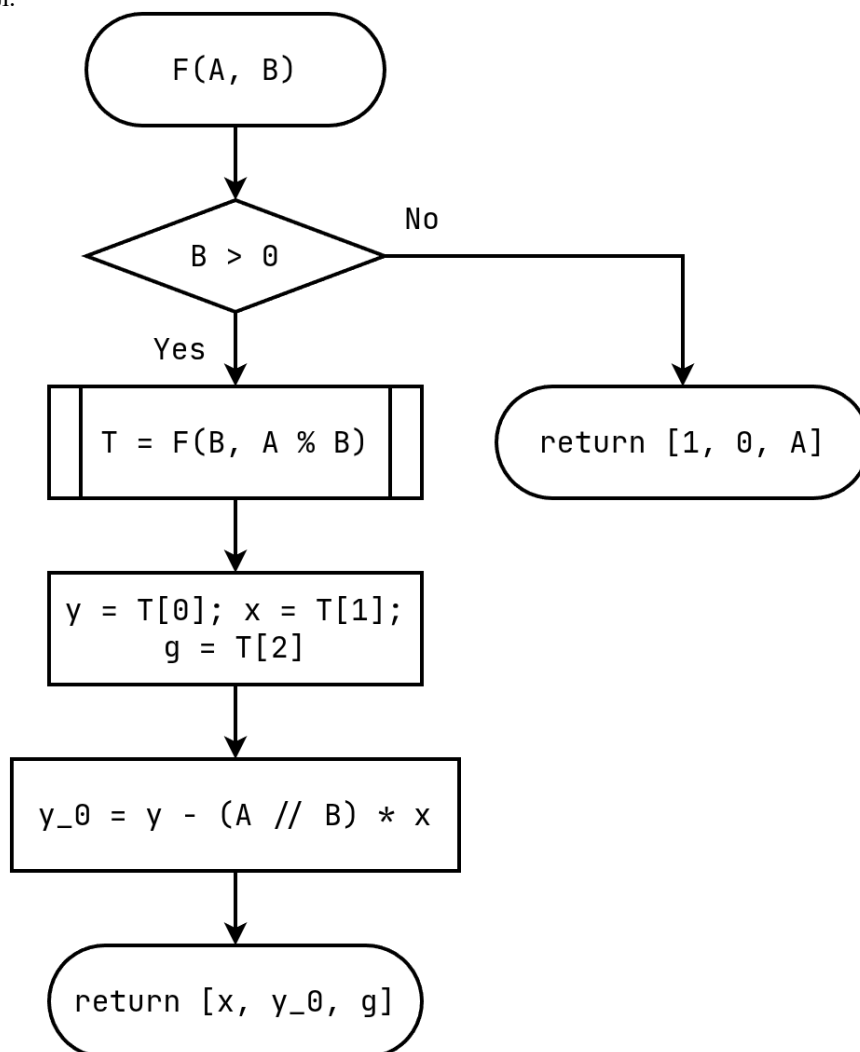
Пример записи ответа: 123456.

Ответ: 112626

8. Алгоритмизация и программирование. Блок-схема, обратная задача (3 балла)

[Безье? Безе?]

Дана блок-схема рекурсивного алгоритма, принимающего на вход два целых положительных числа и возвращающего массив из трёх целых чисел.



Известно, что при запуске алгоритма в качестве значения параметра A было передано число 6104798700. Какое число было передано в качестве параметра B при запуске алгоритма, если он вернул массив $[816, -751, 701]$? В ответе введите одно целое положительное число. Если таких чисел несколько, выберите наименьшее.

Пример ввода ответа: 171717.

Ответ: 6633176749

Отборочный этап. Второй тур (приведен один из вариантов заданий)

1. Электронные таблицы. Адресация ячеек, вычисления и графики (2 балла)

[Скачки]

В ячейках A2:A1001 в порядке возрастания записаны целые числа от 0 до 999.

На рисунке ниже изображен фрагмент электронной таблицы в режиме отображения формул:

Формулу из ячейки B2 скопировали во все ячейки диапазона B2:B1001. В ячейках B1 и C1 записаны целые положительные числа.

По полученным данным построили график, где значения по оси абсцисс берутся из диапазона A2:A2000, а по оси ординат — из диапазона B2:B2000. График изображен ниже:

Какие числа записаны в ячейках B1 и C1? В ответ запишите два числа через пробел, сначала число в B1, затем в C1.

Ответ: 30 50

2. Технологии сортировки и фильтрации данных (3 балла)

[Проверка избыточности]

Максим работает в Университете ИТМО, и одной из его задач является проверка образовательных программ. Каждая образовательная программа состоит из **дисциплин**, которые дают те или иные **компетенции**. Компетенция определяет одно требование к результату обучения. Например, компетенция «знать синтаксис языка программирования Python» может быть получена в ходе изучения дисциплины «Программирование на Python», которая реализуется в рамках образовательной программы «Разработка информационных систем».

Для удобства Максим собрал всю имеющуюся информацию в базу данных, которая имеет следующую структуру:

- Таблица `competency` хранит компетенции, которые может освоить студент в ходе изучения различных дисциплин:
 - `competency_id` — уникальный идентификатор компетенции;
 - `code` — уникальный код компетенции;
 - `name` — название компетенции;
 - `description` — описание компетенции;
- Таблица `discipline` хранит информацию о дисциплинах, которые студент может изучать при освоении образовательных программ:
 - `discipline_id` — уникальный идентификатор дисциплины;
 - `code` — уникальный код дисциплины;
 - `name` — название дисциплины;
 - `description` — описание дисциплины;
- Таблица `program` хранит информацию о программах, которые реализуются в университете:
 - `program_id` — уникальный идентификатор программы;
 - `code` — уникальный код программы;
 - `name` — название программы;
 - `description` — описание программы;
- Таблица `discipline_competency` хранит связку дисциплины и компетенции:
 - `discipline_id` — уникальный идентификатор дисциплины;
 - `competency_id` — уникальный идентификатор компетенции;
- Таблица `program_discipline` хранит связку дисциплины и образовательной программы:
 - `discipline_id` — уникальный идентификатор дисциплины;
 - `program_id` — уникальный идентификатор программы.

При этом гарантируется, что каждая связка дисциплины и компетенции, а также дисциплины и образовательной программы уникальна.

В целях оптимизации учебного процесса Максиму поручили определить количество различных пар дисциплин, которые дают как минимум одну одинаковую компетенцию и при этом реализуются в рамках образовательной программы «Микросервисные системы». При этом пары, которые отличаются только порядком элементов, считаются за одну. Если в образовательной программе есть три дисциплины, которые реализуют одну и ту же компетенцию, эти дисциплины образуют три пары (1 и 2, 2 и 3, 1 и 3).

Максим справился достаточно быстро и ушёл на обед. А сможете ли вы проверить, справился Максим с задачей правильно или нет? Определите, сколько пар дисциплин он должен был получить. В ответе введите целое положительное число.

Примечание: файл с базой данных доступен [на сайте](#).

Пример ввода ответа: 17

Ответ: 19

3. Технологии мультимедиа (2 балла)

[Что с вами сделали...]

Известно, что некоторое изображение состояло из 6 различных цветов. Ниже приведены значения цветовых каналов этих цветов в модели RGB.

Цвет	R	G	B
Цвет 1	90	60	90
Цвет 2	60	30	90
Цвет 3	60	60	240
Цвет 4	30	180	30
Цвет 5	30	90	60
Цвет 6	180	90	90

Изображение было переведено в модель HSB, после чего к изображению были применены ровно 3 из следующих преобразований:

1. Увеличить Hue на 100;
2. Уменьшить Hue на 100;
3. Уменьшить Hue в 2 раза;
4. Увеличить Hue в 2 раза;
5. Увеличить Saturation на 50;
6. Уменьшить Saturation на 50;

7. Увеличить Brightness на 25;
8. Уменьшить Brightness на 25.

Если при применении операций 1-4 получается величина меньшая 0 или большая 359, она берётся по модулю 360. Если при выполнении операций 5-8 получается величина большая 100, она принимается равной 100, а если получается величина меньше 0, она принимается равной 0.

Полученное изображение было переведено обратно в модель RGB. После этого в изображении присутствуют следующие цвета:

Цвет	R	G	B
Цвет А	22	21	26
Цвет Б	26	22	21
Цвет В	26	26	26
Цвет Г	79	91	117
Цвет Д	117	117	117
Цвет Е	176	132	147

Определите, какие цвета соответствовали цветам А-Е. В ответ запишите последовательность из 6 цифр без пробелов и разделяющих символов.

Пример записи ответа: 123654

Примечание: для перевода RGB в HSB необходимо выполнить следующие шаги:

1. Разделить значения R , G и B на 255. Полученные значения назовём R' , G' и B' соответственно.
2. Вычислить величины $MAX = \max(R', G', B')$, $MIN = \min(R', G', B')$, $D = MAX - MIN$.
3. $Br = MAX \times 100\%$
4. $Sat = \frac{D}{MAX} \times 100\%$ (если $MAX = 0$, Sat также принимается равным 0)
5. Если $D = 0$, Hue принимается равным 0° .
 - 5.1. Если $MAX = R'$, $Hue = 60^\circ \times (\frac{G' - B'}{D} \bmod 6)$
 - 5.2. Если $MAX = G'$, $Hue = 60^\circ \times (\frac{B' - R'}{D} + 2)$
 - 5.3. Если $MAX = B'$, $Hue = 60^\circ \times (\frac{R' - G'}{D} + 4)$
 - 5.4. Если выполняется несколько из условий 5.1-5.3, можно выбрать любое
 - 5.5. Если в результате вычислений $Hue < 0^\circ$, прибавить к получившемуся значению 360°

Ответ: 521463

4. Телекоммуникационные технологии (1 балл)

[Восстановление настроек]

Маршрутизаторы (аппаратные или программные) выполняют задачу выбора оптимального маршрута IP-пакета и его отправки по этому маршруту. Для принятия решения анализируется адрес получателя и на основе таблиц маршрутизации устанавливается маршрут.

В таблице маршрутизации присутствуют как минимум следующие поля:

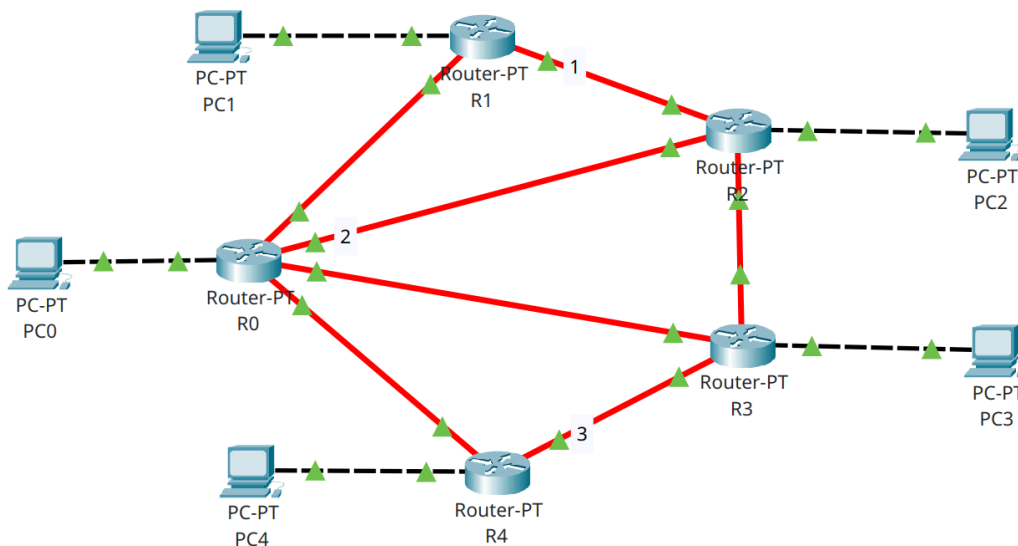
- адрес назначения (IP-адрес сети или конкретного хоста);
- идентификатор порта, через который пакет идет до сети назначения (порт обозначается IP-адресом или внутренним номером);
- шлюз (gate, IP-адрес, принадлежащий к одной из локальных сетей, непосредственно подключенных к маршрутизатору, на который необходимо отправить пакет, после того как пакет покинет порт).

На каждом маршрутизаторе сети присутствует таблица, полностью описывающая структуру всей сети и иногда содержащая записи о маршрутах по умолчанию. Запись по умолчанию отличается тем, что адрес назначения и маска назначения имеют значения, равные 0.0.0.0.

Пятеро друзей часто заходят в один и тот же компьютерный клуб и знают настройки IP для тех компьютеров, за которыми они обычно сидят:

- PC0: address 172.18.19.34/29, gate 172.18.19.33;
- PC1: address 172.18.19.2/29, gate 172.18.19.1;
- PC2: address 172.18.19.10/29, gate 172.18.19.9;
- PC3: address 172.18.19.18/29, gate 172.18.19.17;
- PC4: address 172.18.19.26/29, gate 172.18.19.25.

Им известна общая схема сети, приведенная на рисунке:



Также они смогли получить таблицы маршрутизации некоторых маршрутизаторов.

Таблица А:

IP назначения	Маска назначения	Порт	Шлюз
172.18.19.8	255.255.255.248	10.244.135.182	10.244.135.186
172.18.19.32	255.255.255.248	10.244.133.122	10.244.133.163
172.18.19.16	255.255.255.240	10.244.133.122	10.244.133.163

Таблица В:

IP назначения	Маска назначения	Порт	Шлюз
172.18.19.32	255.255.255.248	10.244.219.99	10.244.219.5
172.18.19.16	255.255.255.248	10.244.13.76	10.244.13.100
172.18.19.24	255.255.255.248	10.244.13.76	10.244.13.100
172.18.19.0	255.255.255.248	10.244.135.186	10.244.135.182

Таблица С:

IP назначения	Маска назначения	Порт	Шлюз
172.18.19.32	255.255.255.248	10.244.145.115	10.244.145.110
172.18.19.0	255.255.255.248	10.244.145.115	10.244.145.110
172.18.19.8	255.255.255.248	10.244.13.100	10.244.13.76
172.18.19.24	255.255.255.248	10.244.6.247	10.244.6.118

Таблица D:

IP назначения	Маска назначения	Порт	Шлюз
172.18.19.16	255.255.255.248	10.244.6.118	10.244.6.247
172.18.19.8	255.255.255.248	10.244.6.118	10.244.6.247
0.0.0.0	0.0.0.0	10.244.110.121	10.244.110.125

Также они установили, что любой маршрут проходит максимум через три маршрутизатора.

По полученным данным восстановите значения IP-адресов на трёх пронумерованных на схеме портах маршрутизаторов.

В ответ приведите IP-адреса для порта 1, 2 и 3 в указанном порядке через пробел.

Ответ: 10.244.135.182 10.244.219.5 10.244.6.118

5. Операционные системы (2 балла)

[Регулярный поиск]

Дана операционная система семейства GNU/Linux. Известны результаты последовательного выполнения нескольких команд в некотором текущем каталоге:

В результате выполнения команды `ls -Rl` был получен следующий вывод:

```

.:
drwxrwxr-x. 4 user user 74 May 22 02:28 Dir2
drwxrwxr-x. 3 user user 61 May 22 02:29 Dir3
-rw-rw-r--. 1 user user  2 May 22 02:30 file1
-rw-rw-r--. 1 user user  2 May 22 02:30 file2
-rw-rw-r--. 1 user user  2 May 22 02:30 file3

./Dir2:
drwxrwxr-x. 2 user user 51 May 22 01:44 Dir21
drwxrwxr-x. 2 user user 51 May 22 02:27 Dir22
-rw-rw-r--. 1 user user  2 May 22 02:28 file21
-rw-rw-r--. 1 user user  2 May 22 02:28 file22
-rw-rw-r--. 1 user user  2 May 22 02:28 file23

./Dir2/Dir21:
-rw-rw-r--. 1 user user  2 May 22 01:44 file211
-rw-rw-r--. 1 user user  2 May 22 01:44 file212
-rw-rw-r--. 1 user user  2 May 22 01:44 file213

./Dir2/Dir22:
-rw-rw-r--. 1 user user  2 May 22 02:26 file221
-rw-rw-r--. 1 user user  2 May 22 02:26 file222
-rw-rw-r--. 1 user user  2 May 22 02:27 file223

./Dir3:
drwxrwxr-x. 2 user user 51 May 22 02:29 Dir32
-rw-rw-r--. 1 user user  2 May 22 02:29 file31
-rw-rw-r--. 1 user user  2 May 22 02:29 file32
-rw-rw-r--. 1 user user  2 May 22 02:29 file33

./Dir3/Dir32:
-rw-rw-r--. 1 user user  2 May 22 02:29 file321
-rw-rw-r--. 1 user user  2 May 22 02:29 file322
-rw-rw-r--. 1 user user  2 May 22 02:29 file323

```

В результате выполнения команды `cat $(find -type f) 2>/dev/null` были последовательно, каждая в своей строке выведены буквы английского алфавита от а до г.

Известно, что файл с именем `file211` содержит символ а, файл с именем `file21` содержит символ g, а файл с именем `file3` содержит символ г.

Затем были последовательно выполнены еще три команды:

```

chmod 222 $(find -type f -regex ".*file.*[13]+")
chmod 664 $(find -type f -regex ".*file.*[23]+1")
cat $(find -type f) 2>/dev/null

```

Какие символы будут выведены в результате выполнения последней команды? В ответе укажите их подряд без пробелов в алфавитном порядке.

Примечания:

- `ls -Rl` — команда, рекурсивно выводящая содержимое текущего и всех вложенных каталогов, включая информацию о типе (каталог или регулярный файл) и правах доступа к файлам и каталогам.
- `cat $(find -type f) 2>/dev/null` — команда, рекурсивно обходящая все подкаталоги и выводящая содержимое всех найденных файлов с игнорированием вывода сообщений об ошибках доступа к файлам.
- `chmod ### $(find #####)` — команда, применяющая маску прав доступа (три восьмеричных числа, соответствующих битам прав на чтение, запись и исполнение файла для владельца, группы и всех пользователей) для файлов, которые найдет команда `find`.
- Ключ `-type f` у команды `find` выводит только полные имена регулярных файлов, пропуская вывод имен файлов, являющихся каталогами.
- Ключ `-regex` позволяет выводить только полные имена файлов, соответствующих регулярному выражению.

Специальные символы, использованные в регулярных выражениях:

Спецсимвол	Описание
.	любой символ
[]	диапазон допустимых символов, например, [abc]
+	предыдущий символ повторяется 1 или более раз
*	предыдущий символ повторяет 0 или более раз

Все команды вызываются от имени пользователя, являющегося владельцем всех файлов и каталогов в текущем каталоге с учетом вложенности.

Ответ: bdeghjkmnpq

6. Технологии программирования (3 балла)

[Эмулятор игры 2048]

Имя входного файла	стандартный ввод
Имя выходного файла	стандартный вывод
Ограничение по времени	1 секунда
Ограничение по памяти	256 МБ

Васе очень нравится игра «2048», но ему не хватает в ней гибкости. Он решил создать свою версию, где размер доски, возможные номиналы плиток и цель игры могут меняться. Ваша задача — написать движок-валидатор для этого турнира.

Игра происходит на квадратном поле размера $X \times X$ (в данной задаче $X = 5$).

- Слияние:** При сдвиге в одном из четырёх направлений плитки с одинаковым номиналом объединяются, если они «налетают» друг на друга. Номинал новой плитки равен сумме двух предыдущих. Одна плитка не может участвовать в слиянии дважды за один ход. Порядок слияния соответствует классической игре 2048. К примеру: строка 22200 при сдвиге вправо превратится в 00024. Под нулём подразумеваются пустые клетки.
- Ход:** Считается совершённым, если хотя бы одна плитка изменила своё положение или произошло слияние.
- Очередность:** После каждого успешного хода на поле должна появиться новая плитка.

Формат входных данных

В первой строке содержится целое число n ($1 \leq n \leq 10$) — количество возможных номиналов новых плиток.

Во второй строке содержится n целых различных чисел $a_1, a_2, \dots, a_n$ ($2 \leq a_i \leq 2^{10}$) — доступные номиналы для новых плиток. Гарантируется, что a_i — степень двойки.

В третьей строке содержится целое число m ($2^{11} \leq m \leq 2^{20}$) — минимальная стоимость плитки для победы.

В четвертой строке содержится целое число q ($2 \leq q \leq 10^4$) — количество запросов.

Далее следует q запросов. Каждый запрос начинается с типа операции T . Номер запроса x считается с нуля.

Типы запросов

- Тип 1:** Вывести текущее состояние доски в виде матрицы $X \times X$. Пустые клетки выводятся как 0.
- Тип 2 (Появление):** На следующей строке даны x, y, b .
 - x, y — координаты ($0 \leq x, y < X$).
 - b — номинал ($2 \leq b \leq 2^{10}$).
- Тип 3 (Ход):** На следующей строке дано число d — направление.
 - 0 — вверх, 1 — вправо, 2 — вниз, 3 — влево.

Примечание: Гарантируется, что самым первым игровым запросом будет запрос типа 2. Проверка на поражение производится сразу после появления новой плитки.

Формат выходных данных

На каждый запрос первого типа нужно вывести 5 строк по 5 чисел через пробел — доску с плитками. Если плитка пустая, следует вывести 0.

Если происходит одно из следующих событий, программа должна вывести соответствующее сообщение, после чего завершить работу.

Событие	Сообщение
Победа: На поле появилась плитка номиналом	Player won. Step: x
Поражение: Нет ни одного хода (тип), который сдвинет плитки	Player lost. Step: x
Нарушение очереди: Два появления или два хода подряд	Incorrect step. Step: x
Неэффективный ход: Ход (тип) не изменил состояние доски	Incorrect step. Step: x
Некорректный спавн: Клетка занята или номинал	Incorrect step. Step: x

Вместо x следует написать номер текущего хода, считая с нуля.

Примеры

Стандартный ввод	Стандартный вывод
2	0 0 0 0 0
8 64	0 0 0 0 0
4096	0 0 0 0 0
9	0 0 0 0 64
2	0 0 0 0 0
3 2 64	0 0 0 0 0
3	0 0 0 0 0

1 1 2 4 4 8 1 3 2 2 0 0 64 1 3 0	0 0 0 0 0 0 0 0 0 64 0 0 0 0 8 Incorrect step. Step: 5
5 2 4 8 16 32 2048 13 2 2 1 2 1 3 0 1 2 0 2 2 3 3 1 2 0 2 4 3 2 2 4 4 4 1 3 3 1	0 0 0 0 0 0 0 0 0 0 0 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 0 4 0 4 0 4 0 4 0 8 4 0 0 0
1 2 2048 7 2 0 0 2 3 2 2 0 1 2 3 0 2 0 2 2 3 1 1	0 0 0 2 4 0
2 2 4 2048 2 2 0 1 2 1	0 2 0

Комментарий

После победы, поражения или некорректного хода, оставшийся ввод следует игнорировать.

7. Технологии программирования (3 балла)

[Поиск пути]

Имя входного файла	стандартный ввод
Имя выходного файла	стандартный вывод
Ограничение по времени	2,5 секунды
Ограничение по памяти	256 МБ

Андрей – студент ИТМО. Он очень любит гулять по Санкт-Петербургу. Город представляет собой граф из n перекрестков и m улиц, по улицам можно ходить в обе стороны.

После каждой прогулки Андрей оценивает, насколько ритмичной она получилась. Он считает, что у прогулки есть ритм k , если число улиц, которые он прошел, кратно k . **Андрей может проходить по одной и той же улице несколько раз (даже подряд).**

Так как ходить одинаковыми маршрутами слишком скучно, ему стало интересно, можно ли начать в перекрестке v и закончить в перекрестке u , чтобы у прогулки был ритм k .

Маршрут — такая последовательность вершин $a_1, \dots, a_t, t > 1$, что соседние вершины соединены ребром (ребра и вершины могут повторяться). Длина такого маршрута считается равной $t - 1$.

Вы должны помочь ему и ответить на q запросов.

Формат входных данных

В первой строке даны числа n и m — количество перекрестков и улиц в городе ($1 \leq n \leq 10^5, 0 \leq m \leq 10^5$).

В последующих m строках дано описание графа, по два числа в строке v и u — улица, соединяющая вершины v и u ($1 \leq v, u \leq n$). В графе могут присутствовать петли и кратные ребра.

В последней строке дано одно число q — количество вопросов у Андрея ($1 \leq q \leq 10^5$).

В последующих q строках дано по три числа v, u, k — стартовый и конечный перекресток и требуемая ритмичность прогулки ($1 \leq v, u \leq n, 1 \leq k \leq 10^6$).

Формат выходных данных

Вы должны вывести q строк.

В i -й строке ответ на i -й запрос: Yes, если существует маршрут с ритмичностью k , и No иначе.

Пример

Стандартный ввод	Стандартный вывод
7 6	Yes
1 2	No
2 3	Yes
3 4	Yes
4 1	No
1 5	
6 6	
5	
1 3 2	
1 2 2	
1 1 6	
6 6 5	
7 7 2	

Задания для 9 класса

Заключительный этап (приведен один из вариантов заданий)

1. Кодирование информации. Системы счисления (1 балл)

[Многие степени единственного X]

Дано число $A = a_1 a_2 \dots a_k N$. В данном числе $a_1 - a_k - K$ цифр числа, N – основание системы счисления. Известно, что и каждая из цифр $a_1 - a_k$, и N будучи переведёнными в десятичную систему окажутся равны некоторой степени некоторого числа X (число X идентично для всех, степень – различается). Известно, что все цифры числа различны и записаны по возрастанию, а N – наименьшее возможное. Определите максимальное количество идущих подряд нулей в числе B , получаемом в результате перевода числа A в систему счисления с основанием X , если $K = 1000$.

Ответ: 999

Решение:

Так как все цифры данного числа – корректные цифры в системе счисления с основанием N и они записаны по возрастанию, тогда $a_1 < a_2 < \dots < a_k < N$, и N – наименьшее возможное, то в десятичной системе счисления мы получим следующий набор чисел: $a_1 = 1 = X^0, a_2 = X^1, \dots, a_k = X^{k-1}, N = X^k$.

Переведём число A в десятичную систему счисления:

$$A = a_1 * N^{k-1} + a_2 * N^{k-2} + \dots + a_k * N^0.$$

Подставим в данное выражение известные нам значения:

$$A = X^0 * (X^k)^{k-1} + X^1 * (X^k)^{k-2} + \dots + X^{k-1} * (X^k)^0.$$

Каждое слагаемое в этой формуле будет равно X^{E_i} , где $E_i = (i-1) + k * (k-i)$, где i – от 1 до k .

При переводе в систему счисления с основанием X число A будет состоять из единиц в позициях E_i и нулей во всех остальных позициях. Нам нужно найти наибольшее расстояние между единицами (это даст нули в середине числа), а также проверить количество нулей в самом конце числа.

Найдём количество 0 между двумя единицами. Для этого найдём разность показателей двух соседних степеней:

$$E_i - E_{i+1} = (i-1) + k * (k-i) - ((i+1-1) + k * (k-(i+1))) = i + k * (k-i) - 1 - (i + k * (k-i) - k) = k - 1$$

Значит, если между показателями ровно $k-1$, то нулей будет на 1 меньше, т.е. $k-2$.

Определим также число нулей в конце числа. Самый младший равный единице разряд определяется через $E_k = k - 1 + k * (k - k) = k - 1$. Это означает, что все меньшие разряды, которых будет ровно $k-1$, будут нолями. Значит, в конце числа будет большего размера блок нулей, чем между единицами. Значит, ответ равен $k-1$, т.е. при $k = 1000$ – это 999.

Также разберём случай с записью цифр по убыванию.

Так как все цифры данного числа – корректные цифры в системе счисления с основанием N и они записаны по возрастанию, тогда $N > a_1 > a_2 > \dots > a_k$, и N – наименьшее возможное, то в десятичной системе счисления мы получим следующий набор чисел: $a_1 = X^{k-1}, a_2 = X^{k-2}, \dots, a_k = 1 = X^0, N = X^k$.

Переведём число A в десятичную систему счисления:

$$A = a_1 * N^{k-1} + a_2 * N^{k-2} + \dots + a_k * N^0.$$

Подставим в данное выражение известные нам значения:

$$A = X^{k-1} * (X^k)^{k-1} + X^{k-2} * (X^k)^{k-2} + \dots + X^0 * (X^k)^0.$$

Каждое слагаемое в этой формуле будет равно X^{E_i} , где $E_i = (k-i) + k * (k-i) = (k-i) * (k+1)$, где i – от 1 до k .

При переводе в систему счисления с основанием X число A будет состоять из единиц в позициях E_i и нулей во всех остальных позициях. Нам нужно найти наибольшее расстояние между единицами (это даст нули в середине числа), а также проверить количество нулей в самом конце числа.

Найдём количество 0 между двумя единицами. Для этого найдём разность показателей двух соседних степеней:

$$E_i - E_{i+1} = (k-i) * (k+1) - (k-i-1) * (k+1) = (k+1)(k-i-k+i+1) = k+1$$

Значит, если между показателями ровно $k+1$, то нулей будет на 1 меньше, т.е. k .

Нетрудно заметить, что при $i = k, E_i = 0$, т.е. число будет оканчиваться на единицу, т.е. на конце числа 0 нулей. Таким образом, наибольшее количество подряд идущих нулей равно k , т.е. ответ – 1000.

Получить ответ на эту задачу можно, обнаружив подобную закономерность на простом примере с минимальным возможным $X = 2$ и меньшим K . Интересующие нас числа будут иметь вид соответственно:

$$\begin{aligned} 124_8 &= 1010100_2 \\ 1248_{16} &= 1001001001000_2 \\ 1248G_{32} &= 10001000100010001000_2 \end{aligned}$$

Легко заметить закономерность – на конце данных чисел ровно на 1 ноль меньше, чем было чисел в числе, а между единицами – на 2 ноля меньше.

Аналогично, с записью по возрастанию:

$$\begin{aligned} 421_8 &= 100010001_2 \\ 8421_{16} &= 1000010000100001_2 \\ G8421_{16} &= 1000001000001000001000001_2 \end{aligned}$$

Легко заметить закономерность – между единицами всегда равное число 0 нулей и их ровно K .

2. Кодирование информации. Объем информации (2 балла)

[3 прямоугольника]

Вася работает в графическом редакторе с поддержкой нескольких слоёв. Графический редактор настроен так, что значения цветов при наложении слоёв (за исключением фона) складываются друг с другом. В этом графическом редакторе

Вася создал картину размером 1920x1080 пикселей из трёх пересекающихся цветных прямоугольников (красного, зелёного и синего) на белом фоне и сохранил её в формате True Color (по 8 бит на каждый из 3 цветовых каналов каждого пикселя). Изначально каждый из прямоугольников находился на собственном слое, однако сохранён был именно итоговый результат, полученный после наложения слоёв.

Его друг Коля счёл такое хранение избыточным и предложил записывать информацию о цветовых каналах с нулевым значением с использованием всего лишь одного бита. Коля посчитал, что в таком случае изображение будет занимать как минимум на 19999 байт меньше.

Определите минимально возможное число пикселей, занимаемых прямоугольниками, при котором это возможно.

Примечание: гарантируется, что все 3 прямоугольника имеют ненулевую площадь и все пересечения непусты.

Ответ: 11431

Решение:

Для начала определим, что пиксели данного изображения могут иметь 1, 2 или 3 ненулевых цвета. Для каждого такого типа пикселей вычислим изменение требуемого количества памяти.

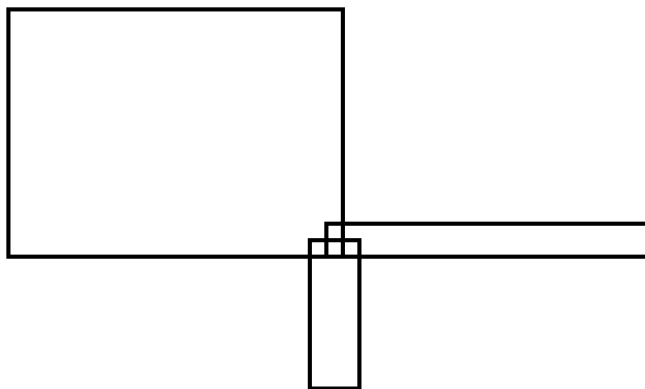
4. Трёхцветные пиксели занимали $3 * 8$ бит по методу Васи и будут занимать ровно столько же по методу Коли.
5. Двухцветные пиксели занимали $3 * 8$ бит по методу Васи и будут занимать $2 * 8 + 1 = 17$ бит, т.е. на 7 бит меньше по методу Коли.
6. Одноцветные пиксели занимали $3 * 8$ бит по методу Васи и будут занимать $8 + 1 + 1 = 10$ бит, т.е. на 14 бит меньше по методу Коли.

То есть, для достижения наибольшей экономии за минимальное число пикселей, необходимо, чтобы наибольшее число пикселей оказались одноцветными и как можно меньше пикселей имели 2 или 3 цвета.

Теперь определим категории пикселей на рисунке:

1. Белые пиксели. Имеют значение 255, 255, 255, т.е. на их хранении сэкономить не выйдет.
2. Пиксели, принадлежащие пересечению всех трёх прямоугольников. Также будут иметь ненулевые значения для всех трёх цветовых каналов, а значит не дадут никакого выигрыша по памяти. Однако, по условию данное пересечение непусто. Тем не менее, нам выгодно, чтобы оно имело минимальный возможный размер – это 1 пиксель.
3. Пиксели, принадлежащие попарным пересечениям прямоугольников. Они будут двухцветными, значит, их тоже должно быть как можно меньше. По условию, все 3 пересечения непусты, значит, таких пикселей будет как минимум 3 – по одному на каждое попарное пересечение.
4. Пиксели, принадлежащие прямоугольникам, но не относящиеся ни к одному из пересечений. Таких пикселей у нас должно быть как можно больше, обозначим их количество за X .

Удостоверимся, что подобное распределение достижимо:



Если прямоугольник справа будет иметь высоту 2 пикселя, а нижний – ширину 3 пикселя, то ситуация с тремя попарными пересечениями по 1 пикселю каждое и пересечению всех трёх также размером в 1 пиксель действительно возможно.

Так как уменьшение требуемого количества памяти основана на одно- и двухцветных прямоугольниках, составим неравенство:

$$X * 14 + 3 * 7 \geq 19999 * 8$$

Упростив, получим $X \geq 11426,5$. Очевидно, что X должно быть целым числом, значит $X = 11427$. Мы выяснили, сколько одноцветных пикселей есть в картине. Остаётся прибавить только пиксели попарных пересечений (3) и пиксель из пересечения всех трёх прямоугольников. Получим: $11427 + 3 + 1 = 11431$.

3. Основы логики (2 балла)

[Стрелка за стрелкой]

Два набора значений переменных A , B и C называют не эквивалентными, если значение хотя бы одной переменной различается.

Дано логическое выражение. В данном выражении A , B и C – логические переменные, F – функция от этих переменных.

$$(((A \rightarrow B) \rightarrow C) \rightarrow ((A \rightarrow C) \rightarrow B)) \rightarrow F$$

Данное логическое выражение истинно при 8 не эквивалентных наборах значений переменных A , B , C , а F – при 7 таких наборах. Определите функцию F , если известно, что функция F содержит все 3 логические переменные и не более чем 3 логические операции. Если таких функций несколько – запишите любую из них.

В ответе запишите формулу, которая содержит логические переменные A , B и C (все 3) и не более чем три логические операции. Если таких функций не существует, запишите в ответ NULL.

Комментарий по вводу ответа: операнды вводятся большими латинскими буквами; логические операции обозначаются, соответственно, как not, and и or. Запись не должна содержать скобок.

Пример записи ответа: A or not B

Ответ: A or B or not C || A or not C or B || B or A or not C || B or not C or A || not C or A or B || not C or B or A

Решение:

Построим таблицу истинности для левой части выражения $((A \rightarrow B) \rightarrow C) \rightarrow ((A \rightarrow C) \rightarrow B)$, для удобства обозначим эту часть выражения как G.

A	B	C	G
FALSE	FALSE	FALSE	TRUE
FALSE	FALSE	TRUE	FALSE
FALSE	TRUE	FALSE	TRUE
FALSE	TRUE	TRUE	TRUE
TRUE	FALSE	FALSE	TRUE
TRUE	FALSE	TRUE	FALSE
TRUE	TRUE	FALSE	TRUE
TRUE	TRUE	TRUE	TRUE

Значит, выражение целиком будет иметь вид not G or F. В таком случае, данное выражение будет истинно либо если G – ложно (т.е. при наборах A = B = FALSE, C = TRUE и A = C = TRUE, B = FALSE) или если F – истинно.

В условии сказано, что выражение истинно при 8 наборах, а F – при 7 различных наборах. Значит, если наличие FALSE у F не повлияло на итоговый результат и совпадает с FALSE у G, то существуют 2 возможных варианта:

1. F(FALSE, FALSE, TRUE) = FALSE и F(TRUE, FALSE, TRUE) = TRUE
2. F(FALSE, FALSE, TRUE) = TRUE и F(TRUE, FALSE, TRUE) = FALSE

Заметим, что функция F содержит все 3 логических переменных, может содержать только операции and, or и not. То есть, будет иметь вид $*X ? *Y ? *Z$, где на месте X, Y, Z будут логические переменные A, B и C (в каком-то порядке), на месте * может быть или не быть операция not, на месте ? – операции and или or. Заметим, что в функции такого вида если на месте обоих знаков вопроса будет and – функция будет истина в 1 случае, если на месте знаков вопроса будут различные операции, то функция будет истинна в 5 случаях, а если на месте обоих знаков вопроса будет or – то функция будет истинна в 7 случаях. Именно этот вариант и соответствует известной информации о функции F.

Заметим, что в случае двух операций or порядок переменных не важен, поэтому подставим на места знаков вопроса операции or, а на места X, Y и Z наши переменные. Получим: $F = *A \text{ or } *B \text{ or } *C$, где на месте * может быть или не быть операции not.

Заметим, что по условию в функции F не более 3 операций. А значит, в данном выражении одна или ноль операций not.

Таким образом, у нас остаётся всего 4 возможные функции. Составим для всех них сводную таблицу истинности, в которой особое внимание уделим интересующим нас наборам A = B = FALSE, C = TRUE и A = C = TRUE, B = FALSE.

A	B	C	A or B or C	not A or B or C	A or not B or C	A or B or not C
FALSE	FALSE	FALSE	FALSE	TRUE	TRUE	TRUE
FALSE	FALSE	TRUE	TRUE	TRUE	TRUE	FALSE
FALSE	TRUE	FALSE	TRUE	TRUE	FALSE	TRUE
FALSE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE
TRUE	FALSE	FALSE	TRUE	FALSE	TRUE	TRUE
TRUE	FALSE	TRUE	TRUE	TRUE	TRUE	TRUE
TRUE	TRUE	FALSE	TRUE	TRUE	TRUE	TRUE
TRUE	TRUE	TRUE	TRUE	TRUE	TRUE	TRUE

Нетрудно заметить, что искомое требование – FALSE для одного из интересующих нас наборов – выполняется только для последней функции. Она и будет нашим ответом – A or B or not C.

4. Алгоритмизация и программирование. Формальные исполнители (3 балла)

[Безвыходное положение]

Робот перемещается по полю 10x10 клеток. В свой ход робот может пойти на 1 клетку вверх, вправо, вниз, или влево.

Известен алгоритм перемещения робота:

1. Выполнить ход в заданном направлении, если в этом направлении есть клетка, эта клетка не была посещена роботом и сумма координат этой клетки не кратна X.
2. Увеличить число X на 1. Если X = 12, изменить значение X на 2.
3. Пометить текущую клетку робота как посещённую (повторная пометка не является ошибкой, если робот пропустил свой ход)
4. Определить направление следующего шага согласно таблице:

Текущее заданное направление шага	Направление следующего шага
Вправо	Вниз
Вниз	Влево
Влево	Вверх
Вверх	Вправо

Если робот не может переместиться в свой ход, он может его пропустить (т.е. не совершать перемещение, выполнив все остальные шаги алгоритма). Однако, робот может пропустить максимум 4 хода подряд, иначе он проигрывает. Определите, через сколько перемещений робот проиграет, если начальное значение $X = 2$ и робот начинает своё движение в клетке с координатами { 3; 7 } и делает первый ход вправо. Строки и столбцы поля нумеруются с нуля, первая координата соответствует номеру строки, вторая – номеру столбца, координаты { 0; 0 } имеет верхний левый угол, координаты { 9; 9 } – правый нижний.

Ответ: 28

Решение:

Хотя данную задачу можно решить вручную, проще всего сделать это программно. В качестве примера решения приведём код на языке Python.

```
field = [[False] * 10 for _ in range(10)]

next_dir = {
    'right': 'down',
    'down': 'left',
    'left': 'up',
    'up': 'right',
}

movement = {
    'right': [0, 1],
    'down': [1, 0],
    'left': [0, -1],
    'up': [-1, 0],
}

row, col = 3, 7
x = 2
dir = 'right'

fails = 0
steps = 0

while True:
    next_pos = [row + movement[dir][0], col + movement[dir][1]]

    if (0 <= next_pos[0] <= 9 and 0 <= next_pos[1] <= 9) and not
field[next_pos[0]][next_pos[1]] and (next_pos[0] + next_pos[1]) % x > 0:
        row, col = next_pos
        fails = 0
        steps += 1
        print('Переходим в', (row, col), 'x =', x)
    else:
        fails += 1
        print('Остаёмся в', (row, col), 'x =', x)

    if fails > 4:
        break

    x += 1
    if x == 12: x = 2

    field[row][col] = True

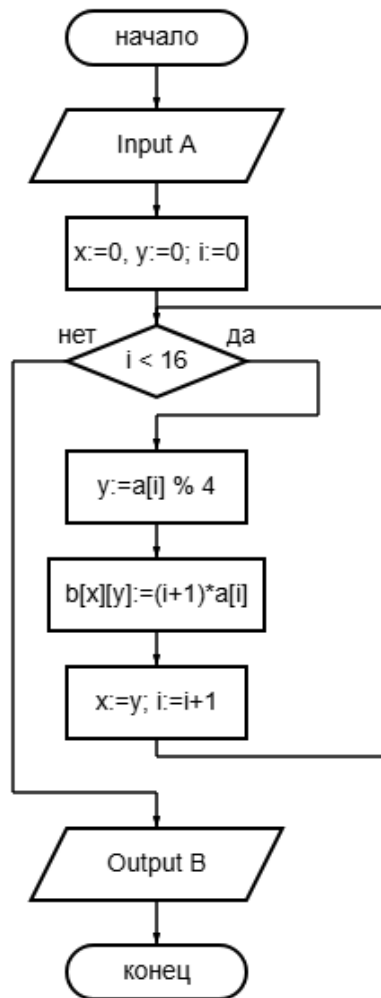
    dir = next_dir[dir]

print(steps)
```

5. Алгоритмизация и программирование. Анализ алгоритма, заданного в виде блок-схемы (1 балл)

[16 цифр]

Дан алгоритм:



На вход данному алгоритму была подана строка A из 16 десятичных цифр, массив B изначально заполнен 0. В результате алгоритм вывел следующий результат:

16	45	2	30
128	6	42	36
36	15	16	14
24	55	28	91

Восстановите строку A.

Примечание: операция % означает взятие по модулю.

Ответ: 2784916243537218

Решение:

Заметим, что в данном алгоритме в столбец y итоговой таблицы помещается произведение $a[i]$, имеющего остаток y по модулю 4 и $i + 1$. Все $a[i]$ являются десятичными цифрами. Значит, все числа в первом столбце были умножены на 4 или 8 (0 тоже является цифрой и имеет остаток 0 по модулю 4, но в таблице нет 0, значит и в исходном числе 0 не было), все числа во втором столбце – на 1, 5 или 9, в третьем – на 2 или 6, в четвертом – на 3 или 7.

16	45	2	30
128	6	42	36
36	15	16	14
24	55	28	91
4, 8	1, 5, 9	2, 6	3, 7

Выделим в таблице те числа, которые имеют в качестве делителей ровно одну из представленных цифр или хотя и имеют в качестве делителей более чем одну цифру, но только одно частное будет не больше 16 (а $i + 1$ не может быть больше 16, т.к. i в алгоритме – от 0 до 15):

16	45	2	30
128	6	42	36
36	15	16	14
24	55	28	91
4, 8	1, 5, 9	2, 6	3, 7

Заменяем данные числа на пару цифра / позиция:

16	45	2 / 1	3 / 10
8 / 16	1 / 6	6 / 7	3 / 12
4 / 9	15	2 / 8	7 / 2
24	5 / 11	2 / 14	7 / 13
4, 8	1, 5, 9	2, 6	3, 7

Заметим, что свободными остались всего 4 позиции: 3, 4, 5, 15.

Результатом умножения на 15 может быть только числа 45 и 15 из 2-го столбика. Однако, цифры 3 в делителях этого столбика нет, значит, 45 нам не подходит, и на 15-й позиции находится 1.

Число 45 может быть цифрой 5 на 9-й позиции или цифрой 9 на 5-й. Однако, 9-я позиция уже занята. Значит, это 9 на 5-й позиции.

Число 16 может быть цифрой 4 на 4-й позиции или цифрой 8 на 2-й позиции. Однако, 2-я позиция уже занята. Значит, это 4 на 4-й.

Число 24 может быть цифрой 4 на 6-й позиции или цифрой 8 на 3-й позиции. Однако, 6-я позиция уже занята. Значит, это 8 на 3-й.

Таким образом, мы определили для каждой позиции цифру, которая на ней была и остаётся только аккуратно записать ответ: 2784916243537218.

6. Телекоммуникационные технологии (2 балла)

[Как-то связанные узлы]

Известно, что в некоторой сети зарегистрировано 15 узлов с различными адресами. Некоторые из этих узлов также управляют подсетями.

Номер	Адрес
1	192.168.106.167/32
2	192.168.106.162/32
3	192.168.106.180/30
4	192.168.106.160/27
5	192.168.106.179/32
6	192.168.106.166/32
7	192.168.106.163/29
8	192.168.106.176/28
9	192.168.106.182/32
10	192.168.106.178/32
11	192.168.106.161/28
12	192.168.106.177/32
13	192.168.106.181/32
14	192.168.106.165/32
15	192.168.106.164/32

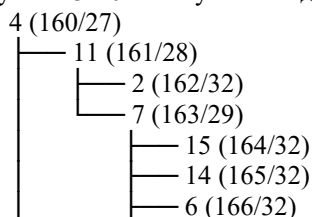
Восстановите карту сети и определите, сколько промежуточных узлов посетит сообщение, передаваемое из узла №2 в узел №13 по кратчайшему пути.

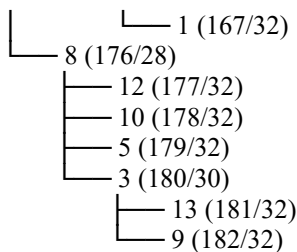
Ответ: 4

Решение:

Заметим, что т.к. подсеть должна вмещать в себя все находящиеся в ней узлы и широковещательный адрес, адреса с маской /32 не будут управлять подсетями, т.е. будут листьями в дереве нашей сети. Из этого также следует, что узел А, управляющий некоторой подсетью, которая содержит узел В, у который также управляет подсетью, должен иметь в маске меньше битов, соответствующих адресу сети.

Рассмотрим узлы с масками, отличными от 32 (3, 4, 7, 8, 11). Очевидно, что узел 4 с маской /27 может адресовать наибольшее число узлов, значит, он и будет корнем дерева. Заметим также, что его дети должны быть либо листьями, либо узлами, управляющими наибольшими возможными подсетями. Адреса 8 и 11 с масками /28 в таком случае должны быть детьми узла 4. Так как узел, управляющий подсетью, имеет наименьший адрес из всех узлов в этой подсети, становится ясно, что узлы 1, 2, 7, 15, 14, 6 и 1 лежат в подсети узла 11, а узлы 12, 10, 5, 3, 13 и 9 – в подсети узла 8. При этом узел 7 управляет собственной подсетью, которой не может принадлежать узел 2, но могут принадлежать все остальные узлы (15, 14, 6, 1). Аналогично, во втором поддереве (подсети) узлы 12, 10 и 5 не могут находиться в подсети узла 3, а узлы 13 и 9 – могут и находятся. Таким образом, мы восстанавливаем схему данной сети:





Остаётся только найти на ней узлы 2 и 13 и выяснить, что на пути между ними ровно 4 промежуточных узла – 11, 4, 8 и 3. Ответ – 4.

7. Технологии обработки информации в электронных таблицах, технологии сортировки и фильтрации данных (2 балла) [И больше, и меньше]

Дан фрагмент таблицы в режиме отображения формул:

	A	B	C
1	=COUNTIFS(B2:J30; ">"&B1; B2:J30; "<"&C1)		
2		1 =MOD(DIVIDE(\$A2 - MOD(\$A2; POW(10;COLUMN()-2)); POW(10;COLUMN()-2));10)	
3	=\$A2*2		

Формулу из ячейки A3 скопировали во все ячейки диапазона A3:A30. Формулу из ячейки B2 поместили во все ячейки диапазона B2:J30. В ячейки B1 и C1 поместили некоторые числа, в результате чего в ячейке A1 отобразилось число 69. Определите, какие числа были помещены в эти ячейки. В качестве ответа укажите через пробел два числа – число из ячейки B1 и число из ячейки C1.

Таблица соответствия имён используемых функций:

Google Sheets	Excel (eng)	Excel (ru)	LibreOffice
COUNTIFS	COUNTIFS	СЧЁТЕСЛИМН	COUNTIFS
MOD	MOD	ОСТАТ	MOD
DIVIDE	QUOTIENT или оператор /	ЧАСТНОЕ или оператор /	QUOTIENT или оператор /
COLUMN	COLUMN	СТОЛБЕЦ	COLUMN
POW	POWER	СТЕПЕНЬ	POWER

Ответ: 2 8

Решение:

Для решения данной задачи воспроизведём указанные формулы. Нетрудно заметить, что в диапазоне A3:A30 окажутся степени 2 по возрастанию, а в ячейках B2:J30 в каждой строке будут последовательно указаны разряды соответствующих чисел из столбца A и нули в случае отсутствия в числе из столбца A искомого разряда.

	A	B	C	D	E	F	G	H	I	J
1	69									
2	1	1	0	0	0	0	0	0	0	0
3	2	2	0	0	0	0	0	0	0	0
4	4	4	0	0	0	0	0	0	0	0
5	8	8	0	0	0	0	0	0	0	0
6	16	6	1	0	0	0	0	0	0	0
7	32	2	3	0	0	0	0	0	0	0
8	64	4	6	0	0	0	0	0	0	0
9	128	8	2	1	0	0	0	0	0	0
10	256	6	5	2	0	0	0	0	0	0

На изображении для наглядности приведены первые несколько строк таблицы.

В ячейке A1, соответственно, будет находиться количество цифр из диапазона B2:J30, больших числа в B1 и меньших числа в C1.

Чтобы найти ответ, добавим в таблицу несколько вспомогательных вычислений. С помощью формулы COUNTIF (СЧЁТЕСЛИ) вычислим, сколько раз в диапазоне встречается каждая из цифр от 0 до 9.

	L	M		L	M
1	0	=COUNTIF(\$B\$2:\$J\$30; "="&L1)	1	0	132
2	1	=COUNTIF(\$B\$2:\$J\$30; "="&L2)	2	1	18
3	2	=COUNTIF(\$B\$2:\$J\$30; "="&L3)	3	2	21
4	3	=COUNTIF(\$B\$2:\$J\$30; "="&L4)	4	3	12
5	4	=COUNTIF(\$B\$2:\$J\$30; "="&L5)	5	4	19
6	5	=COUNTIF(\$B\$2:\$J\$30; "="&L6)	6	5	11
7	6	=COUNTIF(\$B\$2:\$J\$30; "="&L7)	7	6	17
8	7	=COUNTIF(\$B\$2:\$J\$30; "="&L8)	8	7	10
9	8	=COUNTIF(\$B\$2:\$J\$30; "="&L9)	9	8	17
10	9	=COUNTIF(\$B\$2:\$J\$30; "="&L10)	10	9	4

Затем с помощью формулы SUM (СУММ) вычислим возможные значения A1 для всех корректных пар значений в B1 и C1 (значения B1 указаны по горизонтали, значения C1 указаны по вертикали).

	-1	0	1	2	3	4	5	6	7	8
1	=SUM(\$M\$1:\$M1)	-	-	-	-	-	-	-	-	-
2	=SUM(\$M\$1:\$M2)	=SUM(\$M\$2:M2)	-	-	-	-	-	-	-	-
3	=SUM(\$M\$1:\$M3)	=SUM(\$M\$2:M3)	=SUM(\$M\$3:M3)	-	-	-	-	-	-	-
4	=SUM(\$M\$1:\$M4)	=SUM(\$M\$2:M4)	=SUM(\$M\$3:M4)	=SUM(\$M\$4:M4)	-	-	-	-	-	-
5	=SUM(\$M\$1:\$M5)	=SUM(\$M\$2:M5)	=SUM(\$M\$3:M5)	=SUM(\$M\$4:M5)	=SUM(\$M\$5:M5)	-	-	-	-	-
6	=SUM(\$M\$1:\$M6)	=SUM(\$M\$2:M6)	=SUM(\$M\$3:M6)	=SUM(\$M\$4:M6)	=SUM(\$M\$5:M6)	=SUM(\$M\$6:M6)	-	-	-	-
7	=SUM(\$M\$1:\$M7)	=SUM(\$M\$2:M7)	=SUM(\$M\$3:M7)	=SUM(\$M\$4:M7)	=SUM(\$M\$5:M7)	=SUM(\$M\$6:M7)	=SUM(\$M\$7:M7)	-	-	-
8	=SUM(\$M\$1:\$M8)	=SUM(\$M\$2:M8)	=SUM(\$M\$3:M8)	=SUM(\$M\$4:M8)	=SUM(\$M\$5:M8)	=SUM(\$M\$6:M8)	=SUM(\$M\$7:M8)	=SUM(\$M\$8:M8)	-	-
9	=SUM(\$M\$1:\$M9)	=SUM(\$M\$2:M9)	=SUM(\$M\$3:M9)	=SUM(\$M\$4:M9)	=SUM(\$M\$5:M9)	=SUM(\$M\$6:M9)	=SUM(\$M\$7:M9)	=SUM(\$M\$8:M9)	=SUM(\$M\$9:M9)	-
10	=SUM(\$M\$1:\$M10)	=SUM(\$M\$2:M10)	=SUM(\$M\$3:M10)	=SUM(\$M\$4:M10)	=SUM(\$M\$5:M10)	=SUM(\$M\$6:M10)	=SUM(\$M\$7:M10)	=SUM(\$M\$8:M10)	=SUM(\$M\$9:M10)	=SUM(\$M\$10:M10)
	-1	0	1	2	3	4	5	6	7	8
1	132	-	-	-	-	-	-	-	-	-
2	150	18	-	-	-	-	-	-	-	-
3	171	39	21	-	-	-	-	-	-	-
4	183	51	33	12	-	-	-	-	-	-
5	202	70	52	31	19	-	-	-	-	-
6	213	81	63	42	30	11	-	-	-	-
7	230	98	80	59	47	28	17	-	-	-
8	240	108	90	69	57	38	27	10	-	-
9	257	125	107	86	74	55	44	27	17	-
10	261	129	111	90	78	59	48	31	21	4

Нетрудно заметить, что число 69 встречается только один раз – для пары B1=2, C1=8, то есть верным в данной задаче будет ответ 2 8.

8. Технологии программирования (3 балла)
[Химический парсер]

Имя входного файла	стандартный ввод
Имя выходного файла	стандартный вывод
Ограничение по времени	1 секунда
Ограничение по памяти	256 МБ

Перебирая изобретения великого учёного Д. И. Менделеева, Вы перенеслись в другую вселенную и стали его учеником. Но в один день, когда Менделеев отдыхал под яблоней, от порыва ветра на него свалилось пару яблок, после чего великий ученый забыл, как раскрывать скобки в химических формулах.

В его записях вы вычитали, что в этом мире существуют всего 26 элементов, обозначаются они заглавными буквами латинского алфавита, сами же формулы строятся по следующему принципу:

Формула — последовательность латинских букв, разделенных числами, которые обозначают количество элемента, стоящего перед числом; если же числа нет, то количество элемента равно **единице**. Пример: H4ZO2 эквивалентно H4O2Z1.

Также в формуле могут стоять скобки, которые показывают, что количество элементов внутри скобок должно быть умножено на число после скобок. Так, H2(O2Z)3 — это то же самое, что и H2O6Z3.

Напишите программу, которая поможет Менделееву раскрывать скобки в химических формулах.

Формат входных данных

На ввод поддается строка, состоящая из латинских символов, скобок и цифр. Длина строки не превышает 1000. Строка всегда является правильной формулой.

Формат выходных данных

Выведите строку содержащую правильно раскрытую формулу из ввода. Символы должны выписываться в отсортированном относительно алфавита виде.

Примеры

Стандартный ввод	Стандартный вывод
Z3 (O (N2O3) 2) 3	N12O21Z3
N (OY3) 2	N1O2Y6
H4ZO2	H4O2Z1

Комментарий

При выводе ответа после каждой буквы обязательно выводится число, обозначающее количество соответствующего элемента.

Химические элементы должны быть выведены в отсортированном порядке.

Подсказка: для обработки вложенных скобок удобно использовать стек.

Решение:

Заметим, что формулу удобно разбирать слева направо, а для каждой глубины вложенности скобок хранить отдельные количества элементов.

Так как элементов всего 26, на каждом уровне вложенности достаточно хранить массив из 26 чисел. При встрече буквы мы читаем число после неё. Если числа нет, то количество равно 1. При встрече (начинаем новую группу и кладём в стек новый пустой массив. При встрече) читаем множитель после скобки. Если числа нет, множитель равен 1. Затем снимаем верхний массив со стека, умножаем все его значения на этот множитель и прибавляем к предыдущему уровню.

После обработки всей строки внизу стека останутся итоговые количества всех элементов. Остаётся вывести их в алфавитном порядке.

Для второго варианта решение практически идентично первому. Единственное отличие состоит в том, что если после буквы или закрывающей скобки число отсутствует, то вместо множителя 1 используется множитель 2. Поэтому в коде достаточно заменить функцию чтения числа по умолчанию. Если чисел нет, она должна возвращать 2, а не 1.

9. Технологии программирования (4 балла)

[Дима и забытая функция]

Имя входного файла	стандартный ввод
Имя выходного файла	стандартный вывод
Ограничение по времени	1 секунда
Ограничение по памяти	256 МБ

Дима плохо подготовился к тесту по строковым алгоритмам. Он отлично знает, как построить Z-функцию, но вот про префикс-функцию забыл всё, что когда-либо знал.

Вы давно дружите с Димой и решили помочь ему. Он передал вам листок, на котором написал свою Z-функцию. Помогите ему построить по ней префикс-функцию.

Формат входных данных

В первой строке содержится целое число n ($n \leq 10^5$) - длина Z-функции Димы. Во второй строке содержится сама функция.

Формат выходных данных

Выведите полученную префикс-функцию.

Пример

Стандартный ввод	Стандартный вывод
8	0 0 1 0 1 2 3 1
8 0 1 0 3 0 1 1	

Комментарий

Префикс-функция строки s длины n — это массив $\pi[0 \dots n-1]$, где значение $\pi[i]$ равно длине наибольшего собственного префикса подстроки $s[0 \dots i]$, который одновременно является её суффиксом. Более формально:

$$\pi[i] = \max\{k \mid 0 < k \leq i, s[0 \dots k-1] = s[i-k+1 \dots i]\}$$

$s[0 \dots k-1]$ — префикс строки, $s[i-k+1 \dots i]$ — суффикс подстроки $s[0 \dots i]$.

Z-функция строки s длины n — это массив $z[0 \dots n-1]$, где значение $z[i]$ равно длине наибольшего префикса строки s , совпадающего с подстрокой, начинающейся в позиции i . Более формально:

$$z[i] = \max\{k \mid s[0 \dots k-1] = s[i \dots i+k-1]\}$$

Считается, что $z[0] = n$, где n — длина строки.

Решение:

Заметим, что напрямую восстанавливать Z-функцию по префикс-функции необязательно. Достаточно сначала построить любую строку, для которой заданный массив π является корректной префикс-функцией, а затем уже обычным алгоритмом посчитать для этой строки Z-функцию.

Строку удобно строить слева направо. Первый символ можно зафиксировать как **a**. Если $\pi[i] > 0$, то символ в позиции i уже однозначно определяется: он должен быть равен $s[\pi[i]-1]$, потому что на позиции i должен продолжаться префикс длины $\pi[i]$, совпадающий с суффиксом текущей строки. Именно поэтому в этом случае мы просто дописываем $s[\text{prefix_mas}[i]-1]$.

Если же $\pi[i] = 0$, то нужно выбрать такой символ, чтобы после его добавления у строки не появился никакой непустой префикс, совпадающий с её суффиксом. Для этого рассмотрим всю цепочку длин таких префиксов для строки $s[0 \dots i-1]$: сначала $\pi[i-1]$, затем $\pi[\pi[i-1]-1]$, и так далее, пока не дойдём до нуля. Если длина текущего такого префикса равна x , то символ $s[x]$ запрещён, потому что именно он продлит это совпадение ещё на один символ. Кроме того, отдельно запрещаем $s[0]$, иначе получится совпадение длины 1. После этого можно взять любую букву, которая не запрещена.

После того как строка восстановлена, остаётся посчитать её Z-функцию стандартным алгоритмом.

Аналогичная идея используется и в другом варианте, при переводе из Z-функции в префикс-функцию. Восстанавливать префикс функцию напрямую по массиву z необязательно. Можно сначала построить любую строку, для которой заданный массив z является корректной Z-функцией, а затем уже стандартным алгоритмом вычислить для этой строки префикс-функцию.

Отборочный этап. Первый тур (приведен один из вариантов заданий)

1. Кодирование информации. Системы счисления (3 балла)

[Несколько возможных периодов]

Сколько существует чисел X_2 (2 – основание системы счисления), таких, что $0.17_8 < X_2 < 0.89_{12}$ и при этом X является периодической дробью без непериодической части с длиной периода 4? В качестве ответа укажите одно натуральное число – количество таких дробей.

Примечание: в рамках данной задачи будем считать корректными все дроби, которые можно записать, используя период длины 4. Например, хотя дробь $0.(0101)_2$ можно также записать как $0.(01)_2$, мы будем считать такую дробь периодической дробью с периодом 4.

Ответ: 7

2. Кодирование информации. Количество информации. Элементы комбинаторики (2 балла)

[Рубрика «Эксперименты»]

В мешочке содержатся 5 красных шариков, 12 зелёных и 7 синих. Существует 5 сообщений о результате эксперимента, в каждом из которых доставали шарик из мешочка. Все эксперименты проводились независимо и имели одинаковое начальное состояние, в рамках каждого эксперимента шарик доставали последовательно.

- Из мешочка достали 3 шарика, они все оказались разных цветов
- Из мешочка достали 3 шарика, они все оказались зелёными
- Из мешочка достали 3 шарика, два из них оказались одинакового цвета
- Из мешочка достали 5 шариков, три случайных шарика положили обратно, оставшиеся два были разных цветов
- Из мешочка достали 5 шариков, никакие три из них не были одного цвета

Расположите номера этих сообщений в порядке возрастания количества информации, содержащегося в каждом сообщении. В ответ запишите последовательность чисел от 1 до 5, не разделяя их пробелами.

Пример записи ответа: 54321.

Ответ: 34512

3. Кодирование информации. Количество информации. Кодирование текста (1 балл)

[Админ-оптимизатор]

В некоторой системе логины и пароли пользователей кодируются посимвольно с использованием минимально возможного, одинакового для всех символов количества бит на символ. В логине пользователя могут использоваться строчные буквы современного английского алфавита, арабские цифры и пробелы. Длина логина пользователя должна быть равна 20 символам. Пользовательские пароли могут содержать строчные и заглавные буквы, цифры. Однако, при кодировании паролей была допущена ошибка и использовался современный немецкий алфавит (в нём 30 символов) вместо современного английского. Администратор исправил ошибку в кодировании паролей. Он также решил, что в случае логинов будет удобнее, если кодировать не каждый символ в отдельности, а кодировать каждый уникальный логин используя минимальное, одинаковое целое для всех возможных логинов, количество бит на логин целиком. Определите, на сколько бит уменьшилось количество информации, необходимое для хранения информации о пользователе с паролем длиной 16 символов. Если количество бит увеличилось, напишите отрицательное число.

Ответ: 31

4. Кодирование информации. Объем данных (2 балла)

[Фото в багровых тонах]

Дано изображение разрешением 1920x1080 с глубиной цвета 9 бит на каждый из трёх цветовых каналов. К нему применяется цветовой фильтр, который влияет на глубину цвета, изменяя значение выбранного цветового канала по следующим правилам:

- Если текущее значение цветового канала меньше 256, то к нему прибавляется максимальная возможная степень двойки, такая, чтобы сумма была не больше, чем 256.
- Если текущее значение цветового канала больше 256, из него вычитается максимальная возможная степень двойки, такая, чтобы разность была не меньше, чем 256.

Каждый пиксель кодируется следующим образом: для записи цвета пикселя для каждого из цветовых каналов значение кодируется, используя минимальное возможное количество бит. После чего все 3 закодированных значения записываются в память. На сколько бит уменьшится размер хранимого изображения, если применить данный фильтр к красному каналу?

Ответ: 2073600

5. Основы логики. Анализ логических функций (1 балл)

[Зависимые функции]

Даны две логические функции, зависящие от одних и тех же переменных:

$$F(A, B, C) = (A \wedge G(A, B, C) \vee B) \text{ XOR } (G(A, B, C) \rightarrow C)$$

и

$$G(A, B, C) = \bar{A} \rightarrow (B \rightarrow A \wedge C).$$

Сколько существует различных наборов переменных, при которых функция F принимает значение ИСТИНА и функция G принимает значение ИСТИНА?

Ответ: 3

6. Основы логики. Упрощение логического выражения (2 балла)

[Куда уж проще]

Упростите логическое выражение или укажите его результат (при его однозначности).

$$(A \vee B) \wedge (A \vee C) \vee (\overline{A \vee B}) \wedge (B \vee C)$$

Результат упрощения может содержать только операнды, операции инверсии, конъюнкции и дизъюнкции и не должен содержать скобок. Будем считать выражение упрощенным, если не существует эквивалентного ему выражения, содержащего меньшее количество логических операций и скобок.

Комментарий по вводу ответа: операнды вводятся большими латинскими буквами; логические операции обозначаются, соответственно, как not, and и or.

При однозначном ответе – истинный ответ обозначается как 1, а ложный как 0.

Пример записи ответа: A or B and not C

Ответ: A or C || C or A

7. Основы логики. Синтез логического выражения (2 балла)

[По таблице]

Дана таблица истинности некоторого логической функции F, зависящей от трёх переменных – X, Y и Z.

X	Y	Z	F
0	0	0	ИСТИНА
0	0	1	ЛОЖЬ
0	1	0	ИСТИНА
0	1	1	ЛОЖЬ
1	0	0	ИСТИНА
1	0	1	ЛОЖЬ
1	1	0	ИСТИНА
1	1	1	ИСТИНА

Восстановите функцию F и запишите, используя минимальное число операций.

Пример записи ответа: (X or not Y) and Z

Примечание: переменные вводятся большими латинскими буквами; логические операции обозначаются, соответственно, not, and и or. Скобки используются только для изменения порядка выполнения операций. Если порядок выполнения операций очевиден из их приоритетов, дополнительное использование скобок считается ошибкой. Пробелы ставятся между логическими операциями и переменными.

Ответ: X and Y or not Z || Y and X or not Z || not Z or X and Y || not Z or Y and X

8. Алгоритмизация и программирование. Формальный исполнитель (2 балла)

[Альбомная и портретная]

Ксюша решила распечатать свои фотографии и поместить их в альбом, на каждую страницу которого помещаются ровно 3 фотографии. Последовательно каждую фотографию она печатает, затем определяет ориентацию фотографии – альбомная или портретная – и кладёт фотографию в альбом по следующим правилам:

1. Если фотография альбомная и в альбоме есть страница, на которой уже размещены 1 или 2 альбомных фотографии, она добавляет фотографию на эту страницу и переходит к следующей фотографии.
2. Если фотография портретная и в альбоме есть страница, на которой уже размещены 1 или 2 портретных фотографии, она добавляет фотографию на эту страницу и переходит к следующей фотографии.
3. Если страницы с фотографиями того же типа нет, то она размещает фотографию на первой свободной странице, т.е. начинает новую страницу, даже если это самая первая фотография.

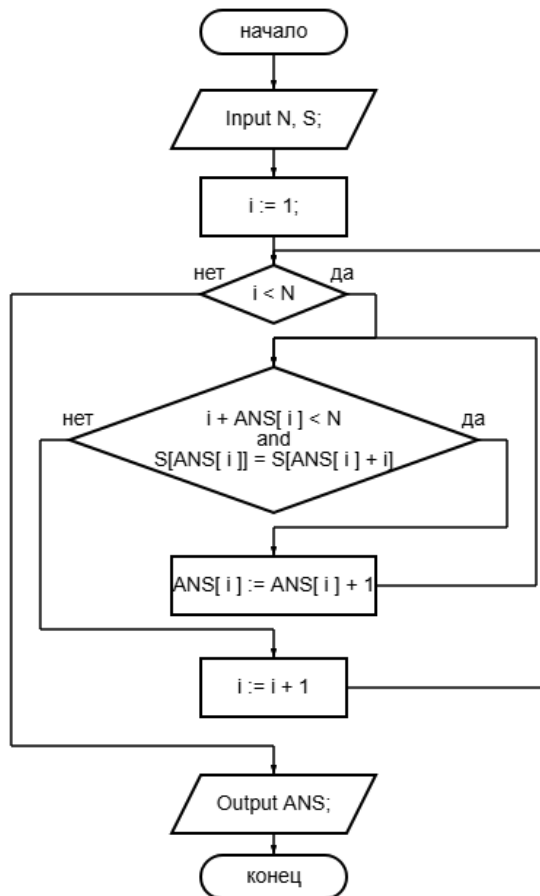
Ксюша заметила, что распечатанные фотографии идут повторяющейся последовательностью ПААААПАА, где А – альбомная, П – портретная. Сколько раз у Ксюши возникнет ситуация, в которой ей нужно будет воспользоваться пунктом 3 своих правил, если она поместит в альбом 350 фотографий?

Ответ: 118

9. Алгоритмизация и программирование. Блок-схема, массивы, обратная задача (3 балла)

[Фоторобот подозреваемого]

Дана блок-схема алгоритма:



Массив ANS изначально заполнен нолями. На вход данному алгоритму была подана строка S длины N=15, в результате чего была выведена строка из N чисел 0 0 0 3 0 0 0 1 0 6 0 0 3 0 0. Известно, что исходная строка S была составлена из символов a, b и c (все 3 вида символов в строке должны быть). Определите, какая строка могла быть подана на вход алгоритму?

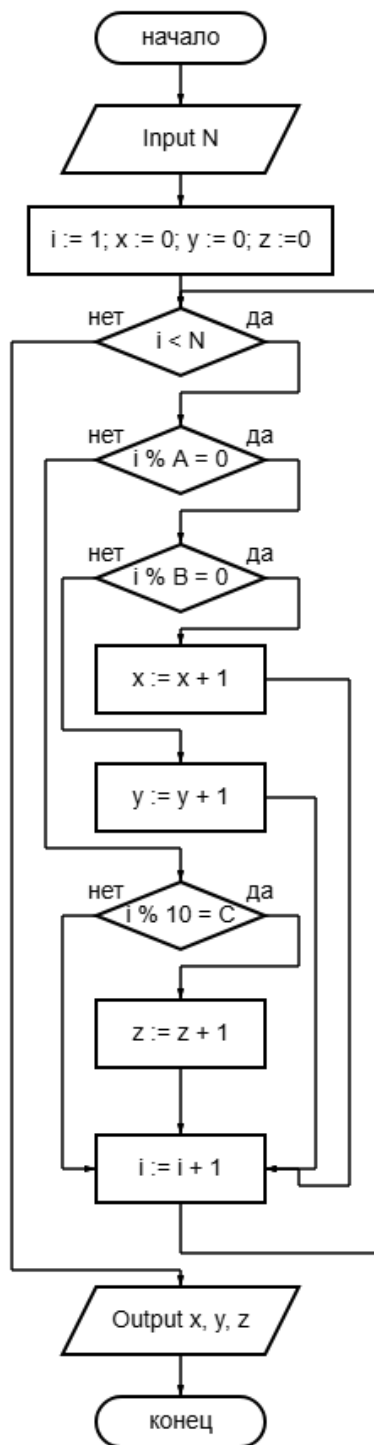
Примечание: если на вход могло быть подано несколько различных строк, укажите в ответе лексикографически минимальную из них (то есть такую, которая идёт раньше всех остальных, если отсортировать строки в алфавитном порядке).

Ответ: abbabbbacabbabb

10. Алгоритмизация и программирование. Блок-схема, обратная задача (1 балл)

[Три сита]

Дана блок-схема алгоритма:



Определите, какое минимальное число N могло быть подано алгоритму на вход, если $A = 3$, $B = 4$, $C = 3$, а на выходе был получен результат 99 300 80.

Примечание: операция $A \% B$ означает взятие остатка от целочисленного деления A на B.

Ответ: 1198

Отборочный этап. Второй тур (приведен один из вариантов заданий)

1. Электронные таблицы. Адресация ячеек и вычисления (2 балла)

[True? True!]

Дан фрагмент электронной таблицы в режиме отображения формул. В ячейку A1 поместили некоторое натуральное число.

	A	B	C	D
1				
2		$0 = \text{POW}(3; \$A2)$	$= \text{SWITCH}(\text{TRUE}(); \$A\$1 - \text{SUM}(\$D3:\$D); \geq 2 * \$B2; 2; \$A\$1 - \text{SUM}(\$D3:\$D) \geq \$B2; 1; 0)$	$= \$B2 * \$C2$
3	$= \$A2 + 1$			

Формулу из ячейки A3 скопировали во все ячейки диапазона A4:A32, формулы из ячеек B2, C2 и D2 скопировали во все ячейки диапазонов B3:B32, C3:C32 и D3:D32 соответственно. Определите наименьшее возможное значение в ячейке A1, если известно, что в столбике C оказалось ровно 7 единиц и 13 двоек.

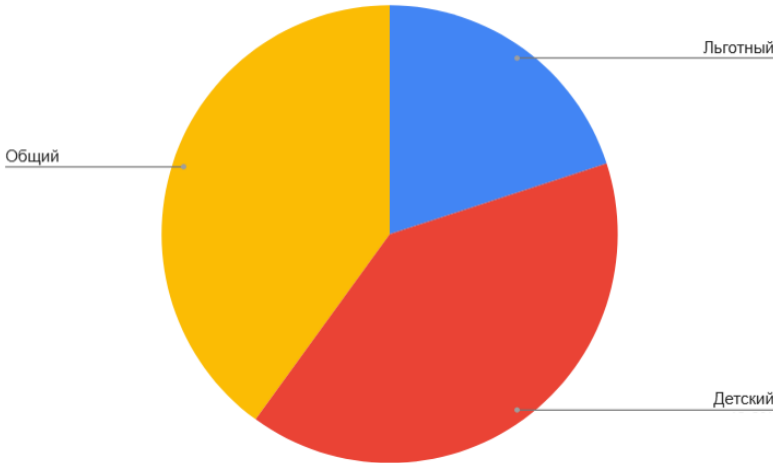
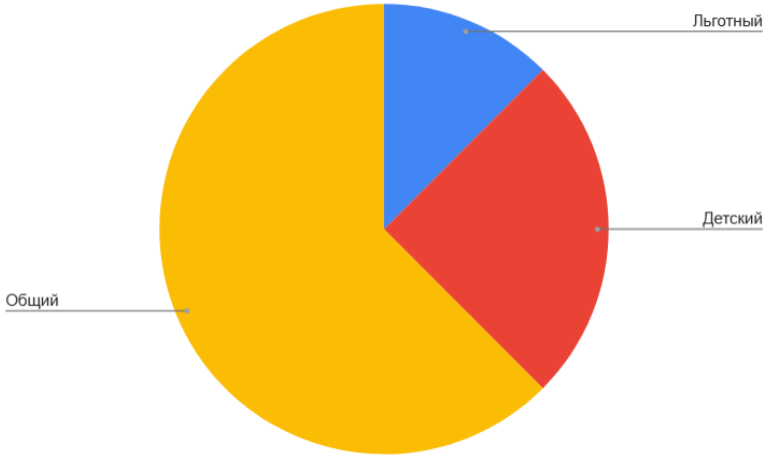
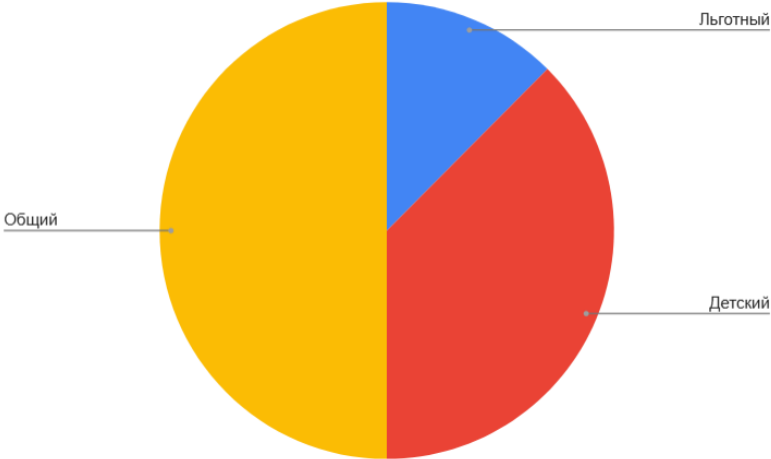
Google Sheets	Excel русский	LibreOffice	Excel английский
POW	СТЕПЕНЬ	POWER	POWER
SWITCH	ПЕРЕКЛЮЧ	SWITCH	SWITCH
TRUE	ИСТИНА	TRUE	TRUE

Ответ: 1744189361

2. Электронные таблицы. Графики и диаграммы (1 балл)

[Точные данные съела собака]

В зоопарке продаются 3 типа входных билетов: Детские, Общие и Льготные. Сотрудники зоопарка составили для трёх различных месяцев диаграммы распределения билетов по типам. Однако, эти диаграммы были утрачены сотрудниками, и они смогли только приблизительно восстановить по памяти их вид и составили новые диаграммы, с минимальной используемой долей в 1/8 круга, а не в 1%.



Также внимательные сотрудники нашли ведомость о числе льготных билетов в каждый из месяцев: 175, 103, 274. Определите среднее число общих билетов в месяц за данный трёхмесячный период. Ответ округлите до двух знаков после запятой.

Ответ: 542,00

3. Сортировка и фильтрация данных (2 балла)

[Кто пустил жадин в подземелье?]

Два любителя ценностей - Даня и Аня - отправились в подземелье на поиски сокровищ. Как опытные любители ценностей, они составили список всего найденного. Их опыт позволяет безошибочно определить стоимость каждого сокровища в монетах и его вес в килограммах.

ID	Цена	Вес
1	2944	46
2	1612	31
3	1248	16
4	516	6
5	284	4
6	1073	29
7	1550	50
8	1075	25
9	352	8
10	1547	17
11	2555	35
12	1078	49
13	1920	20
14	1881	19
15	1152	48
16	360	5
17	2185	23
18	1680	21
19	2448	34
20	248	31

Однако Даня и Аня столкнулись с проблемой: они могут унести с собой только 100 кг добычи. Даня предложил отсортировать список по убыванию ценности и класть в рюкзак самые дорогие предметы, которые может положить. Аня решила, что Даня действует слишком необдуманно и предложила отсортировать список по убыванию цены за килограмм, после чего класть в рюкзак предметы с самым большим соотношением, которые может положить. Помогите Ане проверить свою тактику - вычислите, на сколько монет больше она получит. Если Аня получит меньше монет, чем Даня, укажите отрицательное число.

Ответ: 1313

4. Поиск и фильтрация данных (1 балл)

[Отрезки и прямоугольник]

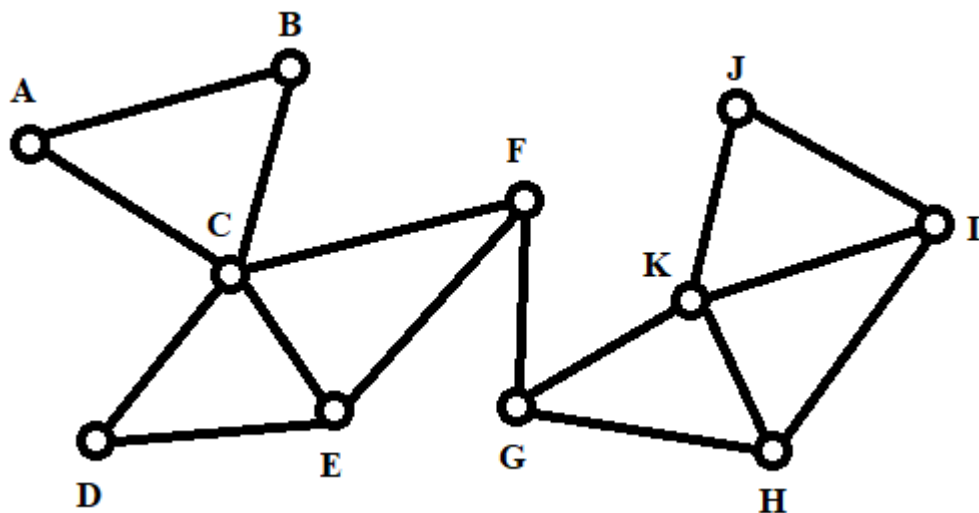
Дан набор отрезков, расположенных на прямых $y=5$ и $y=10$. Отрезки на $y=5$ определяются следующим образом: отрезок начинается в точках $x=x_1$ (x_1 – натуральное число) и имеет длину $2 \cdot x_1$. Отрезки на прямой $y=10$ начинаются в точках x_2 (x_2 – натуральное чётное) и имеют длину x_2 . Дан прямоугольник с координатами углов $\{1000; 1\}$, $\{2000; 1\}$, $\{2000; 15\}$, $\{1000; 15\}$. Определите, сколько отрезков пересекают этот прямоугольник (то есть, начинаются раньше него и заканчиваются позже) и сколько отрезков имеют начало или конец во внутренней области (не включающей в себя границы) данного прямоугольника. В ответе укажите через пробел два искомых числа через пробел.

Ответ: 333 2080

5. Телекоммуникационные технологии (2 балла)

[Пара ключей]

Дана сеть узлов:



Проход по ребру между узлами может быть выполнен, если ключ, с которым осуществляется проход, подходит под маску, соответствующую этому ребру. На месте * должны находиться 1 или более символов.

Ребро	Маска	Ребро	Маска	Ребро	Маска	Ребро	Маска
AB	*00*	CE	1*0	FG	*01*	HK	1*1
AC	*101	CF	*1*	GH	*0*0*	IJ	11*
BC	*1*1	DE	11*110	GK	*0*	IK	10*11
CD	10*111	EF	100*	HI	*0	JK	*11*

Даны 5 наборов десятичных чисел, которые, будучи переведёнными в двоичную систему счисления, могут быть использованы в качестве ключей. Для прохода по каждому из рёбер может быть использован любой из ключей используемого набора.

1. 6, 153
2. 11, 183
3. 25, 70
4. 42, 27
5. 115, 30

Определите, каким набором можно посетить наименьшее число вершин, если начинать из вершины F. В качестве ответа укажите набор и список недостижимых вершин в алфавитном порядке без пробелов. В случае, если таких наборов несколько, укажите вариант с наименьшим номером.

Пример записи ответа: 3 ABC

Ответ: 2 AE

6. Операционные системы (3 балла)

[На первый-второй-третий рассчитайтесь]

Прототип процессора имеет 3 уровня кэшей: L1 - самый маленький самый быстрый, L2 – имеет больший объём, но меньшую скорость взаимодействия и L3 – ещё больший объём и ещё меньшая скорость. Данные, которые не удалось поместить в кэш или которые были вытеснены из него, хранятся в оперативной памяти. Для всех уровней кэша существует процедура очистки. Процедура проводится каждые N секунд (N различно для каждого уровня кэша) - в этом случае на уровень ниже перемещаются все данные, которые не были востребованы за последние N секунд, либо в случае недостатка места в кэше – в этом случае фрагменты данных в порядке убывания количества секунд с момента последнего использования (т.е. начиная с тех, что были использованы наиболее давно) перемещаются на уровень ниже до тех пор, пока свободного места не станет достаточно.

В случае, если процессору требуются некоторые данные, он сначала ищет их в кэше L1, затем в L2, затем в L3, затем в RAM. При этом данные перемещаются на уровень выше (для RAM уровнем выше будет кэш L3, для кэша L3 – кэш L2, для кэша L2 – кэш L1), если такое перемещение возможно (размер фрагмента данных не должен превышать размер кэша). Если операции чтения и перемещения должны произойти одновременно, сначала произведётся операция перемещения, затем операция чтения. Количество секунд, необходимых для чтения, округляется вверх до ближайшего целого.

При этом, если уровень, на который необходимо совершить перемещение, не имеет достаточно свободного места, проводится описанная выше процедура очистки. Если в кэш уровнем ниже невозможно перенести фрагмент данных, в нём также запускается процедура очистки. Если время N тайм-аута совпадает с чтением файла, то вначале производится чтение файла, затем очистка кэша.

Переместить данные в оперативную память возможно всегда. Так как в данной задаче речь идёт о грубом приближении работы процессора, будем считать процессор однопоточным, а перемещение фрагментов данных мгновенным.

Ниже представлена таблица с характеристиками кэшей:

Уровень кэша	Размер кэша, Кбайты	Скорость чтения, Кбайт/с	N, тайм-аут автоматической очистки, с
L1	64	12	50

L2	512	8	250
L3	8192	6	1000
RAM	Бесконечно	1	-

На момент начала работы процессора все 3 кэша пусты. Процессор 5 раз подряд последовательно запрашивает следующие фрагменты данных:

№ фрагмента данных	Размер фрагмента данных, Кбайт
1	53
2	11
3	59
4	6
5	35
6	123
7	1096
8	48
9	95
10	23

Определите суммарный объём фрагментов данных в каждом из кэшей после окончания работы в КБайт. В ответ запишите через пробел 3 числа: количество данных в L1, L2 и L3.

Ответ: 23 430 1096

7. Технологии программирования (2 балла)

[Скобочная последовательность]

Имя входного файла	стандартный ввод
Имя выходного файла	стандартный вывод
Ограничение по времени	1 секунда
Ограничение по памяти	256 МБ

Петя собрал коллекцию из N строк, каждая из которых является последовательностью круглых скобок (символы "(" и ")").

Назовём последовательность скобок сбалансированной, если при просмотре слева направо число открывающих скобок никогда не меньше числа закрывающих, а в конце эти количества равны.

Петя просит у вас помощи. Он хочет составить как можно больше пар из данных строк. Для пары строк (A, B) он берёт их в указанном порядке и склеивает в одну строку $A + B$. Пара считается хорошей, если полученная строка является сбалансированной.

Каждая исходная строка может входить не более чем в одну пару.

Требуется определить максимальное количество хороших пар, которое можно составить.

Формат входных данных

В первой строке дано целое число N ($1 \leq N \leq 50000$) количество скобочных последовательностей.

В каждой из следующих N строк записана одна скобочная последовательность, состоящая только из символов "(" и ")".

Длина каждой последовательности не превосходит 100.

Формат выходных данных

Выведите одно целое число — максимальное количество пар последовательностей, которые можно выбрать и упорядочить так, чтобы склейка каждой пары давала сбалансированную скобочную последовательность.

Примеры

Стандартный ввод	Стандартный вывод
7) ()) ((((())	2
4 (() ()) ()))	1

Комментарий

Сбалансированная скобочная последовательность — это такая последовательность скобок, в которой для каждой открывающей скобки найдётся соответствующая закрывающая, и при этом закрывающая скобка всегда идёт после своей открывающей.

8. Технологии программирования (3 балла)

[ОТЕЛЬ]

Имя входного файла	стандартный ввод
Имя выходного файла	стандартный вывод
Ограничение по времени	1 секунда
Ограничение по памяти	256 МБ

В старом отеле есть очень длинный коридор из n комнат. Отель полностью заполнен, и в каждой комнате уже живёт некоторое количество людей.

У управляющего была странная привычка. Он следил за тем, чтобы в следующей комнате жило не меньше людей, чем в любой из предыдущих. При этом он всегда подписывал, сколько человек он заселил в ту или иную комнату. Со временем свободных номеров не осталось, и гости перестали прибывать.

Однажды в отель приехал инспектор. Коридор был длинный, комнаты тянулись одна за другой, и в каждой из них жило разное, а иногда одинаковое количество людей.

Для отчёта инспектору нужно проверить t комнат. Каждый раз он будет называть число x . Ваша задача — найти самую правую комнату, в которой живёт ровно x человек. Если такой комнаты в отеле не окажется, инспектор ставит прочерк в отчёте и продолжает проверку.

Формат входных данных

В первой строке записано целое число n ($1 \leq n \leq 500000$) — количество комнат в отеле.

Во второй строке записаны n целых чисел $a_0, a_1, \dots, a_{n-1}$, отсортированных по неубыванию — количество человек в каждой комнате.

В третьей строке записано целое число t ($1 \leq t \leq 10000$) — количество запросов на поиск комнаты.

В следующих t строках записано по одному целому числу x ($x \leq 10^9$) — количество человек в комнате, которую ищет инспектор.

Формат выходных данных

Для каждого из t запросов выведите одно целое число — индекс комнаты, в которой количество человек равно x . Если такой комнаты не существует, выведите -1 .

Пример

Стандартный ввод	Стандартный вывод
6 1 2 4 4 4 7 1 4	4

Задания для 5–8 класса

Заключительный этап (приведен один из вариантов заданий)

1. Архитектура компьютера (1 балл)

[Хитрый декодер]

Процессор читает инструкции переменной длины и работает в две стадии:

1) Выборка

За 1 такт процессор может считать из памяти ровно 8 байт (даже если какая-то инструкция при этом считается не полностью) и положить их в буфер выборки. Размер буфера выборки - 16 байт.

Буфер выборки работает как циклическая очередь. Если в буфере нет места для новых данных, они записываются поверх старых, уже декодированных данных, начиная с начала буфера, что позволяет не терять последние считанные байты. Точно также – циклически – происходит последующее чтение данных из буфера.

Если в буфере нет места для ещё 8 байт (в буфере уже находится более 8 ещё не декодированных байт), выборка в этот такт не выполняется.

Байты поступают в конец буфера и читаются с начала, как в очереди.

2) Декодирование

Инструкции декодируются строго по порядку.

Время декодирования зависит от длины инструкции:

- 2 байта = 1 такт
- 4 байта = 2 такта
- 8 байтов = 3 такта

Выборка и декодирование могут идти параллельно (в один и тот же такт). В один такт можно положить инструкцию в буфер и сразу начать её декодирование.

В начале работы буфер пуст, процессор начинает с выборки.

Программа состоит из 1000 инструкций:

- 40% - длиной 2 байта
- 30% - длиной 4 байта
- 30% - длиной 8 байт

Задание:

Найдите общее количество тактов до момента, когда последняя инструкция будет полностью декодирована.

Ответ: 1900

Решение:

Посчитаем сколько есть инструкций каждой длины и определим размер программы:

$$400 \cdot 2 + 300 \cdot 4 + 300 \cdot 8 = 4400 \text{ байт}$$

Определим количество тактов для выборки:

$$4400 / 8 = 550 \text{ тактов.}$$

Определим количество тактов для декодирования и получим ответ:

$$400 \cdot 1 + 300 \cdot 2 + 300 \cdot 3 = 1900 \text{ тактов}$$

2. Кодирование информации (1 балл)

[Образовательная программа]

Света и Костя обсуждают новую образовательную программу в переписке. Формулировки важны, а интернет иногда «шалит»: при передаче сообщения один бит может исказиться. Чтобы восстанавливать смысл, они решили кодировать каждый фрагмент сообщения кодом Хэмминга (7,4), который умеет исправлять ровно одну ошибку.

Каждый фрагмент состоит из 4 информационных битов `d1 d2 d3 d4`. Они кодируются в 7-битное слово, где позиции нумеруются слева направо от 1 до 7:

- позиции 1, 2, 4 — проверочные биты `p1, p2, p4`
- позиции 3, 5, 6, 7 — данные `d1, d2, d3, d4`

Данные представляют собой 4 бита, значения которых могут быть как 0, так и 1.

То есть буква имеет вид (индексация с 1):

Позиция	1	2	3	4	5	6	7
Бит	p1	p2	d1	p4	d2	d3	d4

Проверка:

Для проверки сначала вычисляются $s1, s2, s4$ используя XOR (Исключающее ИЛИ, обозначается как $\oplus$):

$$s1 = p1 \oplus d1 \oplus d2 \oplus d4 \text{ (позиции 1,3,5,7)}$$

$$s2 = p2 \oplus d1 \oplus d3 \oplus d4 \text{ (позиции 2,3,6,7)}$$

$$s4 = p4 \oplus d2 \oplus d3 \oplus d4 \text{ (позиции 4,5,6,7)}$$

Далее рассчитывается Синдром ошибки S :

$$S = s1 \cdot 1 + s2 \cdot 2 + s4 \cdot 4.$$

- если $S = 0$, ошибки нет

- иначе ошибочен бит в позиции S (его нужно инвертировать: $0 \rightarrow 1$ или $1 \rightarrow 0$).

Гарантируется, что в каждом принятом фрагменте либо нет ошибок, либо ровно одна ошибка на фрагмент.

Вам дано закодированное слово (состоит из нескольких фрагментов):

1010101 0111110 1100000 0011001 0110101

В ответ запишите пару чисел “номер фрагмента с ошибкой” и ”номер бита с ошибкой в этом фрагменте” (без пробела между этими двумя числами), если таких пар несколько, то запишите эти пары через пробел в том же порядке, в котором фрагменты даны в условии (индексация с 1 как для бита, так и для номера фрагмента)

Пример записи ответа: 11 27

Примечание: Таблица истинности для XOR (Исключающего ИЛИ):

a	b	a XOR b
0	0	0
0	1	1
1	0	1
1	1	0

Ответ: 26 33 53

Решение:

Фрагменты (без ошибок):

1010101 -> к

0111100 -> у

1110000 -> р

0011001 -> с

0100101 -> ы

Фрагменты (с ошибками):

1010101

0111110 -> ошибка в бите 6

1100000 -> ошибка в бите 3

0011001

0110101 -> ошибка в бите 3

3. Системы счисления (2 балла)

[Новая система счисления?]

Вася на уроке информатики узнал про различные системы счисления, и, вдохновившись, придумал свой собственный способ кодирования чисел.

Для кодирования числа n подбирается такая сумма степеней двойки, чтобы в сумме получилось число n . При этом каждую из степеней двойки можно домножить на любую цифру от 0 (включительно) до 9 (включительно). Таким образом, многие числа можно представить как сумму степеней, домноженных на разные цифры разными способами, например:

$$39 = 2^5 * 1 + 2^0 * 7 = 2^4 * 2 + 2^0 * 7 = 2^4 * 2 + 2^2 * 1 + 2^0 * 3 = \text{и так далее} \dots$$

После того, как для числа n подобраны степени двойки и цифры, на которые эти степени домножаются, определяется максимальная степень двойки, которая участвует в сумме, и к этому числу прибавляется «1» – столько разрядов будет в финальном числе, обозначим это количество разрядов как r .

Далее разряды закодированного числа нумеруются с 0 и до $r-1$. На каждую позицию записывается цифра, на которую домножалась соответствующая степень двойки. Например, если сумма 39 была подобрана как $2^5 * 1 + 2^0 * 7$, то количество разрядов в закодированной записи $r = 5+1=6$, и число, закодированное Васиным способом, записывается в виде 700001:

№ разряда	0	1	2	3	4	5
На что домножали степень 2, соответствующую № разряда	7	0	0	0	0	1

А если сумма 39 была подобрана как $2^4 * 2 + 2^2 * 1 + 2^0 * 3$, то число будет записано уже в виде 30102

№ разряда	0	1	2	3	4
На что домножали степень 2, соответствующую № разряда	3	0	1	0	2

Теперь Вася задался вопросом, как закодировать своим способом число 71 так, чтобы сумма цифр результата кодирования при интерпретировании его как десятичного числа, была минимальным возможным числом.

В ответ запишите одно число - результат кодирования.

Пример: Для числа 2 возможно две записи числа:

$$2 = 2^1 * 1 \text{ (сумма степеней)} = 01 \text{ (запись с помощью Васиного способа);}$$

$$2 = 2^0 * 2 \text{ (сумма степеней)} = 2 \text{ (запись с помощью Васиного способа);}$$

При интерпретации чисел 01 и 2 в десятичной системе, получаем суммы цифр $0+1=1$ и 2, из них минимально число 1, значит в ответ было бы нужно записать запись «01»

Ответ: 1110001

Решение:

Чтобы сумма цифр в записи была минимальна, нужно чтобы в сумме, которую мы составляем, было как можно меньше слагаемых. Чтобы этого достичь, будем рассматривать все степени двойки, меньшие числа 71, и брать в сумму все из них, которые получатся, начиная с максимальной. Первым в сумму берем число 64, осталось набрать $71-64=7$. Соответственно

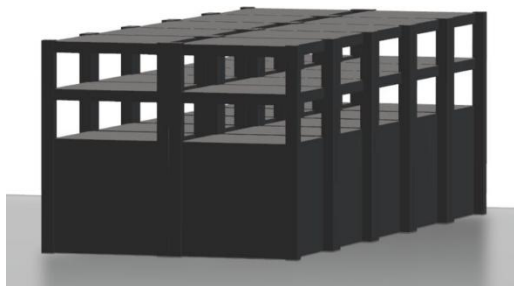
32, 16 и 8 в сумму не берем (они больше 7), но берем 4, 2 и 1. Получается, итого мы взяли в сумму 6-ю, 2-ю, 1-ю и 0-ю степени двойки, такой сумме соответствует запись 1110001.

4. Комбинаторика (2 балла)

[Компьютерщик Артур и серверный стенд]

Компьютерщик Артур собирает серверный стенд для запуска вычислительного кластера.

Стенд имеет размер $2 \times 2 \times 5$:



- 2 уровня (верхний и нижний),
- на каждом уровне - 2 ряда по 5 слотов.

Всего в стенде 20 слотов.

Артуру нужно установить 3 одинаковых мощных GPU-узла. Артур знает, что, если два мощных GPU окажутся слишком близко, система перегреется. Поэтому два GPU-узла не могут находиться в соседних по грани слотах.

Соседними считаются слоты:

- слева или справа в одном ряду,
- спереди или сзади в пределах одного уровня,
- строго над или под друг другом между уровнями.

Расположение по диагонали допустимо.

Пример допустимого расположения узлов:



Сколькими способами Артур может установить 3 GPU-узла так, чтобы не нарушить правило охлаждения?

В ответ укажите единственное число

Ответ: 588

Решение:

Обозначим как $C(n, m)$ количество перестановок из n по m .

$$C(20, 3) = 1140$$

Рассчитаем, сколько существует «неправильных» постановок узлов.

Соседние по грани пары слотов:

по рядам: 2 уровня * 5 позиций = 10

по уровням: 5 позиций * 2 ряда = 10

по длине: $(5 - 1) * 2 * 2 = 16$

Всего соседних пар: $10 + 10 + 16 = 36$

Если зафиксировать одну соседнюю пару, третий GPU можно поставить в любой из оставшихся 18 слотов:

$$36 * 18 = 648$$

Но тройки, где есть две соседние пары, посчитаны дважды.

Такие тройки образуют "угол". Их число:

$$8 * C(3, 2) + 12 * C(4, 2) = 8 * 3 + 12 * 6 = 96$$

$$1140 - 648 + 96 = 588$$

5. Основы логики (3 балла)

[Логическое равенство]

Сколько существует последовательностей из 8 битов, для которых выполняется следующее равенство:

$$A(X) \text{ ИЛИ } B(X) = \text{ИСТИНА}$$

Где X – последовательность из 8-и битов, т.е. $X = (x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7)$.

$A(X)$, $B(X)$ – логические выражения, заданные следующим образом:

$$A(X) = (x_0 \text{ XOR } x_4) \text{ И } (x_1 \text{ XOR } x_5) \text{ И } (x_2 \text{ XOR } x_6) \text{ И } (x_3 \text{ XOR } x_7)$$

$$B(X) = (x_0 \text{ И } x_1) \text{ ИЛИ } (x_1 \text{ И } x_2) \text{ ИЛИ } (x_2 \text{ И } x_3) \text{ ИЛИ } (x_3 \text{ И } x_4) \text{ ИЛИ } (x_4 \text{ И } x_5) \text{ ИЛИ } (x_5 \text{ И } x_6) \text{ ИЛИ } (x_6 \text{ И } x_7)$$

Примечание: Таблица истинности для XOR:

a	b	a XOR b
0	0	0
0	1	1
1	0	1
1	1	0

Ответ: 202

Решение:

Проанализируем первую функцию, $A(X) = (x_0 \text{ XOR } x_4) \text{ И } (x_1 \text{ XOR } x_5) \text{ И } (x_2 \text{ XOR } x_6) \text{ И } (x_3 \text{ XOR } x_7)$. Она будет истинна, когда $(x_0 \text{ XOR } x_4) = \text{ИСТИНА}$, $(x_1 \text{ XOR } x_5) = \text{ИСТИНА}$, $(x_2 \text{ XOR } x_6) \text{ И } (x_3 \text{ XOR } x_7) = \text{ИСТИНА}$. Следовательно, должны выполняться следующие неравенства: $(x_0 \neq x_4) \text{ И } (x_1 \neq x_5) \text{ И } (x_2 \neq x_6) \text{ И } (x_3 \neq x_7)$. То есть нам нужно найти такую битовую последовательность, чтобы биты на соответствующих позициях (0 и 4, 1 и 5, и т.д.) принимали разное значение. Таких последовательностей будет $2 \cdot 2 \cdot 2 \cdot 2 = 16$ (т.е. для первых 4 битов мы можем выбрать, какой бит поставить, а последние 4 определяются однозначно).

Дальше считаем, сколько последовательностей при которых $B(X) = \text{ИСТИНА}$. Пойдем от обратного, и посчитаем, сколько последовательностей при которых $B(X) = \text{ЛОЖЬ}$. Получается, что в каждой паре подряд идущих битов должен быть хоть 1 ноль. Рассмотрим несколько случаев:

- 1) В числе 0 единиц: 1
- 2) В числе 1 единица: 1
- 3) В числе 2 единицы: $6+5+4+3+2+1 = 21$
- 4) В числе 3 единицы: 20
- 5) В числе 4 единицы: 5

Больше единиц в числе быть не может. Тогда кол-во последовательностей, при которых $B(X) = \text{ИСТИНА}$: $2^8 - (1+1+21+20+5) = 201$.

Теперь посчитаем количество последовательностей, подходящих под обе функции. Их будет всего 15 (это можно посчитать перебором или программным способом).

Теперь, мы можем узнать при каких последовательностях $A(X)$ ИЛИ $B(X) = \text{ИСТИНА}$:

$$16+201-15 = 202$$

6. Моделирование (1 балл)

[Аудитории]

Недавно в ИТМО произошли изменения номеров аудиторий, но вот незадача - в вашем расписании остались старые номера. Чтобы получить новые номера аудиторий и попасть на все занятия, вы можете воспользоваться следующей логикой:

Если номер не заканчивается на 0, из него вычитается 50. Затем, независимо от этого, мы всегда уменьшаем номер на 100.

Далее, в зависимости от суммы цифр номера аудитории, полученного на предыдущем шаге, применяются разные действия. Если сумма цифр чётная, номер становится равным квадратному корню из самого себя (округлённому до целого по правилам арифметического округления). Если сумма цифр нечётная, к номеру прибавляется остаток от деления на 7.

Далее, если в номере присутствует цифра 7, это требует особого внимания: в таком случае номер умножается на остаток от деления этого номера на 5, увеличенный на 1. Если цифры 7 нет, номер делится на 3 (целочисленно).

Для каждого номера нужно выполнить 3 итерации этого алгоритма, чтобы получить новый номер аудитории.

Список номеров аудиторий из вашего расписания:

- 483
- 3198
- 9801
- 1944

Ваша задача - вычислить новые номера аудиторий и вывести их в порядке возрастания через пробел.

Пример ответа: 4 100 200 300

Ответ: 18 19 33 47

Решение:

1. Начинаем с исходного номера аудитории и проверяем, заканчивается ли он на 0. Если да, применяем специальное правило, деля число на остаток от деления на 3, увеличенный на 1. Если номер не заканчивается на 0, из него вычитаем 50.
2. Независимо от предыдущего шага, всегда уменьшаем номер на 100.
3. Рассчитываем сумму цифр номера. Если сумма цифр чётная, номер становится равным квадратному корню из самого себя (округлённому до целого). Если сумма нечётная, к номеру прибавляется остаток от деления на 7.
4. Если в номере присутствует цифра 7, умножаем его на остаток от деления на 5, увеличенный на 1. Если цифры 7 нет, номер делится на 3.
5. После выполнения всех операций вычисляем итоговый номер аудитории и возвращаем его.

Пример для числа 483 (3 итерации)

Итерация 1:

- 1) После Шага 1 (число не заканчивается на 0, уменьшили на 50): 433
 - 2) После постоянного уменьшения (уменьшили на 100): 333
 - 3) После Шага 2 (сумма цифр нечётная, прибавили остаток от 7): 337
 - 4) После Шага 3 (в числе есть цифра 7): 1011
- Результат итерации 1: 1011

Итерация 2:

- 1) После Шага 1 (число не заканчивается на 0, уменьшили на 50): 961
 - 2) После постоянного уменьшения (уменьшили на 100): 861
 - 3) После Шага 2 (сумма цифр нечётная, прибавили остаток от 7): 861
 - 4) После Шага 3 (в числе нет цифры 7): 287
- Результат итерации 2: 287

Итерация 3:

- 1) После Шага 1 (число не заканчивается на 0, уменьшили на 50): 237
 - 2) После постоянного уменьшения (уменьшили на 100): 137
 - 3) После Шага 2 (сумма цифр нечётная, прибавили остаток от 7): 141
 - 4) После Шага 3 (в числе нет цифры 7): 47
- Результат итерации 3: 47

Аналогично можно получить для других чисел:

start: 3198

Iteration 0 result: 1017

Iteration 1 result: 3492

Iteration 2 result: 19

start: 9801

Iteration 0 result: 3218

Iteration 1 result: 3070

Iteration 2 result: 18

start: 1944

Iteration 0 result: 3592

Iteration 1 result: 10341

Iteration 2 result: 33

7. Теория игр (3 балла)

[Игра в бинарное дерево]

Граф — это множество вершин (обозначаются на схемах точками или кругами), некоторые из которых соединены между собой ребрами (обозначаются на схемах линиями).

Бинарным деревом называют такой граф, на который наложен ряд ограничений:

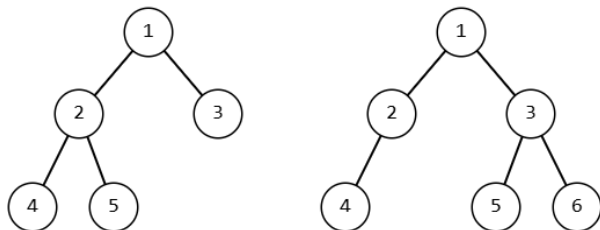
граф не содержит циклов, т.е. в графе нет такой вершины, из которой, проходя по ребрам графа, можно попасть в эту же вершину

граф является связным, т.е. из любой вершины графа можно попасть по ребрам в любую другую

никакая вершина графа не может быть связана более чем с тремя другими вершинами

есть одна вершина, называемая корнем дерева, которая связана не более чем с двумя вершинами

Примеры бинарных деревьев (корень показан как вершина с номером 1):



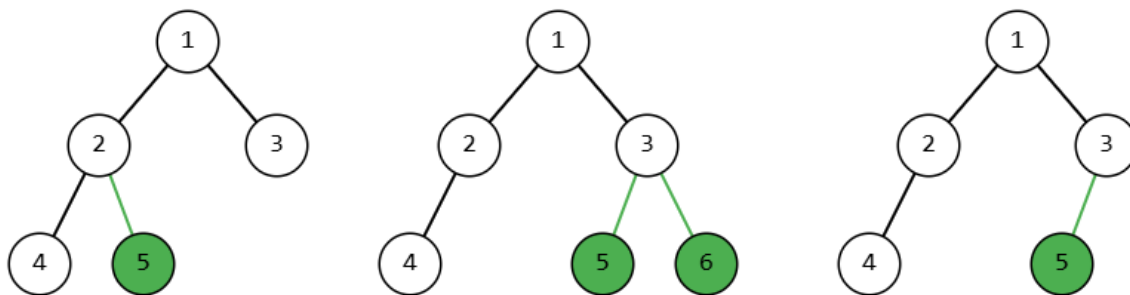
Будем называть высотой бинарного дерева максимальное возможное в этом дереве количество ребер, которое может потребоваться пройти от корня дерева, чтобы добраться до какой-либо вершины графа (при этом нельзя дважды проходить через одно и то же ребро или одну и ту же вершину). На картинках выше изображены бинарные деревья высотой 2.

Петя и Витя решили сыграть в игру по строительству бинарного дерева. Перед началом игры в их бинарном дереве содержится ровно 1 вершина, являющаяся корнем этого дерева.

В свой ход игрок на свой выбор добавляет в бинарное дерево либо одну вершину, связывая ее ребром с одной из уже существующих в дереве вершин, либо две вершины, связывая их ребрами также с одной из уже существующих в дереве

вершин. При добавлении вершин и ребер игроки не могут нарушать ограничений построения графа-бинарного дерева, т.е. к примеру не могут добавить новое ребро к вершине, которая уже соединена с тремя другими вершинами и т.д.

Примеры допустимых ходов (черным показан граф на момент начала хода игрока, зеленым выделены ребра и вершины, которые он добавил в свой ход)



Назовем полным бинарным деревом высоты h такое бинарное дерево, в которое не может быть добавлена ни одна вершина с ребром так, чтобы высота дерева не увеличилась.

Победителем в игре будет считаться игрок, в чей ход бинарное дерево превратится в полное бинарное дерево высоты 12. Игроки также договорились, что ни в какой момент игры высота дерева не должна стать больше 12. Первым ходит Петя.

Кто может победить в такой игре независимо от ходов противника?

В ответ запишите два числа через пробел без кавычек: номер игрока, который может победить независимо от ходов противника (1 – Петя, 2 – Витя) и максимальное количество ходов, которые может потребоваться выполнить *этому игроку* для победы. Пример записи ответа, если выиграть может Петя своим вторым ходом независимо от ходов противника: «1 2»

Решение:

Назовем вершину «ребенком» другой вершины («родителя»), если в графическом представлении графа эта вершина расположена ниже, чем вершина родитель. Назовем вершину открытой, если у нее 0 или 1 ребенок. И полностью открытой, если у нее нет детей. В таком случае ее можно закрыть за 1 ход (добавив ей 1 или 2 вершины-ребенка соответственно). Вершины, которые не удовлетворяют этому условию, считаются закрытыми. Также закрытыми будем считать все вершины листья на последнем уровне, который нужно заполнить в игре.

Проигрышными будут позиции, в которых открыто четное число вершин. В конце игры, если останется только 2 вершины, к которым можно присоединить новую, выиграть не получится, а при правильной стратегии первого игрока он сможет добиться того, чтобы всегда после своего хода оставлять четное количество вершин (стратегия описана позже).

Выигрышными будут позиции, в которых открыто нечетное количество вершин. Таковой позицией является старт игры, так как в игре пока только одна вершина и она открыта

Заметим, что четность количества открытых вершин может меняться в зависимости от действий игроков

Четность изменится, если:

мы добавляем 2 вершины к полностью пустой – то минус одна открытая, + 2 открытых, итого +1;

мы добавляем 1 вершину к полностью открытой – то открытых вершин меньше не становится, появляется одна новая вершина;

Четность не изменится, если

добавляем 1 вершину к вершине, у которой уже был один ребенок, то минус 1 открытая, + 1 открытая = 0).

Первый игрок может не допускать ситуации при которой второй игрок сможет не менять четности (если где-то есть ситуация что можно добавить 1 вершину к вершине, у которой уже был один ребенок – то первый игрок это делает. В другом случае он добавляет 2 новых вершины к полностью открытой вершине).

Исключением является предпоследний уровень: в нем нужно внимательнее следить за четностью, так как она меняется немного по другим правилам (так как листья последнего уровня считаются закрытыми).

Четность изменится, если:

мы добавляем 2 вершины к полностью пустой – то минус одна открытая;

добавляем 1 вершину к вершине, у которой уже был один ребенок, то минус 1 открытая).

Четность не изменится, если

мы добавляем 1 вершину к полностью открытой – то открытых вершин меньше не становится, новых открытых не появляется;

Нам нужно сделать количество открытых вершин четным (0) ($1+2+4+\dots$ степени двойки – изначально нечетное (1)), и первый игрок может после своего хода поддерживать то, чтобы всегда после его хода оставалось четное количество открытых вершин

Осталось выяснить, какое максимальное количество ходов ему может потребоваться для выигрыша. Так как стратегия 1 игрока определена, заметим, что максимальное количество ходов на то, чтобы добавить все необходимые вершины, потребуется совершить в случае, если первый игрок каждый раз будет добавлять по одной вершине. По его стратегии он это делает после того, как появляется ситуация, в которой это необходимо, а эта ситуация возникает если противник в свой ход добавил ровно 1 вершину (в разные места на предпоследнем уровне и во всем остальном графе, но тем не менее). В самом начале игры первый игрок по этой стратегии должен поставить 2 вершины, а дальше всю игру они будут с противником добавлять по одной вершине и это будет самая длинная возможная партия

1 (корень) + $2 + 1 + 1 + 1 + 1 + \dots =$ количество вершин в дереве высоты $12 = 1 + 2 + 4 + \dots + 2 + 4096 = 2^{13} - 1 = 8192 - 1 = 8191$

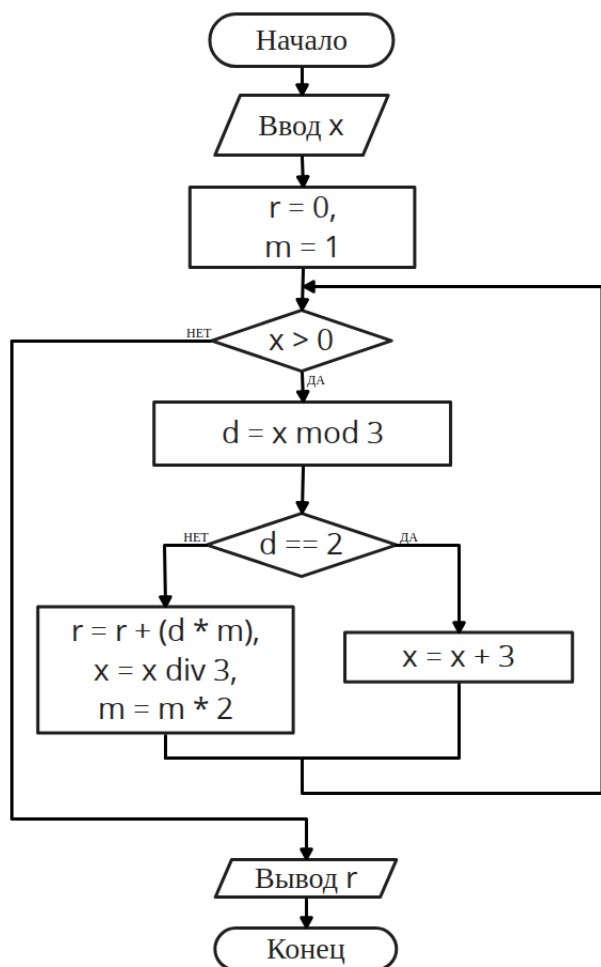
$8191 - (1 \text{ стартовая} + 2 \text{ из хода первого игрока}) = 8188$. Делим на два, так как дальше игроки поочередно добавляли по единичке – получается 4094 на человека. Итого первый игрок совершил первый ход + еще 4094 , итого 4095

Ответ: 1 4095

8. Программирование и алгоритмизация (3 балла)

[Блок-схема]

Дана блок-схема алгоритма:



Известно, что на вход алгоритма подаются целые положительные x , но не гарантировано, что алгоритм завершит свою работу для любых x . Какое наименьшее x нужно подать на вход, чтобы алгоритм завершил свою работу и на выходе было получено число 2026 ?

Примечание:

$a \bmod b$ – остаток от деления a на b

$a \operatorname{div} b$ – целочисленное деление a на b

Ответ: 88482

Решение:

Заметим, что в цикле постоянно проверяется делимость числа x на 3 . Также заметим, что если остаток при делении на 3 будет равен двум, то тогда $x = x + 3$, и остаток от деления нового $x = x + 3$ на 3 также будет равен двум. То есть остаток от деления X на 3 никогда не должен быть равен 2 , а только 1 или 0 . По многократной проверки остатка при делении на три числа d и его делении на три, можем заметить, в данном алгоритме r строится из цифр троичной системы счисления.

С другой стороны, обратим внимание на $r = r + (d * m)$ и $m = m * 2$ – эти две команды строят нам в r запись в двоичной системе счисления.

Переведем 2026 в 2 сс: 11111101010_2 .

И это число будет являться троичной записью исходного x . $11111101010_3 = 88482_{10}$.

9. Сортировка и фильтрация данных. Моделирование (2 балла)

[Борьба за PlayStation]

В столовой ИТМО стоит 2 PlayStation. Начинается борьба за PlayStation, но студенты ИТМО - приличные люди, поэтому они соблюдают правила очереди. Каждый раз, когда студент или пара друзей решают поиграть на одной из консолей, они создают заявку в ИСУ (электронная система университета), становясь частью очереди на использование PlayStation.

Заявка содержит следующую информацию:

- 1) имя студента (или пары студентов)
 - 2) время прихода (целое, номер минуты)
 - 3) длительность игры первого игрока (минуты)
 - 4) длительность игры второго игрока (минуты)
- (если один закончил раньше, второй продолжает играть один; консоль освобождается, когда закончит последний).

Правила очереди

1. Когда PlayStation освобождается, система ИСУ автоматически выбирает следующую заявку из очереди ожидания, и доступ к PlayStation получают создатели этой заявки.
2. Выбирается заявка с минимальным временем прихода (пришла раньше).
3. Если время прихода одинаково, выбирается та, что раньше во входных данных.
4. Если в момент освобождения никто не ждёт, консоль простаивает до следующего прихода.
5. Если одновременно свободны несколько PlayStation, заявки распределяются по консолям в порядке их номеров (1, 2, ...).

В один день в столовой ИТМО было подано 8 заявок на использование PlayStation. Отсчет времени начинается с 0 в момент запуска электронной системы для создания заявок и идёт в минутах; первая заявка пришла спустя 1 минуту после запуска. Время ожидания заявки - это период, прошедший с момента подачи заявки, но до того, как её инициаторы начали играть на PlayStation. Общее время ожидания - это сумма времени ожидания всех заявок

Вам дана таблица заявок студентов:

Кто	Время прихода (мин)	Время игры первого игрока (мин)	Время игры второго игрока (мин)
Антон	2	4	-
Коля+Миша	1	3	6
Маша	3	2	-
Гриша	2	5	-
Ильяс	1	2	-
Света+Саша	3	4	1
Андрей	2	1	-
Богдан	1	4	-

Вам нужно вывести два числа через пробел:

- 1) Общее время ожидания всех заявок. Общее время использования консолей измеряется в минутах, начиная с нуля.
- 2) Номер минуты, когда освободилась последняя PlayStation, после завершения последней заявки.

Пример записи ответа: «12 34»

Ответ: 39 16

Решение:

Заметим, что если играет двое ребят – их общее время игры равно максимальному времени игры из этих двух. Тогда мы можем упростить таблицу входных данных:

Кто	Время прихода (мин)	Время игры (мин)
Антон	2	4
Коля+Миша	1	6
Маша	3	2
Гриша	2	5
Ильяс	1	2
Света+Саша	3	4
Андрей	2	1
Богдан	1	4

Далее отсортируем эту таблицу по времени прихода игроков, оставляя неизменным относительный порядок тех, кто пришел в одну и ту же минуту.

Кто	Время прихода (мин)	Время игры (мин)
Коля+Миша	1	6
Ильяс	1	2
Богдан	1	4
Антон	2	4

Гриша	2	5
Андрей	2	1
Маша	3	2
Света+Саша	3	4

А далее будем распределять ребят последовательно по списку на свободные консоли и отслеживать, кто в какой момент начнет играть:

Момент времени	PS1	PS2
0		
1	Коля+Миша	Ильяс
2		Богдан
3		
4		
5		
6	Антон	Гриша
7		
8		
9		
10	Андрей	Света + Саша
11		
12	Маша	
13		
14		
15		
16		

Получаем, что консоли освободятся к 16ой минуте.
Теперь нужно найти общее время ожидания. Для каждого игрока оно вычисляется как время начала «минус» время прихода.

Кто	Время прихода (мин)	Время начала игры (мин)	Время ожидания (мин)
Коля+Миша	1	1	0
Ильяс	1	1	0
Богдан	1	3	2
Антон	2	7	5
Гриша	2	7	5
Андрей	2	11	9
Маша	3	12	9
Света+Саша	3	12	9

Суммарное время ожидания: 2 + 5 + 5 + 9 + 9 + 9 = 39
В ответ записываем два найденных числа – 39 и 16.

10. Сортировка и фильтрация данных. (2 балла) [Турагенство]

Вы работаете в туристическом агентстве, и к вам обратился постоянный клиент с запросом на подбор маршрута. Вам нужно помочь ему выбрать оптимальный маршрут, учитывая его предпочтения, бюджет и возможные дополнительные затраты, связанные с использование нашего агентства.

Данные о клиенте:
Максимальный бюджет — 10 500 рублей.
Предпочтения клиента — для выбора маршрута важны три ключевых критерия, из которых хотя бы два должны быть выполнены:

Красивые виды — клиент ищет маршруты с живописными природными пейзажами.
Хороший отель — клиент предпочитает маршруты с отелями высокого уровня.
Экскурсии и развлечения — клиент заинтересован в маршрутах с культурными мероприятиями и активным отдыхом.
Дата, до которой клиент хочет улететь — 01.07.2026 (это крайний срок, до которого клиент готов ждать рейс, рейсы позже он не будет рассматривать ни при каких обстоятельствах).
Желаемой даты нет: учитывается «сегодняшняя дата» и крайний срок.

Данные о маршрутах:
Стоимость маршрута — сколько стоит участие в маршруте (например, 5000 рублей).
Время в пути — сколько времени займет поездка (например, 5 часов).
Дата начала рейса — когда начинается маршрут (например, 1 июля).
Тип маршрута — описание маршрута, например, экскурсия по Москве, шоппинг в Питере и так далее.
Особенности маршрута — наличие таких характеристик как красивый отель, наличие экскурсий и т.д.

Дополнительные затраты за день использования нашего агентства:

Если клиент решит подождать более поздний рейс (но до 01.07.2026), ему нужно будет заплатить дополнительную сумму за каждый день использования нашего агентства (800 рублей за день).

Маршрут подойдет клиенту, если он удовлетворяет следующим критериям:

Он укладывается в бюджет клиента.

Учитывает его предпочтения по маршруту, причем должно быть выполнено хотя бы два из трех критериев (красивые виды, хороший отель, экскурсии).

Учитывает возможные дополнительные затраты за день использования нашего агентства (если дата начала рейса позже сегодняшней).

Ваш босс поставил задачу получить максимальную выручку для агентства. Иногда может быть выгодно заставить клиента подождать рейс, чтобы выбрать более прибыльный маршрут, поэтому вам нужно взвесить возможные затраты за использование нашего агентства и выбрать тот маршрут, который обеспечит наибольшую выручку для агентства, при этом удовлетворяя требования клиента. Выручкой считать все деньги, которые клиент выплатит агентству: стоимость маршрута + при необходимости плата за дни использования агентства во время ожидания рейса.

В ответе укажите два числа через пробел: номер выбранного маршрута и общую выручку, которую агентство получит от этого маршрута.

За «сегодняшнюю дату» при расчетах брать 18.06.2026.

№	Стоимость маршрута	Время в пути, ч	Дата начала рейса	Тип маршрута	Красивые виды	Хороший отель	Экскурсии
1	5 095 Р	10	25.06.2026	Экскурсия по Москве	Да	Нет	Да
2	7 476 Р	5	25.06.2026	Шопинг-тур в Санкт-Петербург	Да	Нет	Нет
3	11 776 Р	10	25.06.2026	Исторические места	Нет	Да	Да
4	4 015 Р	8	21.06.2026	Шопинг-тур в Санкт-Петербург	Нет	Да	Да
5	5 689 Р	8	21.06.2026	Гастротур	Нет	Нет	Да
6	3 553 Р	9	17.06.2026	Термальные источники	Да	Да	Да
7	4 756 Р	2	22.06.2026	Шопинг-тур в Санкт-Петербург	Да	Да	Да
8	11 928 Р	10	01.07.2026	Горы и озёра (Карелия)	Да	Нет	Да
9	4 040 Р	2	26.06.2026	Природные заповедники	Да	Да	Да
10	5 145 Р	10	20.06.2026	Шопинг-тур в Санкт-Петербург	Да	Да	Да
11	11 323 Р	10	23.06.2026	Активный отдых (рафтинг)	Нет	Нет	Нет
12	6 183 Р	3	20.06.2026	Золотое кольцо	Да	Да	Нет
13	6 114 Р	5	04.07.2026	Экскурсия по Москве	Нет	Нет	Нет
14	8 521 Р	9	18.06.2026	Исторические места	Нет	Да	Да
15	8 024 Р	11	23.06.2026	Экскурсия по Москве	Да	Да	Да
16	6 783 Р	12	05.07.2026	Культурный уикенд	Да	Нет	Нет
17	4 694 Р	7	02.07.2026	Активный отдых (рафтинг)	Да	Да	Да
18	11 317 Р	10	28.06.2026	Золотое кольцо	Да	Да	Нет
19	5 932 Р	2	27.06.2026	Термальные источники	Нет	Да	Да
20	5 703 Р	10	18.06.2026	Исторические места	Да	Да	Да
21	7 203 Р	12	27.06.2026	Горы и озёра (Карелия)	Да	Нет	Да
22	11 981 Р	6	19.06.2026	Природные заповедники	Нет	Нет	Да
23	6 276 Р	10	20.06.2026	Активный отдых (рафтинг)	Да	Да	Нет
24	5 453 Р	7	17.06.2026	Пляжный отдых (Сочи)	Да	Нет	Да

Ответ: 9 10440

Решение:

Фильтрация маршрутов по предпочтениям клиента. В Excel используем фильтрацию, чтобы отобрать маршруты, которые удовлетворяют хотя бы двум из трех предпочтений клиента (красивые виды, хороший отель, экскурсии), а также укладываются в его бюджет.

Сортировка по стоимости и дате начала. После фильтрации оставшихся маршрутов сортируем их по стоимости, начиная с самых дешевых. Если стоимость маршрутов одинаковая, затем сортируем их по дате начала рейса (выбираем более поздний рейс, если это необходимо для максимальной выручки).

Расчет дополнительных затрат за день использования нашего агентства. Для каждого маршрута, дата начала которого позже желаемой даты клиента, рассчитываем дополнительные затраты за каждый день использования нашего агентства с помощью следующей формулы:

$$=IF(\text{Дата начала} > \text{Дата клиента}, (\text{Дата начала} - \text{Дата клиента}) * \text{Стоимость_за_день_использования нашего агентства}, 0)$$

Это позволяет учесть дополнительные расходы клиента, если ему придется подождать более поздний рейс.

Выбор маршрута с максимальной выручкой для агентства. Мы выбираем маршрут, который не только соответствует бюджету клиента и его предпочтениям, но и приносит максимальную выручку агентству, учитывая возможные дополнительные затраты за использование нашего агентства. Иногда это может означать, что клиенту придется подождать более поздний рейс, если это приведет к более прибыльному маршруту для агентства.

Количество дней использования агентства = $\max(0; \text{Дата начала рейса} - \text{Сегодняшняя дата})$. Плата начисляется за каждый такой день.

Отфильтровав маршруты и найдя выручку агентства для каждого из оставшихся - получаем, что наиболее выгодным для агентства является 9-й маршрут, а выручка от него составит 10440.

Отборочный этап. Первый тур (приведен один из вариантов заданий)

1. Теоретические основы информатики (1 балл)

[Алгоритмы шифрования]

Какие симметричные алгоритмы шифрования считаются безопасными для использования в современном мире? В ответе запишите все возможные варианты через пробел. Пример ответа: "1 2 3".

1. AES
2. RSA
3. MD5
4. SHA-256
5. Serpent

Ответ: 1 5

2. Комбинаторика (1 балл)

[Билеты]

В стопке лежат билеты для экзамена по дискретной математике в ИТМО. В стопке 7 билетов: три на "А", два на "В", два на "С". Буквы обозначают тип (тему) билета. Билеты одного типа между собой неразличимы. Студенты подходят по очереди, ассистент каждый раз выдает верхний билет из стопки.

Правило: если сейчас должны выдать билет, и у двух предыдущих студентов уже были выданы "А" и "А" (то есть подряд уже было две "А"), и сейчас сверху тоже лежит "А", то перед выдачей ассистент перемешивает верхние три билета в произвольном порядке и затем тут же выдает верхний из этих трех, даже если это снова оказался "А" - повторной проверки и дополнительного перемешивания в тот же момент не происходит. Если условие не выполняется, ассистент просто выдает верхний билет.

Сколько различных последовательностей выдачи этих 7 билетов можно получить при таком правиле, если исходный порядок стопки любой, а при перемешивании ассистент может выбрать любой порядок верхних трех?

Ответ: 210

3. Системы счисления (1 балл)

[Пароль]

Наташа забыла свой пароль, состоящий из двух цифр. Она помнит, что:

1. Пароль состоит из двух десятичных цифр.
2. В восьмеричной системе пароль записывается ровно тремя символами (без ведущих нулей).
3. В шестнадцатеричной системе пароль записывается ровно двумя символами (без ведущих нулей).
4. Сумма десятичных цифр пароля равна 7.
5. Требуется наименьшее десятичное число, удовлетворяющее условиям.

Найдите значение пароля, ответ запишите в десятичной системе счисления.

Ответ: 70

4. Системы счисления (2 балла)

[Два числа]

Даны два числа: $A = 120x_4$ и $B = 130y_5$, где x и y — неизвестные цифры в системах счисления с основаниями 4 и 5 соответственно.

Известно, что для чисел A и B выполняются два условия:

1. $A + B$ кратно 7_{10} .
2. $|A - B|$ минимально при выполнении всех остальных условий задачи.

Найдите пару чисел A и B , удовлетворяющую условиям. В ответе укажите A и B , записанные в десятичной системе счисления через пробел (сперва A , потом B). Пример ответа: "33 149".

Ответ: 99 202

5. Основы логики (3 балла)

[Проблема Кота Матроскина]

У Кота Матроскина проблема. У него есть следующее логическое выражение, зависящее от трех переменных:

$$X_0(A, B, C) = (A \text{ И НЕ}(B)) \text{ ИЛИ } (A \text{ И НЕ}(A) \text{ И НЕ}(C)) \text{ ИЛИ } (C \text{ И } B \text{ И НЕ}(A) \text{ И НЕ}(C)) \text{ ИЛИ } C$$

Над этим выражением ему нужно проделать следующий алгоритм. Первоначальное значение $i=1$ и после каждого вычисления выражения значение i увеличивается на 1.

1) Вычислить выражение X_i , подставив X_{i-1} в данное выражение:

$$X_i(A, B, C) = (C \text{ И } X_{i-1}(A, B, 1)) \text{ ИЛИ } (\text{НЕ}(C) \text{ И } X_{i-1}(A, B, 0))$$

2) Повторить первый пункт 10 раз.

3) Вычислить выражение X_i , подставив X_{i-1} в данное выражение:

$$X_i(A, B, C) = (A \text{ И } X_{i-1}(1, B, C)) \text{ ИЛИ } (\text{НЕ}(A) \text{ И } X_{i-1}(0, B, C))$$

4) Повторить третий пункт 10 раз.

5) Вычислить выражение X_i , подставив X_{i-1} в данное выражение:

$$X_i(A, B, C) = (B \text{ И } X_{i-1}(A, 1, C)) \text{ ИЛИ } (\text{НЕ}(B) \text{ И } X_{i-1}(A, 0, C))$$

6) Повторить пятый пункт 10 раз.

Под слагаемым подразумевается переменная/переменная с отрицанием/константа или последовательность из переменных/переменных с отрицаниями/констант, соединенных операцией И.

Примерами слагаемых являются: $A \text{ И } B \text{ И } C$ – 1 слагаемое, $A \text{ И НЕ}(B)$ – 1 слагаемое.

Пример: в выражении $A \text{ ИЛИ } B \text{ ИЛИ } C$ – 3 слагаемых, $(A \text{ И НЕ}(B) \text{ И } 1) \text{ ИЛИ } (0 \text{ И } A)$ – 2 слагаемых

Важное примечание! Изначальную формулу X_0 изменять нельзя. Для каждого из пунктов вычисление происходит по следующим правилам (X, Y, Z – любые переменные или функции):

- Раскрываем скобки по правилу: $X \text{ И } (Y \text{ ИЛИ } Z) = (X \text{ И } Y) \text{ ИЛИ } (X \text{ И } Z)$
- Исключаем слагаемые следуя правилам, описанным ниже:
 - если внутри одного слагаемого есть 0, то это слагаемое исключаем
 - если есть повторяющиеся слагаемые – оставляем только одно, остальные исключаем
 - если в слагаемом одновременно встречаются X и $\text{НЕ}(X)$ – слагаемое исключаем
 - $\text{НЕ}(1) = 0$, $\text{НЕ}(0) = 1$
 - $1 \text{ И } X = X$
 - слагаемые можно менять местами, так же, как и порядок переменных в слагаемом
 - делаем так, пока можно что-то исключить
 - правила, не описанные в этом списке, применять строго запрещено!

Сколько слагаемых получится в итоговом выражении?

Ответ: 5

6. Основы логики (2 балла)

[Ингредиенты для печенья]

Оля решила узнать у бабушки рецепт ее фирменного печенья. Бабушка очень любит информатику, поэтому дала внучке список ингредиентов, записанный с помощью 7 логических высказываний и сказала, что одно из них ложно, а остальные истинны. Логические высказывания:

1. Если 33 в троичной системе записывается как 1010, то муки нужно использовать на 3 стакана больше, чем сахара.
2. Если мука и сахар используются в равных пропорциях, то растопленного сливочного масла нужно использовать в 4 раза меньше, чем использовалось муки.
3. Сахара нужно взять на 2 стакана меньше, чем муки.
4. Растопленного сливочного масла используется в 2 раза меньше, чем муки.
5. Муки используется в 2 раза больше, чем сахара.
6. Число стаканов сахара является минимальным числом в двоичной системе счисления, содержащим хотя бы одну единицу и хотя бы один ноль в значащих разрядах.
7. Если используется 2 стакана сахара, то существует ровно 1 система счисления, запись в которой количества стаканов муки будет начинаться с цифры 1.

Примечание: все ингредиенты отмеряются с помощью стакана, каждого ингредиента используется целое положительное количество стаканов.

Сколько стаканов муки нужно использовать, чтобы приготовить бабушкино фирменное печенье? В ответ запишите одно число - количество стаканов муки.

Ответ: 4

7. Кодирование, шифрование (2 балла)

[Сообщение из Котинска]

Кот Матроскин ждет невероятно важное сообщение от своих родственников из Котинска. Но так как его родственники очень тревожные и боятся, что сообщение будет перехвачено, они его закодировали.

Кодирование, которое использовали коты, основано на том, что если какая-то подстрока уже встречалась ранее в строке, то на нее можно сослаться, указав на сколько нужно сместиться относительно текущего элемента, чтобы попасть на первый символ нужной нам подстроки, и размер подстроки. Закодированное сообщение представляет из себя строку, в которой последовательно идут тройки элементов:

- первый элемент тройки - цифра, на которую нужно сместиться влево в раскодированной части исходной строки, чтобы получить первый символ повторяющейся ранее подстроки;
- второй элемент тройки - цифра, сколько символов нужно забрать (или, другими словами, размер подстроки);
- третий элемент - буква, которая будет стоять за скопированной подстрокой.

Пример: закодированная строка 00a00b00c33d

Исходная строка будет строиться следующим образом:

1. Изначально у нас ничего нет, то есть пустая строка «»
2. 00a: смещаемся на 0 символов влево и копируем подстроку длиной 0, после нее ставим символ a. Получилась строка «a»
3. 00b: смещаемся на 0 символов влево и копируем подстроку длиной 0, после нее ставим символ b. Получилась строка «ab»
4. 00c: смещаемся на 0 символов влево и копируем подстроку длиной 0, после нее ставим символ c. Получилась строка «abc»
5. 33d: смещаемся на 3 символа влево и копируем подстроку длиной 3, после нее ставим символ d. Получилась строка «abcabcd»

«abcabcd» - раскодированная строка.

Матроскин получил одно и то же закодированное сообщение от двух родственников, но проблема в том, что оба родственника допустили по одной ошибке в своих кодах. Кот уже выяснил, что оба родственника ошиблись в записи третьего элемента одной из троек (каждый в своей тройке).

Помогите Матроскину восстановить сообщение.

Полученные сообщения от родственников из Котинска:

6. 00a00a00b00c31c11b73b00e00f00a
7. 00a11b00c41c00c51c73e00f00a

Примечание: подстрокой называется непрерывная последовательность символов исходной строки. Например «a», «bc» - будут подстроками для строки «abc»

Ответ: aabcaccbabcfeaf

8. Теория игр. Моделирование на графе (3 балла)

[ИТМО-курьер]

Внутри кампуса ИТМО настроили систему доставки между разными корпусами, для сокращения их решили обозначать цифрами, начиная с единицы. Корпуса соединены односторонними дорогами, по данным дорогам можно двигаться лишь в одном направлении. По этим дорогам ездит один курьер-робот. Им по очереди управляют два игрока: сначала 1 ход делает Света, потом 1 ход делает Богдан, потом снова Света и т. д.

В начале игры робот стоит в корпусе 1.

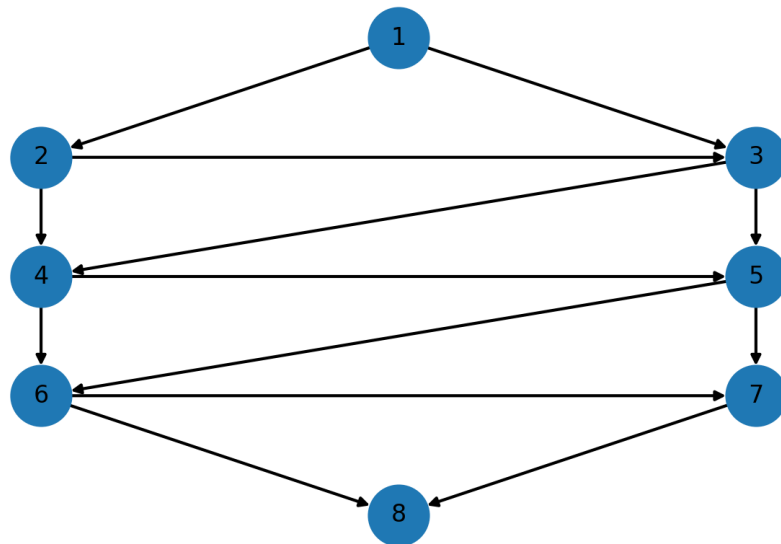
За один ход игрок обязан:

- выбрать одну из исходящих из текущего корпуса дорог;
- перевезти робота по ней в следующий корпус.

Важный момент: т. к. при создании системы старались сэкономить деньги, у робота оказались плохие колёса, из-за этого после того, как он проезжает по выбранной дороге, она становится непригодной для дальнейшей эксплуатации и больше никогда не может быть использована.

Если в свой ход игрок не может выбрать ни одной дороги (из текущего корпуса нет неиспорченных исходящих дорог), то этот игрок проигрывает.

Схема соединения корпусов дорогами:



Считайте, что оба игрока хотят победить и прикладывают все свои силы для этого. Света и Богдан играют честно и изначально знают схему соединения корпусов и дорог полностью.

Кто из игроков может победить в этой игре независимо от ходов противника, и какое максимальное количество его ходов ему может на это потребоваться?

В ответ запишите через пробел два числа: сначала номер игрока (Света — 1, Богдан — 2, если невозможно сказать — 0), а затем максимальное количество ходов, которое может потребоваться этому игроку для победы.

Ответ: 1 3

9. Моделирование на таблице (1 балл)

[Склад]

Склад 7x7: стоят 7 стеллажей в ряд, у каждого стеллажа 7 полок по высоте. На каждой полке изначально лежит от 0 до 3 ящиков. Стеллажи нумеруются начиная с 1. Робот начинает со стеллажа 1 и движется по столбцам слева направо. Приехав к очередному стеллажу, он проходит полки сверху вниз и для каждой полки пытается увеличить число ящиков на 1: если на полке меньше 5, добавляет один; если уже 5, пропускает и идет дальше. Максимальная вместимость каждой полки - 5 ящиков. Если в текущем стеллаже в какой-то момент оказалось 3 полки со значением 5, робот убирает все ящики с этого стеллажа (обнуляет его). После завершения визита робот едет к следующему стеллажу, когда робот прошел все стеллажи слева направо, он возвращается к первому и продолжает руководствоваться теми же правилами, пока не израсходует заданное число ходов. Ход – это один визит к одному стеллажу (полная обработка стеллажа сверху вниз). Изначальная конфигурация полок дана ниже (строки - полки сверху вниз, столбцы - стеллажи слева направо):

```
3 1 3 2 3 2 0
1 3 3 3 1 3 1
0 3 1 0 2 1 3
3 0 0 3 0 0 0
2 3 2 1 3 2 3
3 2 0 3 0 3 2
0 0 3 0 3 0 0
```

Всего робот делает 14 ходов, после чего останавливается.

В ответ укажите одно число: сколько раз за эти 14 ходов произойдет обнуление какого-либо стеллажа.

Ответ: 5

10. Кодирование информации, изображения. Комбинаторика. (2 балла)

[Брелоки]

Татьяна Олеговна делает брелоки из бисера. Брелоки представляют собой нить, на которую нанизаны 12 бусин. Татьяна использует только белые, серые и черные бусины. С одной стороны этой нити расположено крепление для ключей. Раньше Татьяна записывала схемы брелоков, которые она делала, с помощью последовательностей нулей, единиц и двоек, где «0» означал белую бусину, «1» – черную, а «2» - серую. Крепление для ключей всегда предполагается в левой части схемы.

Пример схемы брелока, состоящего из 10 черных бусин и 2 белых, в котором часть брелока, ближайшая к креплению, черная, а вторая часть – белая:

«111111111100» - итоговая запись содержит 12 символов – по одному на каждую бусину.

Но вот однажды Татьяна придумала, как можно сократить запись: вместо того, чтобы записывать подряд 10 «1», она решила записывать число бусин одного цвета, которые идут подряд, например 10, ставить дефис «-», после чего записывать цвет бусинки (цвет записывается так же, как и раньше: «1» означает черная, «0» - белая, «2» - серая). Между таким образом записанными блоками, состоящими из одного цвета, ставится запятая. Подряд не может записываться два блока, описывающих бусины одного и того же цвета – они должны быть объединены в один блок. В блоке не может быть 0 или отрицательное число бусин. Таким образом, запись схемы брелока из примера выше сокращается до «10-1,2-0» – итого 8

символов. Это на 4 символа меньше, чем содержала исходная запись. Однако выяснилось, что в некоторых случаях новая запись становится даже длиннее, чем исходная, например брелок из чередующихся белых и черных бусин.

Исходная форма записи: «010101010101» - итого 12 символов.

Новая форма записи: «1-0,1-1,1-0,1-1,1-0,1-1,1-0,1-1,1-0,1-1» - итого 47 символов.

Сколько существует различных брелоков длины 12, схемы которых в новой и исходной формах записи содержат по одинаковому количеству символов? Брелоки считаются различными, если отличаются хотя бы одной бусиной. В ответ запишите одно число – количество брелоков.

Ответ: 36

Отборочный этап. Второй тур (приведен один из вариантов заданий)

1. Архитектура компьютера (1 балл)

[Архитектура бывает разная]

Используя поиск информации в глобальной сети Интернет, определите, что является ключевым отличием наиболее распространенной модифицированной Гарвардской архитектуры, используемой во многих современных процессорах (например, в ядрах ARM), от её «чистой» формы.

1. Память для инструкций и память для данных физически и логически полностью разделены.
2. Наличие на уровне процессора отдельных, независимых кэшей для инструкций и данных, которые, работают с единым адресным пространством основной оперативной памяти (ОЗУ).
3. Использование только одного канала для обмена с памятью, как в архитектуре фон Неймана.
4. Отказ от принципа хранимой программы (принципа фон Неймана), согласно которому команды и данные хранятся в одной и той же памяти и обрабатываются процессором одинаково.

Ответ: 2

2. Архитектура компьютера (2 балла)

[Настя и быстрый шкафчик]

Настя работает с детьми в кружке МОТИ и учит их архитектуре компьютера. Она объясняет кэш как “быстрый шкафчик”, куда заглядывает процессор, когда ему нужно что-то взять.

Есть 5 терминов, которые описывают работу “шкафчика”:

1. Ситуация, когда процессору что-то потребовалось, и это что-то оказалось в шкафчике, называется **hit** (попадание).
2. Время, которое тратится на обращение к шкафчику при попадании (**hit**), называется **hit time** (время попадания) и измеряется в наносекундах.
3. Ситуация, когда процессору понадобились данные, но в шкафчике их не оказалось, называется **miss** (промах) - тогда приходится идти на склад.
4. Вероятность того, что при обращении к шкафчику произойдет промах, называется **miss rate** (доля промахов) и выражается в процентах.
5. Время, которое дополнительно тратится при промахе на поход на склад, называется **miss penalty** (штраф промаха) и измеряется в наносекундах.

На сегодняшнем занятии Настя говорит: “Представим, что процессор делает 100 обращений к памяти. Каждый раз он сначала заглядывает в шкафчик и тратит hit time.

Если происходит промах, то в дополнение к этому времени он тратит еще и miss penalty на поход на склад.

Среднее время доступа к памяти — это общее время, потраченное на все обращения, деленное на их количество.”

После этого Настя показывает детям несколько разных шкафчиков с разными характеристиками:

Шкафчик	hit time (ns)	miss rate	miss penalty (ns)
A	1	5%	50
B	2	2%	60
C	1	10%	30
D	0.5	8%	40
E	3	1%	80

Задание:

1. Для каждого шкафчика найдите среднее время доступа к памяти.
2. Определите, какой шкафчик самый быстрый в среднем.
3. Укажите его среднее время доступа к памяти.

Формат ответа:

Буква самого быстрого шкафчика и его среднее время доступа к памяти, записанные слитно, без пробелов.

Среднее время указывается в наносекундах (ns), десятичная дробь записывается через точку (например, 1.5). Округлять до одного знака после запятой. Пример записи ответа: «A1.1»

Ответ: B3.2

3. Моделирование состояния (2 балла)

[Жизнь студента]

В течение семестра первокурсник может находиться в одном из состояний первокурсника:

- 0 — «всё хорошо»
- 1 — «всё нормально»
- 2 — «я отчисляюсь»
- 3 — «ладно, передумал»

На состояние влияют события в его жизни. События бывают двух типов:

- В — «сон / культурные мероприятия»

Текущее состояние	Состояние, которое наступит, если случится событие А	Состояние, которое наступит, если случится событие В
0	1	0
1	1	2
2	3	0
3	2	4
4	4	4

Пример, как пользоваться таблицей:

- Если студент был в состоянии 1 и случилось событие В, то по таблице он перейдёт в состояние 2.

Вам дана последовательность событий, происходивших в жизни студента:

Вам нужно записать в ответ два числа через пробел:

2. Сколько раз студент оказывался в состоянии 2 (после очередного события из последовательности).

4. Моделирование на строке (2 балла)

Дана строка АВСССАВВВС. Над ней выполняется следующий алгоритм:

2. Если в строке нечетное число букв А, то в конец строки добавляется символ, которого меньше всего в строке на данный момент. Например, для строки АААВВСС – будет добавлен символ С, получится строка АААВВССС. Если символов, которых в строке меньше всего, несколько (их количества совпадают), тогда в конец строки дописывается символ, идущий в алфавите раньше. Например, для строки АААВВССС будет добавлен символ В, так как он идет в алфавите раньше С – В результате получится строка АААВВСССВ.

Ответ: 2027

5. Моделирование на клетчатом поле (2 балла)

Имеется поле 10x10. У каждой клетки есть координата (x, y), где x – номер строки на поле, y – номер столбца на поле. Левая верхняя клетка имеет координаты (1,1).

Данный алгоритм выглядит следующим образом:

5. Если у шара нет возможности найти пустую клетку, то он доходит до любой угловой клетки (исходные угловые клетки, и любые другие угловые клетки), и удаляется вместе с ней. Это значит, что данная угловая клетка навсегда пропадает с поля, и шары, которые были в ней, соответственно тоже пропадают. Угловой считается клетка, у которой две смежные стороны не граничат с другими клетками.

Определите, сколько шаров на поле останется после выполнения алгоритма балансировки.

55

6. Моделирование на списке (1 балл)

[ПропускИТМО]

В ИТМО запустили внутренний сервис "ПропускИТМО" - через него подают заявки на разовый вход: гости на хакатон, ассистенты на семинар, подрядчики, доставщики и т.п.

Все эти заявки стекаются на пост охраны конкретного корпуса. На посту стоит старенький принтер, который подключён к локальной системе и печатает не по одной заявке, а пакетами, чтобы охранник мог сразу выдать несколько пропусков и отметить людей в журнале. Пакет — это накопленная очередь заявок, которую принтер печатает одним разом (максимум 6 заявок). Очистить пакет — означает удалить из него все заявки, то есть сделать пакет пустым.

На пост охраны корпуса ИТМО за одну смену пришли 24 заявки на печать разовых пропусков в таком порядке:

1. Петров
2. Сорокин
3. Иванченко
4. Егорова
5. Орлова
6. Галка
7. Василенко
8. Панюкова
9. Овсянников
10. Ерёмин
11. Овчинников
12. Губанов
13. Юдина
14. Хачатуров
15. Каймакова
16. Лебедев
17. Миронов
18. Фролов
19. Демидов
20. Романенко
21. Тихонов
22. Чистяков
23. Бутова
24. Назаров

Правила печати:

1. Принтер печатает пакетами максимум по 6 заявок.
2. Заявки обрабатываются поочерёдно (сверху вниз).
3. Множество сигнальных букв: {Е, О, Г}.
4. Если в текущем пакете в какой-то момент встречаются 3 подряд идущие фамилии, которые начинаются на буквы из описанного множества сигнальных букв (например – подряд шли фамилии, начинающиеся на Е, О и Е соответственно), то принтер печатает накопленный пакет сразу, не дожидаясь накопления 6 заявок, после чего очищает его.
5. Если таких фамилий не было, но заявок уже 6, то печатаем пакет и очищаем его.
6. Когда все 24 заявки обработаны и остаются не напечатанные заявки в пакете, пакет печатается, после чего обработка списка завершается.

В ответ укажите одно число — сколько пакетов было напечатано за смену.

Ответ: 5

7. Моделирование системы (3 балла)

[Аква-купол]

Вариант 1

Вам дана экосистема, в состоянии которой происходят циклические изменения. Она обладает следующими параметрами:

- О — кислород в конце итерации.
- А — количество водорослей типа А в конце итерации.
- В — количество водорослей типа В в конце итерации.
- U — количество улиток в конце итерации.
- К — количество креветок в конце итерации.

После каждой итерации мы записываем новое состояние системы.

Состояние системы на каждой итерации меняется по следующим правилам (именно в таком порядке):

1) Водоросли добавляют кислород:

- если $O \leq 25$, то $O_{\text{tmp}} = O + 4 \cdot A + 7 \cdot B$;
- если $O > 25$, то $O_{\text{tmp}} = O + 4 \cdot A$ (водоросли В “не работают”).

O_{tmp} — это сколько кислорода получилось после работы водорослей на данной итерации, до того, как улитки и креветки начали дышать.

2) Улитки и креветки тратят кислород:

$$O_{after} = O_{tmp} - 3 \cdot U - 1 \cdot K.$$

O_{after} — это сколько кислорода осталось в конце итерации после дыхания животных (то есть после того, как улитки и креветки потратили кислород).

3) Проверка условий:

Если $O_{after} < 8$ — креветки погибают, и экосистема не может дальше функционировать.

Если $O_{after} > 40$ — экосистема перенасыщается и не может дальше функционировать.

4) Выбираем ровно одно действие d_i :

- $d_i = 0$ — ничего не делать;
- $d_i = 1$ — посадить 1 водоросль А (А увеличится на 1);
- $d_i = 2$ — посадить 1 водоросль В (В увеличится на 1);
- $d_i = 3$ — убрать 1 улитку (можно только если улиток было хотя бы 1).

В конце итерации получаем следующие значения:

- $O_i = O_{after}$
- $K_i = K$
- $A_i = A + 1$ (если $d_i=1$, иначе А)
- $B_i = B + 1$ (если $d_i=2$, иначе В)
- $U_i = U - 1$ (если $d_i=3$, иначе U)

Вам нужно выбрать действия на каждой итерации.

Выбирайте действия так, чтобы экосистема прожила максимальное количество итераций. Итерация i считается прожитой, даже если после проверки условий на этой итерации экосистема перестаёт функционировать. В ответ запишите максимальное количество итераций, которое сможет прожить экосистема.

Стартовые данные:

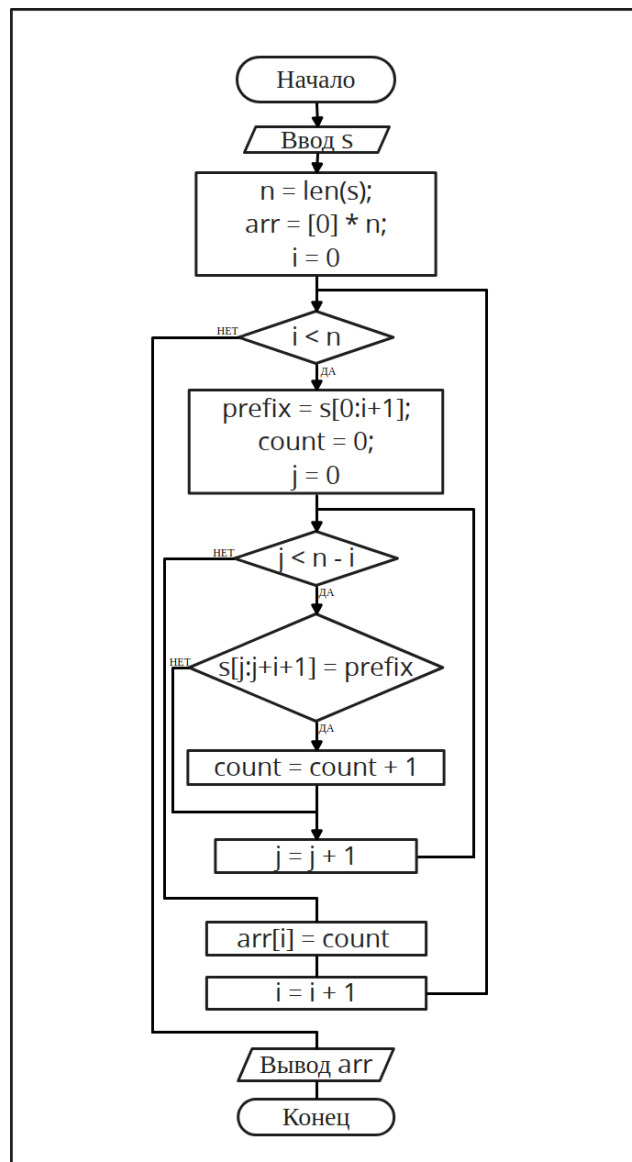
$$O_0=31, A_0=3, B_0=0, U_0=7, K_0=6.$$

Ответ: 3

8. Программирование и алгоритмизация (3 балла)

[Блок-схема]

Дана блок-схема алгоритма:



На вход алгоритму дали следующую строку:

1. Ее длина = 7.
2. Состоит только из символов «a» и «b».
3. Начинается с символа «a».

Нужно выяснить, какую строку подали на вход, если на выходе мы получили следующий массив arr: [3, 2, 2, 2, 1, 1, 1].

Примечание. Обозначения некоторых операций:

- $[k]*n$ – создается массив из n элементов, каждый из которых равен k
- Пример: $[3]*5 = [3, 3, 3, 3, 3]$
- $arr[i:j]$ – берется подпоследовательность с i -ого элемента (включительно) по j -ый (не включительно). Для строк – берется подстрока с i -ого символа (включительно) по j -ый (не включительно).
- $len(s)$ – возвращает длину строки (кол-во символов в строке).

Ответ: abbabba

9. Сортировка и фильтрация данных (2 балла)

[Рейтинг]

В учебной группе М3107 ребята сдают выполненные работы в электронную систему.

Работа каждого студента характеризуется тремя числами:

- Score — базовый балл за работу по шкале от 0 до 100 (чем больше, тем лучше).
Это то, сколько поставил преподаватель за качество решения без штрафов.
- Delay — опоздание в минутах:
если студент сдал вовремя, то Delay = 0,
если сдал на 7 минут позже дедлайна (времени сдачи), то Delay = 7.
- Attempts — количество попыток сдачи (сколько раз отправлял решение).
Если сдал с первого раза — Attempts = 1. Если пересдавал/перезагружал ещё 2 раза — Attempts = 3.

Преподаватель решил составить рейтинг студентов по итогам полугодия на основе результатов работ, загруженных в электронную систему.

Чтобы рейтинг был честнее, преподаватель считает **итоговый балл Final** по следующим правилам:

- за каждую минуту опоздания снимается 2 балла;
- за каждую дополнительную попытку (кроме первой) снимается 5 баллов;
- итоговый балл не может быть меньше 0.

Рейтинг строится по следующим правилам:

1. больше Final — студент выше в рейтинге;
2. если Final одинаковый — выше тот, у кого меньше Delay;
3. если и `Delay` одинаковый — выше тот, у кого меньше Attempts;
4. если всё одинаково — сравниваем ID, выше будет тот, у кого ID меньше.

Вам дан список из 10 студентов. Каждый студент описывается строкой из 4 целых чисел, записанных последовательно через пробел: ID, Score, Delay, Attempts.

ID	Score	Delay	Attempts
1	92	4	1
2	88	2	2
3	80	0	1
4	75	5	1
5	90	7	1
6	84	1	3
7	70	0	1
8	95	10	1
9	86	3	1
10	78	2	1

В ответ запишите первые 5 значений ID в рейтинге (сверху вниз), через пробел.

Ответ: 1 3 9 2 5

10. Сортировка и фильтрация данных (1 балл)

[Тройной просмотр Корнедуда]

Корнедуд – маг-архивариус Гильдии Каталогов. В его архиве заклинания лежат в каталоге: у каждого заклинания есть значение силы. По уставу архива Корнедуд может просматривать только первые три заклинания от начала каталога – это называется "тройной просмотр". Его работа заключается в проведении некоторого ритуала.

Силы заклинаний:

4, 3, 5, 3, 4, 3, 2, 2, 4, 3, 5, 3, 4, 3, 2, 2, 4, 3, 5, 3, 4, 3, 2, 2

Ритуал выполняется 20 раз:

1. Корнедуд смотрит только на первые 3 заклинания.
2. Из этих трех он выбирает самое сильное заклинание. Если максимумов несколько - выбирает то, которое стоит раньше среди этих трех. Его сила = t . Это заклинание вычеркивается из каталога (удаляется).
3. Каталог смещается:
 1. если t четное - циклически сдвигаем вправо на t позиций;
 2. если t нечетное - циклически сдвигаем влево на t позиций.

Циклический сдвиг на t позиций означает, что элементы, выходящие за край каталога, возвращаются с другой стороны, сохраняя порядок. Например, для каталога [1, 2, 3, 4, 5] циклический сдвиг влево на 2 позиции даёт [3, 4, 5, 1, 2], а циклический сдвиг вправо на 2 позиции - [4, 5, 1, 2, 3].

После сдвига началом каталога считается первый элемент получившегося списка.

В ответ следует указать единственное число: какая сила будет у двадцатого выбранного заклинания.

Ответ: 3